

Embedded Assembly for the ATmega328P

Embedded Assembly for the ATmega328P

*From your first ldi to a driver library,
verified in simulation*

Erik Pihl

QRTECH ACADEMY

First edition · version 1.0.0 · September 2026

Embedded Assembly for the ATmega328P

First edition, version 1.0.0, September 2026.

Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). Every line of code in the book and in the companion repository is released under the MIT License. The repository holds the assembly contracts, the test suites and the simulator harness referred to throughout:

<https://github.com/qrtech-academy/embedded-asm>

This edition matches the repository at tag v1.0.0. Later changes to the main branch are not reflected here; clone the tag to get the code and test suites exactly as the book describes them. The worked solutions are not part of the book, apart from the short answers in Appendix A; they are published later, on the main branch, beside the test suites.

Every register address and every cycle count quoted in this book comes from one of two documents, and the text says which: the *ATmega328P datasheet* and the *AVR Instruction Set Manual*, both published by Microchip Technology. ATmega, AVR and Microchip Studio are trademarks of Microchip Technology Inc.

Set in TeX Gyre Pagella and DejaVu Sans Mono with Lua $\text{\LaTeX}$. The figures are drawn from code, and the code that draws them is in the repository beside the text.

Contents

Preface	vii
1 The AVR Core and the Toolchain	1
1.1 The AVR Core	2
1.2 The Toolchain	8
1.3 Counting Cycles.	14
1.4 Your First Subroutine.	17
1.5 Review	21
1.6 Exercises	23
2 I/O Ports and Subroutines	29
2.1 I/O Ports	30
2.2 Subroutines and the Calling Contract	35
2.3 The LED Driver	39
2.4 Putting It on a Real Board	42
2.5 Review	45
2.6 Exercises	46
3 Interrupts	51
3.1 Interrupts and the Vector Table.	52
3.2 The ISR Contract, and Atomicity	59
3.3 The Button Driver.	62
3.4 Review	66
3.5 Exercises	68
4 Pointers, Arrays and Structs in SRAM	73
4.1 The Pointer Registers	74
4.2 Structures, Arrays, and Where They Live	77
4.3 The Stack, and How Deep It Goes	80
4.4 What to Build	83
4.5 Review	85

4.6	Exercises	86
5	Timers	91
5.1	Timers.	92
5.2	The Frequency You Asked For	97
5.3	What to Build	100
5.4	Review	103
5.5	Exercises	104
6	Watchdog, Sleep and the C ABI	109
6.1	The Watchdog	110
6.2	Sleep	114
6.3	The C ABI	118
6.4	What to Build	125
6.5	Review	127
6.6	Exercises	128
A	Short Answers	133

Preface

This book is for embedded engineers who write C or C++ for microcontrollers and have never written assembly, or have read some and never produced any. It takes you from your first `ldi` to a driver library with interrupts, timers and a watchdog, written entirely in AVR assembly and verified by tests that run on your laptop.

The subject is the machine. Assembly is how you get at it; it is not being taught here as a way to write applications, and this book will not tell you it is faster than your compiler. What it is good for is narrower and more durable:

- Reading a disassembly and knowing what you are looking at.
- Writing the handful of things C genuinely cannot express.
- Understanding what your compiler was doing all along.

One body of code is built across the book, and it is yours to write: an AVR assembly driver library, grown one chapter at a time. `led.asm`, `btn.asm`, `led_array.asm`, `timer.asm`, `watchdog.asm` and the utilities under them, plus an application program that composes them.

What you are assumed to know

The subject of this book is **AVR assembly and the machine underneath it**, not embedded programming in general. You are expected to arrive already comfortable with:

- C or C++ for microcontrollers: pointers, structs, bitwise operators, `volatile`.
- Binary and hexadecimal, two's complement, and reading a bit mask such as `(1 << 5)`.
- What a microcontroller peripheral is, and the idea that a register write configures hardware.
- Reading a datasheet section: finding a register, its bit fields, and their meanings.

None of that is taught from scratch. What the book does teach from nothing is the instruction set, the register file, the calling contract, the vector table, and the toolchain: `avra` as an assembler, its listing and map files, and `simavr` as the thing that tells you whether you were right.

No assembly experience of any kind is assumed, on the AVR or anywhere else. If you have written x86 or ARM assembly, Chapter 1 will be short work and the rest will not be: the AVR's 16-bit-pointer-in-three-register-pairs arrangement, its separate program and data address spaces, and its I/O register window are all different enough to be worth reading rather than skimming.

How verification works

Every quantitative claim in this book is checked twice, and the whole method is built on the two not always agreeing:

- **By hand.** You compute the number from the instruction set summary and the datasheet, before any code exists. Nothing automated checks this step, deliberately, and that is the point of it.
- **In the simulator.** The assembly you wrote is assembled, loaded into `simavr` by a harness the companion repository provides, and measured: cycles consumed, pins driven, SRAM written, interrupts taken.

Where the two disagree, the reconciliation is the lesson, and every chapter has one exercise that is exactly this. Your hand count of `shift_bits` and the figure the simulator reports at a call site differ by a fixed amount, and the difference is the `ldi` that set the argument up and the `rcall` that got you there. Your timer's computed period at 1 kHz and its measured period agree, and then at 3 kHz they do not, and the gap is not the timer's error but the two-cycle instruction each interrupt has to wait for.

And once, in Chapter 3, they disagree because the **simulator** is wrong: `simavr` takes an interrupt without charging the four cycles the datasheet says the hardware spends pushing the program counter. Knowing which of your two instruments to believe, and why, is the skill this arrangement exists to build.

No exercise in this book needs hardware. Everything is written, assembled, simulated and verified on an ordinary laptop. If you own an Arduino Uno or Nano with an ATmega328P and want to watch a real LED, §2.4 covers flashing it with `avrdude`; nothing depends on it.

What the exercises look like

Every chapter ends with six to ten exercises, and each one is labelled with its kind:

Kind	What it asks of you
RECALL	State something from the chapter, in your own words. No tools.
HAND CALCULATION	Work a number out on paper, from the datasheet and the instruction set summary.
DESIGN	Decide something before writing it: a register allocation, a struct layout, a prescaler.
CODE	Write assembly. Checked by that chapter's test suite, in the simulator.
CROSS-CHECK	Exactly one per chapter. See below.

The Cross-check is the signature exercise of this book. You compute a number by hand, you run your own code on the same problem, and you reconcile the two. Its solution says how large a discrepancy to expect and where it comes from, because “your two answers differ by 8%” is the lesson, not a sign that you did it wrong. Some of these discrepancies close to zero and some do not, and knowing which is which is most of what separates an engineer who can time a routine from one who assumes the datasheet did it for them.

The worked solutions are not in this book. They will be published in the companion repository, beside the lecture each chapter is typeset from, and until then most of the code exercises need nothing more than the chapter's test suite to tell you whether you are right. The hand calculations have short answers in Appendix A, numbers only, so that you can check one without a laptop. Clone the repository before you start; the book assumes you have it open.

Setting up

Everything runs on WSL or Ubuntu. Clone the companion repository with its two sub-modules, the test framework and the simulator harness, at the tag this edition was written against, and install the toolchain. The tag is v1.0.0: the code and the test suites exactly as this book describes them, whatever the main branch has done since.

```
git clone --recursive --branch v1.0.0 \
    https://github.com/qrtech-academy/embedded-asm.git
sudo apt -y update
sudo apt -y install git make g++ avr binutils-avr simavr \
    libsimavr-dev libelf-dev clang-format
sudo apt -y install gcc-avr avr-libc # Chapter 6 only
avr --version # 1.4.2 or newer
```

Four commands then cover everything the book asks of the repository:

```
make help          # List every target.  
make build         # Assemble the drivers and build the simulator harness.  
make test         # Build and run every chapter's test suite.  
make clean        # Remove everything the build generates.
```

Every one of those runs from the very first commit, when almost nothing exists, and reports what it could not do by name. A check that silently did not run must never read like one that passed, so `make build` on a fresh clone builds the two things that ship, prints a `SKIP` line for the drivers you have not written, and exits 0. That is the truth about a repository where you have not written anything yet.

The AVR Core and the Toolchain

What the machine is, and how to make it run something.

IN THIS CHAPTER

- What kind of machine an ATmega328P is, and which of your C intuitions survive contact with it.
- The 32 registers, and why only half of them can be loaded with a constant.
- SREG: the flags every arithmetic instruction writes and every branch reads.
- Three separate address spaces, and why address 0x0060 names three different bytes.
- What an instruction *is*: `ldi r16, 0x2A` taken apart into the sixteen bits it assembles to.
- The toolchain: `avra` as an assembler, its listing and map files, and a simulator that runs your code and counts its cycles.
- Your first subroutine, `shift_bits`, costed on paper before it is run.
- A program that lights an LED in two instructions, which is as far as one pin gets you.

The chapter is worked in the order a first session at the machine should go. The machine comes first, before any code: registers, flags and the three address spaces, read off the figures in §1.1. Nothing runs yet; the point is to know what you are writing *for*. Then one instruction, taken apart, and the toolchain end to end: one `.asm` file through `avra` to a hex, then the same hex into a disassembler and into a simulator, three views of one artifact. Then counting cycles, and finally writing something, so that the chapter ends with something you can see.

Two predictions are worth making before anything runs:

- what `ldi r16, 0x2A` assembles to, from the encoding alone.
- how many cycles `shift_bits` takes for an argument of 5, from the instruction table alone.

You will check both, and one of them is the subject of this chapter's cross-check exercise.

1.1 The AVR Core

1.1.1 What kind of machine this is

An ATmega328P is an 8-bit microcontroller with 32 registers, 2 KB of RAM, 32 KB of flash, and no operating system, no memory protection, and no cache. It runs one instruction at a time, and almost every instruction takes one clock cycle.

Three of its properties will contradict something you know from C, and it is worth naming them before they surprise you rather than after.

It is a load-store machine. Arithmetic happens between registers and nowhere else. There is no instruction that adds two bytes of memory together; you load one into a register, load the other into another register, add, and store the result back. In C you write `a += b` and the compiler decides how many instructions that is. Here you are the compiler.

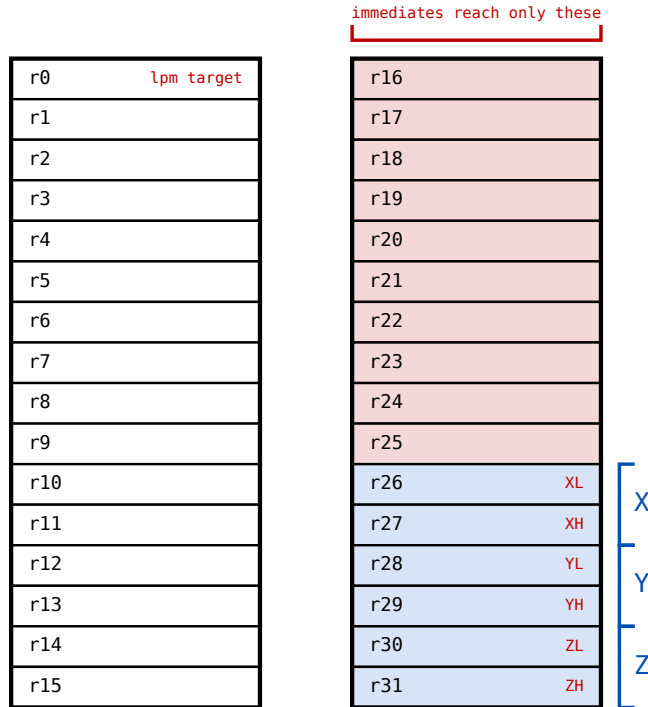
It is a Harvard machine. Program memory and data memory are separate address spaces with separate instructions. A pointer to a constant string in flash and a pointer to a variable in RAM are both 16-bit numbers, and they are not interchangeable, and nothing checks. This is the single biggest departure from the flat address space C leads you to expect (§1.1.4).

It has 32 registers, and they are not all equal. They are equal to most instructions and unequal to a handful of important ones, and the handful is what shapes real AVR code (§1.1.2).

What it is *not* is slow or crude. At 16 MHz it executes 16 million instructions a second, and almost every one of them does something useful. The reason to write assembly here is not that the compiler is bad; it is that on a machine this small you can hold the whole of it in your head, and knowing exactly what a routine costs is worth something.

1.1.2 The register file

Thirty-two 8-bit registers, named `r0` to `r31`. Every one of them can be a source or a destination for `mov`, `add`, `and`, `or`, `ld`, `st` and most of the rest of the instruction set.



All 32 are equal to `ld`, `st`, `mov` and `add`.
Only `ldi`, `andi`, `ori`, `subi` and `cpi` are restricted to the upper half.

Figure 1.1 The 32 AVR registers as two columns of sixteen. The upper half is the half immediate instructions can reach, and r26 to r31 are the pointer pairs X, Y and Z.

Two things about that picture matter more than the other thirty.

Only r16 to r31 can be loaded with a constant. `ldi r16, 0x2A` is legal; `ldi r15, 0x2A` is not, and the assembler will tell you so. The same restriction applies to `andi`, `ori`, `subi`, `cpi` and `sbc i`. The reason is not arbitrary: those instructions spend eight of their sixteen bits on the constant, which leaves four bits for the register number, and four bits can only count to sixteen (§1.1.6).

The practical consequence is that the upper half is where the work happens and the lower half is where you park things. A routine that needs five working registers and reaches for r10 will find it cannot put a constant in it, and will end up loading the constant into r16 and moving it, which costs an extra instruction every time.

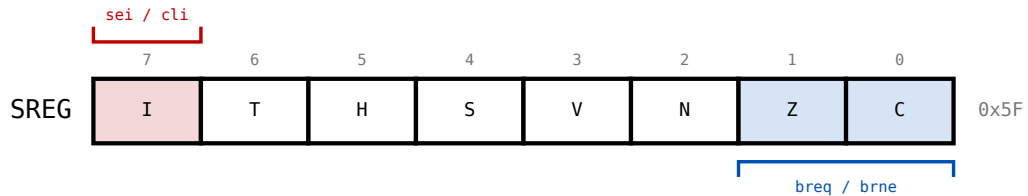
r26 to r31 are also the three pointer registers. Taken in pairs they form the 16-bit pointers X (r27:r26), Y (r29:r28) and Z (r31:r30). They are still ordinary registers; nothing stops you using r30 to hold a loop counter. But the moment you want to read memory through a

pointer, they are the only six that can do it, and a routine that has casually filled them with something else has to spill them first. Chapter 4 is about this in earnest.

`r0` and `r1` deserve a note. `r0` is where `lpm` deposits the byte it read from program memory, so a routine using `lpm` cannot also be keeping something in `r0`. `r1` has no hardware role at all, but the C compiler keeps zero in it permanently, which matters the moment you call C from assembly or the reverse. That is Chapter 6's problem; for now, leaving both alone costs you nothing.

1.1.3 The status register

SREG is one 8-bit register holding the flags that arithmetic writes and branches read.



Set by arithmetic and logic, read by the conditional branches.

Figure 1.2 SREG as its eight bits. Bit 7, I, is the global interrupt enable; bits 1 and 0, Z and C, are the ones the conditional branches read.

Bit	Name	Set when
7	I	Interrupts are globally enabled. Set by <code>sei</code> , cleared by <code>cli</code> .
6	T	A bit copied by <code>bst</code> , to be copied back by <code>bld</code> . Rarely used.
5	H	A carry out of bit 3, for BCD arithmetic. Rarely used.
4	S	N exclusive-or V: the sign of a <i>signed</i> comparison.
3	V	A signed overflow occurred.
2	N	The result's most significant bit was set.
1	Z	The result was zero.
0	C	A carry out of bit 7, or a borrow.

In practice you will use two of them constantly and the rest almost never. Z and C are what `cp`, `cpi` and every arithmetic instruction leave behind, and what the branches test:

Branch	Taken when	Reads
breq	the two values were equal	Z set
brne	they were not equal	Z clear
brlo	the first was lower, unsigned	C set
brsh	the first was the same or higher, unsigned	C clear

cp and cpi are subtractions whose result is thrown away. That is the whole of what a comparison is on this machine: `cp r19, r24` computes $r19 - r24$, discards the difference, and keeps the flags. So `cp` followed by `breq` means “branch if equal”, and `cp` followed by `brlo` means “branch if the first was lower”. If you find yourself unsure which way round a comparison reads, write out the subtraction and the ambiguity disappears.

The flags are the most volatile state on the machine. Nearly every instruction writes some of them, so `cp` and its branch must be adjacent, or nearly so. A `mov` between them is harmless because `mov` writes no flags; an `inc` between them is a bug, and one that shows up as a branch going the wrong way for no visible reason. When you cannot remember whether an instruction touches the flags, the instruction set summary has a column for it, and that column is the reason it exists.

Bit 7, `I`, is different in kind from the other seven. It is not a result of anything; it is a switch, and it controls whether interrupts can happen at all. Chapter 3 is where it matters.

1.1.4 Three address spaces

This is the part that most often goes wrong for a reader arriving from C, so it is worth being slow about.

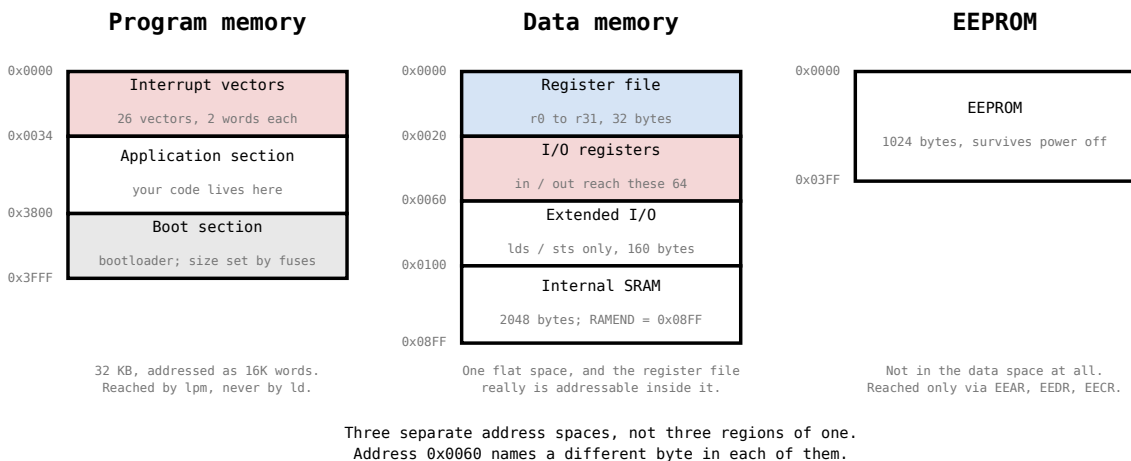


Figure 1.3 Three separate address maps side by side: program memory in word addresses, data memory from the register file up to internal SRAM, and EEPROM on its own.

There is no single address space on this machine. There are three, they overlap numerically, and nothing in the instruction set or the assembler will stop you reading one when you meant another.

Program memory holds your instructions and any constants you put there deliberately. It is addressed in **words**, not bytes, which is why PCINT0's slot is quoted as 0x06 while GNU as wants 0x0C. The entries being 2 apart is a separate fact: each slot is two words, so that a two-word `jmp` fits in one (§3.1.2).

Only `lpm` reads it. An ordinary `ld` pointed at “address 0x0100” reads SRAM, not the code at word 0x0100, and does so silently.

Data memory is one flat space containing four things in order: the register file, the 64 I/O registers, 160 extended I/O registers, and finally 2048 bytes of actual SRAM starting at 0x0100. RAMEND is 0x08FF, and it is where you point the stack (§1.4.6).

The register file really is addressable there. `lds r16, 0x0005` loads the contents of r5. This is true, occasionally useful for a debugger, and something no ordinary program does, because `mov r16, r5` does the same thing in one cycle instead of two.

EEPROM is 1024 bytes of memory that survives power being removed. It has its own address space and **no load or store instruction reaches it at all**; you write an address into EEAR, a byte into EEDR, and set a bit in EECR. This book does not use it, but knowing it is a third space rather than a region of the second is the point.

1.1.5 The I/O window: two names for one register

Every I/O register has two addresses, and both appear in the datasheet.

PORTB is I/O address 0x05 and data space address 0x25. They differ by 0x20, because the I/O registers begin 0x20 bytes into the data space. Which number you use depends entirely on which instruction you are writing:

Instruction	Takes	Reaches	Cost
<code>in / out</code>	an I/O address, 0 to 63	the low 64 I/O registers	1 cycle
<code>sbi / cbi</code>	an I/O address, 0 to 31	the low 32, one bit at a time	2 cycles
<code>lds / sts</code>	a data space address	anywhere in data memory	2 cycles
<code>ld / st</code>	a pointer register	anywhere in data memory	2 cycles

Giving one of them the other's number is the classic first AVR bug. `out 0x25, r16` writes to I/O register 0x25, which is data space 0x45, which is TCCR0B: Timer0's clock select and its waveform mode. Your LED does nothing, your program looks correct, and somewhere else a timer you have not written yet is running at a rate you did not choose.

In this book you will rarely write either number, because `.include "m328Pdef.inc"` defines PORTB for you, and it defines it as the I/O address, 0x05. So `out PORTB, r16` is right, and reaching the same register with `sts` means writing the 0x20 yourself:


```
out PORTB, r16           ; the I/O address, 0x05
sts PORTB + 0x20, r16     ; the same register through the data space, 0x25
```

Not every register works that way, and the device file tells you which. WDTCSR is 0x60 and TCCR1B is 0x81, data space addresses already, because those registers live above the I/O window and out cannot reach them at all. m328Pdef.inc marks them ; MEMORY MAPPED, and that comment is the whole rule: if it is there, the number is a data space address and you add nothing; if it is not, the number is an I/O address and sts needs + 0x20.

The mistake this invites is worse than a wrong register. Write sts PORTB, r16 and it assembles without complaint to data space 0x0005, which is not a peripheral at all, but the general-purpose register r5. Your port never changes, a register you were using quietly takes the value instead, and nothing anywhere reports a problem. Assembly written for avr-gcc does exactly this, because there PORTB already means 0x25; carrying such a line across unchanged is the single most productive source of this bug. §1.2.7 has the rest of that comparison.

1.1.6 What an instruction is

Every AVR instruction is one or two 16-bit words. ldi is one word, and every bit of it is accounted for.

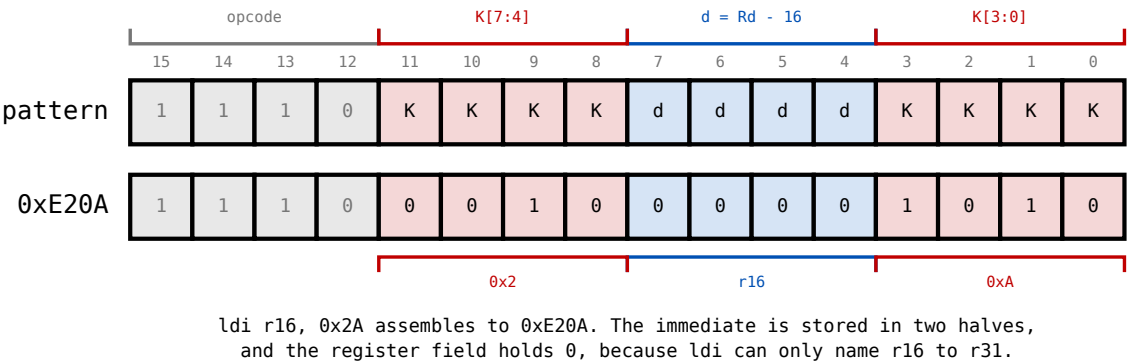


Figure 1.4 ldi r16, 0x2A as sixteen bit cells: the pattern 1110 KKKK dddd KKKK above the assembled word, with the opcode, the two halves of the immediate and the register field braced separately.

$$\text{ldi } R_d, K \rightarrow 1110 K_{7:4} d_{3:0} K_{3:0} \quad \text{where } d = R_d - 16$$

Three things fall out of that layout, and all three are facts about the machine rather than about notation.

The immediate is split in half. Bits 11 to 8 carry the top nibble and bits 3 to 0 the bottom one, with the register number in between. There is no reason for this beyond the encoding being easier to decode in hardware that way, and it is the detail that makes hand-assembling an `ldi` feel like a puzzle the first time.

The register field is four bits, so it counts 0 to 15, and the machine adds 16. That is the whole explanation for §1.1.2's restriction: `ldi` cannot name `r15` because there is nowhere in the instruction to put the number 15 and mean the sixteenth register.

The opcode is only four bits. `1110` means `ldi` and nothing else, which is a lot of encoding space for one instruction, and is why the immediate can afford to be a full byte.

So `ldi r16, 0x2A` is `1110 0010 0000 1010 = 0xE20A`. You will confirm that number three ways in this chapter: by hand, in the listing `avra` writes beside your source, and by looking at what `avr-objdump` says the assembler produced (§1.2.3).

1.1.7 What this machine does not have

Naming the absences early saves you looking for them later.

No barrel shifter. `lsl r18` shifts by exactly one bit. There is no instruction that shifts by a variable number of places, so `1 << n` for a runtime `n` is a *loop*, and it costs about six cycles per bit. That is the whole reason `shift_bits` (§1.4.3) exists, and it is worth sitting with: an operation that is free in C is a loop here, and a driver that calls it with a pin number costs different amounts for different pins.

No division. There is a hardware multiply (`mul`, two cycles), but division is a library routine, and a slow one. Powers of two are shifts; everything else is expensive.

No stack frame, and no stack pointer discipline. There is a stack pointer, `SP`, and `rcall` and `ret` use it, and that is the entire mechanism. Local variables, parameter passing beyond a few registers, and reentrancy are conventions you follow or fail to follow. Nothing enforces them. Chapter 2 is about the convention; Chapter 4 is about the stack itself.

No protection of any kind. Writing past the end of an array walks into whatever is next. Pointing `Z` at `0x0000` and storing writes to `r0`. Recursing too deep grows the stack down into your variables and there is no fault, no message, and no crash, just wrong answers.

And no nop-free timing. There is no cache, no branch predictor and no pipeline stall you have to reason about, which is the good news hiding inside all of the above: an instruction's cost is a number in a table, the same number every time. That is what makes §1.3 possible, and it is a property most machines lost decades ago.

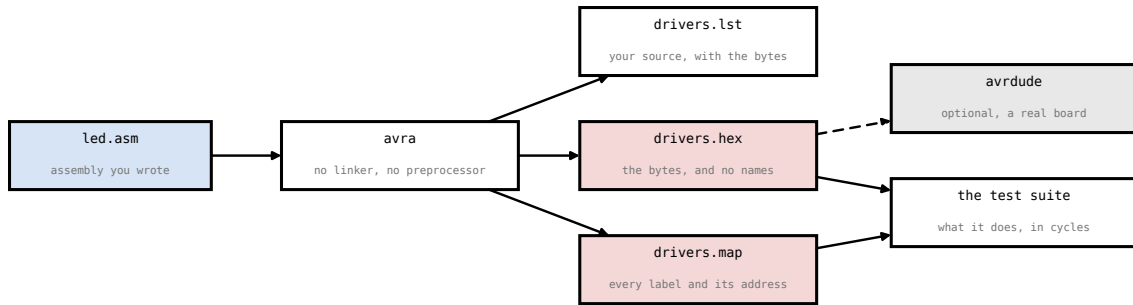
1.2 The Toolchain

1.2.1 What assembling actually does

An assembler is the least clever program in your toolchain. It reads one line at a time, turns each instruction into the one or two words that encode it (§1.1.6), resolves the labels, and writes the result out. It does not reorder anything, it does not allocate registers, and it does

not check that what you wrote makes sense. If you write `ldi r16, 0x2A`, you get `0xE20A`, and if you write a loop with no exit you get a loop with no exit.

That is the whole appeal. Everything the machine does is something you wrote.



A hex file carries addresses and bytes and nothing else, so the harness reads the map beside it. That pair is what lets a test call `led_init` by name, set `r25:r24` and `r22`, run it, and read `DDRB` afterwards.

Figure 1.5 The toolchain: `led.asm` through `avra` into a listing, a hex and a map. The hex and the map together feed the test suite; the hex alone feeds `avrdude`.

1.2.2 avra as an assembler

This book assembles with `avra`, which takes AVRASM2, the syntax Microchip Studio assembles, and never uses the C compiler except in Chapter 6:

```

avra -I /usr/share/avra -I drivers/include -D F_CPU=16000000 \
-l drivers/build/drivers.lst -m drivers/build/drivers.map \
-o drivers/build/drivers.hex drivers/build/drivers_unit.asm

```

Each of those is load-bearing:

Flag	Why
<code>-I /usr/share/avra</code>	Where <code>m328Pdef.inc</code> lives, which is what gives you <code>PORTB</code> and <code>RAMEND</code> .
<code>-I drivers/include</code>	Where the course's own <code>.inc</code> contracts live.
<code>-D F_CPU=16000000</code>	The clock, as a symbol your own source may use. Nothing in the book requires it; Chapter 5's arithmetic is easier to write against a name than a literal.
<code>-l ...lst</code>	The listing: your source with the encoding beside every line. §1.2.3.
<code>-m ...map</code>	The map: every label and its address. The tests read this.
<code>-o ...hex</code>	Intel hex, which is what the simulator loads.

You will rarely type that, because `make build` does it. Two of the values are overridable from the environment for the same reason: `F_CPU=8000000 make build` assembles against a

different clock, and `AVRA_INCLUDE=...` `make build` is what a non-apt install of `avra` needs, because its device files will not be in `/usr/share/avra`.

The device is chosen by the file you include, not by a flag. There is no `-mmcu` here; `.include "m328Pdef.inc"` is the whole of that decision, and it is why the line belongs at the top of every source file you write.

avra has no linker. It assembles exactly one file, so `ci/build.sh` writes a top-level unit that `.includes` every source in `drivers/source` and assembles that instead. You never edit it. When you want to know what was assembled and in what order, read `drivers/build/drivers_unit.asm`; it is one line per source plus two, and generated fresh on every build.

Two conventions the files in this book follow:

- **;** starts a comment, to the end of the line.
- **Every label is global already.** `avra` exports all of them, so there is no `.global` to write and no way to forget it. A test that cannot find your subroutine is telling you about a spelling or a file that did not assemble, never about a missing directive.

1.2.3 Reading a disassembly

The listing file is the assembler telling you what it produced, line by line, against the source you gave it. `make build` writes one every time:

```
less drivers/build/drivers.lst
```

```
C:000000 01fc      movw r30, r24          ; Z = the struct
C:000001 e203      ldi  r16, low(PINB + 0x20)
C:000002 8300      std  Z + LED_PIN_REG, r16
```

The first column is the **word** address, the second is the encoding, and the rest is your own line, comment included. That is the source you wrote, the bits it became, and the address each landed at, all on one line.

To disassemble instead, starting from the machine code and recovering the instructions, which is the skill this book is actually after, point `avr-objdump` at the hex:

```
avr-objdump -D -m avr:5 -b ihex drivers/build/drivers.hex
```

`-b ihex` is needed because a hex file says nothing about what architecture it is for, and `-m avr:5` supplies the answer. **The symbol names are gone**, and the output says `.sec1` where an ELF would have said `<led_init>`; a hex file carries addresses and bytes and nothing else, which is exactly why the tests read the map file alongside it.

Disassembly of section .sec1:

```
00000000 <.sec1>:
0: 21 e0      ldi    r18, 0x01      ; 1
2: 30 e0      ldi    r19, 0x00      ; 0
4: 38 17      cp     r19, r24
6: 19 f0      breq   .+6              ; 0xe
8: 22 0f      add    r18, r18
a: 33 95      inc    r19
c: fb cf      rjmp   .-10         ; 0x4
e: 82 2f      mov    r24, r18
10: 08 95      ret
```

Four columns: the byte address, the encoded words, the instruction, and a comment. Four things in that listing are worth noticing on your first read.

There is one block and it is called .sec1. Not `shift_bits`, not `shift_bits_loop`: a hex file has no symbol table, so `objdump` invents one section name for the whole run of bytes and labels every branch target with a bare address. The names are in `drivers.map`, which is the other file `avra` wrote and the one the tests read. Compare this against the listing above, where your own label sits on the line that produced it. The listing knows the names because it is the assembler talking, and the disassembly does not because it is only ever given bytes.

The bytes are little-endian. `ldi r18, 0x01` shows as `21 e0`, and the word is `0xE021`. Hold that against §1.1.6: opcode `1110`, `K[7:4] = 0000`, `d = 0010` (so `r18`), `K[3:0] = 0001`. It checks out, and reading it in the wrong byte order is a small rite of passage.

Addresses here are bytes, not words. The `rjmp` at the bottom goes back to `0x04`, which is word `0x02` and so the *third* instruction: words `0x00` and `0x01` hold the two `ldis` above it. The vector table is quoted in words and the disassembly in bytes, and keeping the two apart is a recurring nuisance that this section cannot make go away.

`lsl r18` came back as `add r18, r18`. They are the same instruction: shifting left by one and adding a number to itself do the same thing to the bits and set the same flags, so the assembler encodes `lsl` as `add` and the disassembler has no way to know which you wrote. Several AVR mnemonics are aliases like this, and a disassembly that does not match your source line for line is usually one of them rather than a mistake.

1.2.4 Two images, for two jobs

`make build` produces two, and confusing them wastes an afternoon.

`drivers/build/drivers.hex` is the subroutines alone. It has no reset vector and no vector table; nothing ever runs it from the start. The unit tests set the program counter straight to a symbol and call one subroutine at a time. This is the image nearly everything in this book loads, and it is the one `make measure` uses by default.

drivers/build/app.hex is **drivers/app/main.asm** assembled together with those same subroutines: a whole program with a vector table and a main loop. This chapter's program test and Chapter 3's integration test load this one and let it run, because a program cannot be called and an interrupt has to arrive. Reach it with `make measure IMAGE=app`.

Each comes with a `.map` beside it, and the two are read together: the hex is the program and the map is the names. Delete one and the harness reports the other missing.

1.2.5 Running the tests

From the repository root:

```
make test
```

Every chapter's suite is cumulative, so this runs Chapter 1's now and will still be running it in Chapter 5. What is in a suite decides for itself whether to run: the tests that check the pinned device constants always run, and the ones that call your assembly are switched on by their `.asm` file existing. Nothing has to be registered anywhere; the README beside the suite, in `lectures/L01/exercises/test`, has the details.

Two failures worth recognising before you meet them:

- **Utils.SubroutinesAreDefined fails and the file is plainly there.** The label is spelled differently from the name the test asks for, or the file did not assemble and the build said so further up. `avra` needs no `.global`, so it is never that.
- **A cycle count comes back as 100001.** That is the simulator's budget running out, which is what an infinite loop looks like from outside.

1.2.6 Measuring a subroutine

A test tells you pass or fail. A cross-check needs the number itself, so the companion repository ships a tool:

```
make measure SYMBOL=shift_bits ARG=5
```

```
shift_bits(r25:r24 = 0x0005, r22 = 0)
cycles      40
time        2.500 us at 16.0 MHz
returned    r24 = 32 (0x20)
```

```
deepest stack: SP reached 0x08FD, 2 bytes below RAMEND
```

It assembles whatever is in `drivers/source`, loads it, sets `r25:r24` and `r22` to the arguments you gave, calls the symbol, and reports what came back and what it cost. `IMAGE=app`

measures against the whole program instead of the library, which is what an interrupt handler needs (§1.2.4).

The figure excludes the `rcall` that would have got you there, because there was no `rcall`: the tool sets the program counter directly, the way the tests do. That is not a defect and it is not hidden from you; it is the first term of the discrepancy that Exercise 1.8, the cross-check, asks you to account for.

1.2.7 Opening this book's code in Microchip Studio

Everything in `drivers/source` is AVRASM2, which is what Microchip Studio's assembler takes. A file from this book opens there and single-steps without being translated first, and that is the reason for the choice: the Processor Status window showing R16 change, and the I/O view letting you tick a bit of PINB to fake a button press, apply to **your** code rather than to a port of it.

Nothing in this book requires Studio, and no exercise depends on it. It is Windows-only, and `make test` measures everything Studio would show you and rather more besides. But if you have it, this is how it fits:

1. **File** → **New** → **Project** → **AVR Assembler Project**, targeting the ATmega328P.
2. Replace the generated `.asm` with one of yours, or add yours to the project.
3. **Debug** → **Start Debugging and Break**, and pick **Simulator** as the tool when asked.
4. **Debug** → **Windows** → **Processor Status** for the register file, and **I/O** for the peripherals.

Two differences from the build here, neither of which is a syntax question:

- **Studio assembles one file and this repository assembles all of them.** `ci/build.sh` generates a unit that includes every source (§1.2.2); a Studio project has whatever you put in it. A subroutine that assembles here and is missing there is usually a file you did not add.
- **m328Pdef.inc comes from a different place.** Studio ships its own copy and finds it automatically; `avra` uses the one in `/usr/share/avra`. They agree about the ATmega328P, which is the only device this book targets.

The vector table, in the units the datasheet uses

`.org` counts **words** in AVRASM2, and so does the datasheet's vector table. `.org 0x0006` is PCINT0 in both, and no conversion stands between them. Better still, `m328Pdef.inc` names them:

```
.org PCI0addr
rjmp isr_pcint0
```

`PCI0addr`, `WDTaddr`, `OC1Aaddr` and the rest are defined in the device file, so the magic number need not appear in your program at all. Chapter 3 is where this matters.

Reading GNU as code elsewhere? Most AVR assembly published online is written for `avr-gcc`, where `.equ` takes a comma, register names come from `<avr/io.h>`, `PORTB` means the data space address `0x25` rather than the I/O address `0x05`, and `.org` counts bytes, so the same vector table is written `.org 0x000C`. §6.3.3 shows that dialect properly, because Chapter 6 is the one chapter where this book links against a C compiler.

1.3 Counting Cycles

1.3.1 Why bother

On most machines you cannot say what a routine costs. Caches, branch prediction and out-of-order execution mean the same code takes different times on different runs, and the honest answer to “how long does this take” is “measure it, repeatedly, and take a distribution”.

The ATmega328P has none of those. Every instruction takes a fixed number of cycles, the number is in a table, and it is the same number every time. So “how long does this take” has an exact answer you can work out on paper before the code exists, and that changes what is worth doing:

- You can decide between two implementations without writing either.
- You can tell whether a delay loop will meet its deadline without a scope.
- When a measurement disagrees with your count, one of you is wrong about the code, and finding out which is a productive afternoon rather than a guess about the hardware.

This section is that table, and the arithmetic on top of it.

1.3.2 What each instruction costs

Every figure here comes from the AVR Instruction Set Manual for this core, and every one was confirmed by running that instruction in the simulator and subtracting the `ret` around it. Where the two had disagreed, the disagreement would be written down here. They did not.

Cycles	Instructions
1	<code>ldi</code> <code>mov</code> <code>movw</code> <code>add</code> <code>adc</code> <code>sub</code> <code>subi</code> <code>and</code> <code>andi</code> <code>or</code> <code>ori</code> <code>eor</code> <code>com</code> <code>neg</code> <code>cp</code> <code>cpc</code> <code>cpi</code> <code>lsl</code> <code>lsr</code> <code>rol</code> <code>ror</code> <code>asr</code> <code>inc</code> <code>dec</code> <code>nop</code> <code>in</code> <code>out</code> <code>sei</code> <code>cli</code>
2	<code>rjmp</code> <code>lds</code> <code>sts</code> <code>ld</code> <code>ldd</code> <code>st</code> <code>std</code> <code>push</code> <code>pop</code> <code>adiw</code> <code>sbiw</code> <code>sbi</code> <code>cbi</code>
3	<code>rcall</code> <code>lpm</code>
4	<code>ret</code> <code>reti</code>
1 or 2	<code>breq</code> <code>brne</code> <code>brlo</code> <code>brsh</code> <code>brcc</code> <code>brcs</code> and the other conditional branches

Three rows in that table are worth more attention than the rest.

`ret` costs four cycles and `rcall` costs three. Calling a subroutine that does nothing at all costs seven cycles, which is more than most loops’ bodies. That is not an argument against

subroutines; it is the reason a driver's cost is never just the cost of its body, and the reason the cross-check in Exercise 1.8 comes out the way it does.

sbi and cbi cost two cycles, not one. They look like single-bit versions of out, and they are not: they read the register, modify one bit, and write it back. Worth remembering when you are counting a routine that sets several bits one at a time and wondering where the cycles went.

A memory access costs twice what a register operation does. lds is two cycles and mov is one, which is the whole argument for keeping working values in registers. On a machine with 32 of them that is usually possible, and arranging for it is most of what makes assembly fast.

1.3.3 A branch costs more when it branches

A conditional branch takes **one cycle if it falls through and two if it is taken**. The reason is that a taken branch changes the program counter, and the instruction already being fetched has to be thrown away.

This is the single most common source of an off-by-a-few cycle count, because it means a loop's cost is not the number of iterations times the body.

Take a loop with its test at the top, which is the shape the subroutine in §1.4 needs: compare a running count against the argument, branch out of the loop when they are equal, do the work, add one to the count, and jump back to the top.

Step	Cycles	Times it runs
compare the count against the argument	1	$n + 1$
branch out when they are equal	1 falling through, 2 taken	$n + 1$
the work, one instruction here	1	n
add one to the count	1	n
jump back to the top	2	n

A loop running n times executes the branch $n + 1$ **times**: n of them fall through into the body, and one is taken to leave. So the branch contributes $n \times 1 + 1 \times 2 = n + 2$ cycles, not $n + 1$ and not $2n$.

Everything else in the loop runs exactly n times, except the comparison, which also runs on the final pass that leaves the loop and so runs $n + 1$ times as well. That extra comparison is the one people forget, and it is why the answer comes out ten rather than nine when n is zero.

1.3.4 Counting a routine on paper

The table in §1.3.3 is the method, and it is worth stating as a method because you will use it in every chapter of this book.

One row per instruction, not per line executed. Write the instruction, its cost from §1.3.2, and how many times it runs. Multiply and sum. A branch gets two rows, because it costs differently on the passes that fall through and on the one that is taken.

For `shift_bits` with an argument of n :

Instruction	Cycles	Times	Total
<code>ldi</code> $\times 2$	1	2	2
<code>cp</code>	1	$n + 1$	$n + 1$
<code>breq</code> , falling through	1	n	n
<code>breq</code> , taken	2	1	2
<code>lsl</code>	1	n	n
<code>inc</code>	1	n	n
<code>rjmp</code>	2	n	$2n$
<code>mov</code>	1	1	1
<code>ret</code>	4	1	4

which adds to $6n + 10$.

Order does not matter and that is not a simplification. Cost is a sum, so a table that tracked which instruction ran when would be carrying weight for nothing. What the method does require is that you have already worked out **how many times each line runs**, and that is the part that is actually hard. §1.3.3 is about exactly that, and the $n + 1$ on the comparison is where most wrong answers come from.

1.3.5 What a hand count is blind to

It is worth being explicit, because a method that quietly stops being right is worse than no method.

It does not know about the `rcall` that reached the routine. Your table is the body. The three cycles of getting there, and the instructions that set up the arguments, belong to the caller, and counting them is the caller's table's job.

It does not know about interrupts. An interrupt arriving mid-routine adds the handler's cost plus six to nine cycles of entry and four of `reti`, at a moment nothing in this model can predict. From Chapter 3 onwards that is a real effect and a hand count will simply be wrong about it, by an amount that depends on what else is running. Where that matters, the answer is to measure.

It does not know whether your table matches your code. Nothing checks that the rows you wrote describe the routine you assembled. Two things guard against that: the simulator, which costs the real thing, and the discipline of writing the table from the source rather than from memory. When they disagree, the table is usually the one that is wrong, because it was written by a human who was thinking about something else.

It says nothing about correctness. A routine can cost exactly what you predicted and return entirely the wrong answer. Cycle counting tells you whether something fits in the time available, and nothing whatever about whether it should be running at all.

1.4 Your First Subroutine

1.4.1 The task

Write two subroutines in `drivers/source/utils.asm`, and one program in `drivers/app/main.asm`.

The subroutines compute $1 \ll n$ and $\sim(1 \ll n)$ for a value of n that is not known until the program runs. In C that is one operator each and you would not give it a second thought. Here it is a loop, because the AVR has no barrel shifter (§1.1.7): `lsl` shifts by exactly one place, and shifting by n places means going round n times.

That is not a curiosity. Every driver in this book is handed a pin number and has to turn it into a bit mask, and these two subroutines are how. `led_on` in Chapter 2 calls `shift_bits`; `btn_init` in Chapter 3 calls both. Getting them right now means the rest of the book is about peripherals rather than about bit twiddling.

Creating the file is all it takes to switch its tests on; the README beside the suite, in `lectures/L01/exercises/test`, says how.

1.4.2 Where the arguments live

This book follows the AVR C calling convention from the first line of assembly it writes, and keeps following it, so that Chapter 6’s “call your assembly from C” is a demonstration rather than a rewrite. The whole of what you need for now:

Register	Role
r24	First argument, if it is a byte. Also where a byte-sized return value goes.
r25:r24	First argument, if it is a pointer or another 16-bit value.
r22	Second argument, if it is a byte.
r18 to r27, r30, r31	Call-clobbered. A subroutine may destroy these freely.
r2 to r17, r28, r29	Call-saved. A subroutine that uses one must put it back.

So both subroutines here take their argument in r24 and leave their result in r24, and both may use r18 and r19 as scratch without saving anything.

1.4.3 `shift_bits`

Returns 1 shifted left by the number of places given.

Given r24	Returns r24	Because
0	0x01	Shifted no times at all.
1	0x02	
5	0x20	Which is bit 5, the Arduino Uno’s pin 13.
7	0x80	
8	0x00	The bit has been shifted off the end of an eight-bit register.

Write it like this:

1. Put 0x01 in a scratch register; this is the value being shifted. Put 0 in a second scratch register; this is the count of shifts done so far.
2. At the top of the loop, compare the count against r24. If they are equal, leave the loop.
3. Otherwise shift the value left once, add one to the count, and jump back to the top.
4. On leaving, move the value into r24 and return.

Two things about that shape are worth being deliberate about, because they are what the cost depends on (§1.3.3):

The comparison is at the top, not the bottom. So an argument of zero does no shifts at all, which is what makes `shift_bits(0)` return 0x01 rather than 0x02. A loop with the test at the bottom always runs its body once, and would be wrong for exactly one input, which is the input your driver will use for pin 8.

The exit branch runs once more than the body does. That last comparison, the one that finds the count has reached r24 and leaves, is a real instruction with a real cost. It is the reason the answer is 10 cycles at $n = 0$ and not 7.

The last row of the table is the interesting one. Nothing in the routine says “stop at eight”; the zero falls out of the register being eight bits wide, and the bit simply leaves. A reader thinking in C, where $1 \ll 8$ is 256, will predict 0x100 and there is nowhere to put it.

1.4.4 `shift_bits_inverted`

Returns the complement of the same value: $\sim(1 \ll n)$.

Given r24	Returns r24
0	0xFE
1	0xFD
5	0xDF
7	0x7F
8	0xFF

The obvious implementation is to call `shift_bits` and complement the result with `com`. Do not; write it as its own loop. Start the scratch value at 0xFE instead of 0x01, and each time round shift left *and then increment*, which shifts a zero in at the bottom and turns it back into a one.

The reason for the detour is that this routine exists to produce a mask for clearing a bit, and Chapter 2’s driver calls it directly. Writing it as its own loop also makes the point that §1.3.3 is really making: two routines differing by a single `inc` differ in cost by that `inc` on every trip, and you can say by how much before running either.

1.4.5 Why r18 and r19

Use r18 and r19 as the scratch registers, not r16 and r17.

Both pairs are in the upper half, so both can take an `ldi` (§1.1.2). The difference is the calling convention in §1.4.2: `r18` and `r19` are call-clobbered, and `r16` and `r17` are call-saved. A subroutine that writes to `r16` without restoring it has broken its contract with whoever called it, and the caller has no way to know.

Right now nothing calls these subroutines but a test, and a test does not care. By Chapter 6 the caller may be a C function that had a live value in `r16`, and then the bug is a variable that changes on its own across a function call, which is about as unpleasant a thing to debug as this book contains. Choosing the right registers now costs nothing and means that never happens.

If you are comparing against AVR assembly you have found elsewhere, note that a good deal of it uses `r16` and `r17` here. It works, right up until it does not.

1.4.6 The program

`drivers/app/main.asm` is the first thing in this book that is a program rather than a subroutine. For this chapter it needs four things:

A reset vector. The first word of program memory is where the machine starts, so the first instruction in your file has to be a jump to your code. Use `.org 0` followed by an `rjmp`.

A stack pointer. `rcall` pushes a return address and `ret` pops one, and both use `SP`. Set it to `RAMEND` by writing `high(RAMEND)` to `SPH` and then `low(RAMEND)` to `SPL`, with out rather than `sts` (§1.1.5). `low()` and `high()` are AVRASM2's; GNU as spells the same two functions `lo8()` and `hi8()`, which matters once in Chapter 6 and nowhere else.

The hardware already does this on the ATmega328P, and it does it after **every** reset, watchdog resets included. You will find a good deal of AVR code carrying a comment saying otherwise; on this device that comment is wrong, and repeating it is how the belief survives.

Write the four instructions anyway, for the two reasons that are actually true. Older AVR's reset `SP` to `0x0000` rather than to `RAMEND`, so code that omits this is not portable to them, and the failure there is a stack that grows down out of memory on the first `rcall`. And a reset is not the only way to arrive at your code: a bootloader that jumps into it has performed no reset at all, and `SP` holds whatever the bootloader left in it.

Neither of those applies to anything in this book. Knowing *which* reason applies, rather than carrying a habit you cannot justify, is the point.

Something to do, and somewhere to stop. Call `shift_bits` with an argument, then sit in a loop that jumps to itself. There is no operating system to return to; a program that runs off the end of its own code executes whatever is in flash after it, which is `0xFF` repeated, and the results are not interesting. An infinite loop is how an embedded program ends.

And something you can see. Light the LED on `PB5`.

```
sbi DDRB, 5      ; PB5 is an output
sbi PORTB, 5     ; and it is high
```

That is the whole of it, and every part of it is already in this chapter. `DDRB` is `0x04` and `PORTB` is `0x05`, both I/O addresses, so `sbi` reaches them and `sts` would not (§1.1.5); `sbi` costs two cycles and not one, which §1.3.2 says and which is worth noticing now rather than in Chapter 5 when you are counting a timer’s handler.

The one thing that is new is what the two registers mean, and it is one sentence: **DDRx decides whether a pin is an output, and PORTx decides what an output drives**. Both are per-bit, both default to zero, and zero means “input, not driven”. So an LED needs both writes and in that order: setting `PORTB` first would briefly enable a pull-up on an input pin instead, which is a different thing that happens to look similar.

PB5 is Arduino pin 13, which is the one with an LED already fitted to the board, and the reason this book lights that one rather than any other. On a real Uno there is a resistor beside it; in the simulator there is nothing to protect, and §2.1.4 is where that stops being a detail.

Two instructions, one pin, hard-coded. That is deliberate, and it is exactly as far as this chapter goes. It only works for PB5, it cannot be told which LED to light, and nothing else in the program can reuse it. Turning it into something a program can call with a pin number is Chapter 2, and so is the reason that turns out to be more work than it sounds.

1.4.7 How to check it

Three things, in this order.

Does it assemble?

```
make build
```

Does it do the right thing, and cost the right amount?

```
make test
```

Nine tests were passing before you wrote anything, eighteen once `utils.asm` exists, and twenty once `main.asm` does. If `Utils.SubroutinesAreDefined` fails while the file is plainly there, the label is spelled differently from the name the test asks for, or the file did not assemble (§1.2.2).

The tests for `main.asm` work differently from the rest, and it is worth knowing why before one of them fails. A subroutine can be called: a test sets the program counter to `shift_bits`, puts a number in `r24` and reads `r24` back. A program cannot. It has no arguments, it never returns, and the only way to find out what it does is to **let it run** and then look at the machine:

```
mcu.run(2000);           // 2000 cycles of your program
mcu.data(0x24) & (1 << 5) // DDRB, in the data space
```

Note the `0x24`. The test reads the **data space** address where your program wrote the I/O address `0x04`, and both are correct, because they are two names for one register. That is §1.1.5 again, seen from the third side, and it is the last time this book will point it out.

What does it actually cost?

```
make measure SYMBOL=shift_bits ARG=5
```

Do not run that until you have predicted the answer. Predicting it by hand, and then reconciling the prediction against the measurement, is the cross-check in Exercise 1.8, and it is the exercise this whole chapter exists to set up.

1.5 Review

What you should be able to do now

- Say what the 32 registers are, which sixteen an immediate instruction can name, and why the other sixteen cannot be named that way.
- Read a line of SREG's contents and say which branch would be taken next.
- Given an address, say which of the three address spaces it belongs to and which instruction reaches it; and say why `in`, `lds` and `lpm` are three different instructions rather than one.
- Encode an `ldi` by hand and confirm it against a disassembly.
- Assemble a `.asm` file, disassemble the result, and run it in the simulator.
- Count the cycles of a straight-line routine and of a loop, from the instruction set summary, and say why a measured figure at a call site comes out higher than your count of the body.
- Write a subroutine that takes an argument in `r24`, returns a value in `r24`, and returns.
- Write a program that runs on its own: a reset vector, a stack pointer, an LED lit through `DDRB` and `PORTB`, and a loop to stop in.

Questions to test yourself

1. Why can `ldi` name `r16` but not `r15`, and what does that cost you when you run out of registers in the upper half?
2. Data space address `0x0025` and I/O address `0x05` are the same register. Which instructions take which of the two numbers, and what happens if you give one the other's?
3. The register file is addressable as data at `0x0000` to `0x001F`. What could you do with that, and why does essentially no program do it?
4. A conditional branch costs one cycle when it falls through and two when it branches. A loop running n times executes its exit branch $n + 1$ times. How many of those are taken?
5. Your hand count of a subroutine says 40 cycles and the simulator agrees, yet the *call* costs 44 and the loop around it costs 46 per iteration. Where are the other four, and the two after that, without looking anything up?

6. Why does `shift_bits(7)` cost more than five times what `shift_bits(0)` costs, and what does that imply about a driver that calls it with a pin number?
7. Your program writes `DDRB` at I/O address `0x04`, and the test that checks it reads data space address `0x24`. Both are right. Which instructions take which of the two numbers?
8. You light an LED with `sbi PORTB, 5` and it works. Why can a driver that has to light *any* LED not be written that way at all?

Looking ahead

Chapter 2 takes the same LED and lights it with something that can be told which LED to light:

- `DDRx`, `PORTx` and `PINx`: three registers per port, and what each of the three actually does, which is more than the two `sbi` instructions at the end of this chapter needed to know.
- Why writing a one to `PINx` toggles an output, which is not what the name suggests at all.
- The calling contract: which registers a subroutine may destroy, and which it must give back.
- The LED driver: the same LED, lit by something that can be told which LED to light.

Sources

Every number this chapter quotes comes from one of two primary documents, and the text says which:

- *ATmega328P Datasheet*, Microchip Technology.
https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
- *AVR Instruction Set Manual*, Microchip Technology.
<https://ww1.microchip.com/downloads/en/devicedoc/atmel-0856-avr-instruction-set-manual.pdf>

1.6 Exercises

How to check your work. Exercises 1.5 and 1.6 are checked by this chapter’s test suite: write the file at the path the specification gives, then run `make test` from the repository root. Nothing has to be registered; creating the file is what switches its tests on.

The rest will have worked solutions in the companion repository, published later as `lectures/L01/appendix/f_solutions.md`, and the hand calculations have short answers in Appendix A. Work each one before you read it. Where an exercise has a plausible wrong answer, the solution gives that too, and says why it is wrong, because being told the right answer teaches you less than being shown the one you nearly gave.

No hardware is needed for any of this. Everything is written, assembled, simulated and measured on your laptop.

Each exercise is labelled with its kind. There is exactly one **Cross-check** per chapter, and in this one it is Exercise 1.8.

Exercise 1.1 The upper half

RECALL

- a) State which registers `ldi` can name, and which it cannot.
- b) Explain *why*, from the instruction’s encoding rather than from the datasheet’s table. Your answer should mention a specific number of bits.
- c) You are writing a routine that needs six working registers, and you have used `r16` to `r21`. A seventh value is a constant you need to load. Somebody suggests `r10`, which is free. What does that cost you, in instructions and in cycles, every time you set it up?
- d) Name two instructions other than `ldi` that have the same restriction, and one instruction that does not.

Exercise 1.2 Assembling four instructions by hand

HAND CALCULATION

Using the encoding in §1.1.6, and nothing else, work out the 16-bit word each of these assembles to. Show the four nibbles for at least the first one.

- a) `ldi r16, 0x2A`
- b) `ldi r24, 0x2A`
- c) `ldi r31, 0xFF`
- d) `ldi r20, 0x03`
- e) One of these is not a legal instruction: `ldi r15, 0x2A`. Say what the assembler does with it, and what would go wrong if it silently produced a word anyway.

Check yourself: the encoding is 1110 KKKK dddd KKKK with $d = R_d - 16$, and the assembler will settle it. Put the instructions in a `.asm` file, assemble it, and run `avr-objdump -D -m avr:5 -b ihex` over the hex; the bytes come out reversed, for the reason in §1.2.3.

Exercise 1.3 Which space, which instruction

RECALL

For each of the following, say which address space it names, what is stored there, and which instruction or instructions can reach it.

- a) Data space `0x0005`
- b) Data space `0x0025`
- c) I/O address `0x05`
- d) Data space `0x0100`
- e) Program word address `0x0006`
- f) EEPROM address `0x0000`
- g) Two of the above name the same physical register. Which two, and what happens if you use one of the two numbers with an instruction expecting the other?

Exercise 1.4 Choosing the branch

DESIGN

`r19` holds a loop counter and `r24` holds a limit. Both are unsigned.

- a) You want to leave the loop when the counter has reached the limit. Write the two instructions, and say which flag the second one reads.
- b) You want to stay in the loop while the counter is strictly below the limit. Write the two instructions.
- c) Somebody inserts an `inc r19` between the comparison and the branch. Explain what breaks, and why the symptom is a branch that goes the wrong way rather than an assembler error.

- d) Somebody inserts a `mov r20, r19` in the same place instead. Explain why this one is harmless, and say where you would look it up rather than guessing.

Exercise 1.5 Your first subroutines

CODE

Write `shift_bits` and `shift_bits_inverted` in `drivers/source/utils.asm`, to the specification in §1.4.

- Write both. Use `r18` and `r19` as scratch, for the reason in §1.4.5.
- Assemble with `make build`, then disassemble your own routine with `avr-objdump -D -m avr:5 -b ihex drivers/build/drivers.hex` and read it against what you wrote. Both flags are required, for the reason in §1.2.3: a hex file says nothing about its own architecture. At least one line will have come back as a different instruction than you typed. Say which, and why that is not a mistake.
- Run `make test`.
- Before running anything else: predict what `shift_bits` returns for an argument of 8, and say what a reader thinking in C would predict instead.

Exercise 1.6 A program that runs on its own

CODE

Write `drivers/app/main.asm` to the specification in §1.4.6.

- Write the reset vector, the stack pointer initialisation, a call to `shift_bits`, the two instructions that light `PB5`, and the final loop.
- `make build` should now report that it built `app.hex` as well as `drivers.hex`, and `make test` should report two more passing tests than it did before.
- The hardware sets `SP` to `RAMEND` at reset by itself, so on a cold start your four instructions change nothing. Give the case where they do change something, and say what would go wrong without them.
- Remove the final loop and describe, without running it, what the machine does when it reaches the end of your code. What is in flash after your last instruction?
- Swap the two `sbi` instructions, so that `PORTB` is written before `DDRB`. The LED still ends up lit. Say what the pin is doing during the one instruction in between, and why that is not the same thing as being an output that happens to be low.
- You wrote `sbi DDRB, 5`, and the test that checks it reads data space address `0x24`. Both numbers are right. Explain, and give the one instruction that would write the same register using `0x24` directly.

Exercise 1.7 Counting by hand

HAND CALCULATION

Using only the table in §1.3.2 and the rule in §1.3.3, and without running anything:

- Count the cycles `shift_bits` takes for an argument of 0. Show your working as a line per instruction, with how many times each one runs.
- Do the same for an argument of 7.
- From those two, write down the general formula in terms of n .
- How many times does the `cp` execute when n is 3? How many times does the `breq`? How many of those `breq` executions are taken? These three numbers are not all the same, and getting them confused is the usual reason a hand count comes out wrong.
- Do the same for `shift_bits_inverted`, and state the difference between the two formulas in one sentence about a single instruction.

Exercise 1.8 What a subroutine costs

CROSS-CHECK

Compute it by hand, measure it, reconcile.

This is the exercise the chapter exists for. Do the parts in order, and write each answer down before starting the next; the point is lost if you look at the measurement first.

- By hand.** From the exercise above, you have a formula. Evaluate it for $n = 0$ and for $n = 7$, and convert both to microseconds at 16 MHz. One cycle is 62.5 nanoseconds, and doing that conversion in your head once is worth more than a function that does it for you.
- The table, in full.** Write out the count for $n = 7$ in the shape §1.3.4 gives: one row per instruction, its cost, how many times it runs, and the product. You will need it in (d), and a formula you cannot decompose is a formula you cannot find the mistake in.
- By measurement.**

```
make measure SYMBOL=shift_bits ARG=0
make measure SYMBOL=shift_bits ARG=7
```

- Reconcile the two.** They should agree exactly. If they do not, the table is more likely wrong than the simulator, and the row to check first is the `cp`, which runs $n + 1$ times rather than n . Find the mistake before continuing rather than adjusting the formula until it matches.
- Now the call site.** Your `main.asm` does not just execute `shift_bits`; it loads an argument and calls it. Count, by hand, the cycles of those two extra instructions, and write down what a loop that called `shift_bits(7)` over and over would cost *per iteration*.
- The discrepancy.** Your answer to (e) is larger than your answer to (a). State the difference in cycles, and state it again as a percentage of the (a) figure, once for $n = 0$ and

once for $n = 7$. **The two percentages are very different.** Explain why, in one sentence, without using the word “overhead”.

- g) **What it means for a driver.** Chapter 2’s `led_on` calls `shift_bits` with a pin number. Using your formula, how many times longer does `led_on` take for Arduino pin 13 than for Arduino pin 8? (Pin 13 is bit 5 of port B; pin 8 is bit 0 of port B.) Say whether that difference would matter, and give one situation in which it would.
- h) `make_measure` reports a figure that does not include the `recall`. Is that a defect in the tool? Give the case for and against, in a sentence each, and say what you would have it do instead if you disagree with the choice.

I/O Ports and Subroutines

The first driver, and the contract that makes a subroutine reusable.

IN THIS CHAPTER

- DDRx, PORTx and PINx: three registers per port, and what each of the three actually does, which is more than Chapter 1's two sbi instructions had to know.
- The internal pull-up, which is a resistor you did not fit and cannot see.
- Writing a one to PINx, which toggles the output and is not what the name suggests at all.
- Arduino pin numbers against port bits, and why that translation is your driver's first job.
- rcall and ret: what goes on the stack, what comes back off it, and what it costs.
- The calling contract: which registers a subroutine may destroy and which it must give back.
- The LED driver: five subroutines, a structure in SRAM, and a cost that depends on the pin.

The chapter starts with one pin and its four states. DDRx and PORTx are two bits, so there are four combinations, and each of them does something different; predict what a floating input reads before you are told. Then the pull-up, and the button that reads backwards: a pressed button reads zero, which is not a convention anybody chose but falls out of the circuit, and the circuit is drawn. Then the translation: Arduino pin 13 is bit 5 of port B, and the mapping is worth deriving rather than reading. Then subroutines, properly: what rcall pushes, what ret does not undo, and the register contract that makes a subroutine safe to call from code you did not write. And finally the driver: five subroutines over a structure in SRAM, so that two LEDs can exist without the driver knowing there are two.

This chapter calls `shift_bits`, and the LED driver does not work without it, so finish Chapter 1 first.

Two predictions are worth making before anything runs:

- what `led_on` costs for Arduino pin 13, given what you already know `shift_bits` costs.
- whether `led_enabled` takes the same time whether the LED is on or off.

The second is this chapter's cross-check, and the answer is no.

2.1 I/O Ports

2.1.1 Three registers per port

You have already used two of these. `sbi DDRB, 5` and `sbi PORTB, 5` lit an LED at the end of Chapter 1 (§1.4.6), on one sentence’s worth of explanation: `DDRx` decides whether a pin is an output and `PORTx` decides what an output drives. That sentence is true and it is about a third of the story, and the missing two thirds are what makes a driver possible.

The ATmega328P has three I/O ports available on an Arduino Uno, called B, C and D, and each is controlled by three registers. For port B those are `DDRB`, `PORTB` and `PINB`, at data space addresses 0x24, 0x25 and 0x23 (§1.1.5).

The names are unhelpful in a specific way, so it is worth stating what each one is before anything else:

Register	What it is	Read or write
<code>DDRx</code>	D ata D irection R egister. One bit per pin: 0 is input, 1 is output.	Both
<code>PORTx</code>	For an output pin, the level to drive. For an input pin, the pull-up switch.	Both
<code>PINx</code>	The level the pin is actually at. Writing a 1 to it toggles <code>PORTx</code> .	Both, differently

`PORTx` is the one that trips people up, because it does two entirely unrelated jobs depending on what `DDRx` says. And `PINx` is the one that surprises them later, because writing to it does something that has nothing to do with reading it.

2.1.2 Four states of a pin

Two control bits, four combinations, and each one is a different piece of hardware behaviour.

	PORTx = 0	PORTx = 1
DDRx = 0 input	Input, high impedance floating: reads whatever is on the wire	Input, pull-up enabled reads 1 until something pulls it down
DDRx = 1 output	Output, driven low sinks current, holds the pin at 0 V	Output, driven high sources current, holds the pin at 5 V

PINx is the third register and is not part of this grid: reading it gives the pin's actual level, and writing a 1 to it toggles PORTx. Writing a 0 to it does nothing.

Figure 2.1 DDRx against PORTx: a high-impedance input, an input with its pull-up enabled, an output driven low, and an output driven high. PINx is the third register and not part of the grid.

A floating input is the dangerous one. With $\text{DDRx} = 0$ and $\text{PORTx} = 0$ the pin is connected to nothing at all, and reading PINx gives you whatever the surrounding electrical noise induced on it. It is not random in a useful sense and it is not stable; it will read 1 while your finger is near the board and 0 when it is not. An input you have not either pulled up or wired to something is a bug, not a default.

The pull-up is the reason the other three states are easy. Setting $\text{PORTx} = 1$ on an input pin connects an internal resistor of 20 to 50 k Ω between the pin and 5 V. Now the pin reads 1 unless something actively pulls it down, which is exactly what a button to ground does.

An output ignores the pull-up entirely. Once $\text{DDRx} = 1$, PORTx means “the level to drive”, and the resistor is out of the picture.

2.1.3 Writing to PINx toggles the output

This is the one genuinely strange thing in this section, and it is worth being explicit about because the name gives no hint of it.

Writing a **1** to a bit of PINx **toggles the corresponding bit of PORTx**. Writing a 0 does nothing at all. Reading PINx is unaffected and still gives the pin's actual level.

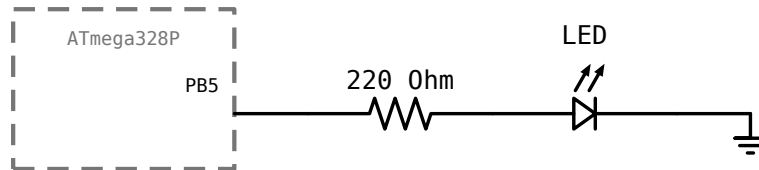
So storing a mask into PINB flips exactly the bits the mask selects, and leaves the rest alone without having to read anything first. A toggle is one store, and the alternative is a load, an exclusive-or and a store. It is faster and, more importantly, it cannot be interrupted halfway through, which is a property Chapter 3 will make matter.

Two things this does **not** mean:

- It does not toggle the pin directly. It toggles PORTx, so on an input pin it toggles the pull-up, which is occasionally useful and usually a mistake.

- It is not “writing to the input register”, whatever the name suggests. Nothing you write to PINx is ever read back from PINx.

2.1.4 An LED, and a resistor that is not optional



PORTB bit 5 high lights it, low turns it off. The resistor is not optional: the pin will happily deliver enough current to destroy the LED, and then itself.

Figure 2.2 A pin leaving the ATmega328P through a 220 Ω series resistor into an LED and then to ground. The chip boundary is drawn dashed.

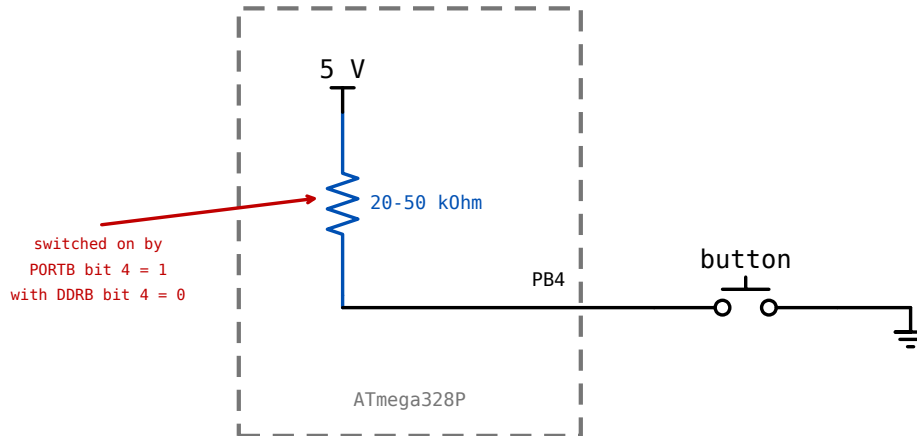
Configure the pin as an output, drive it high, and current flows through the LED. Drive it low and it stops. That is the whole circuit, and the driver in §2.3 is a way of saying it in assembly.

The resistor sets the current. An LED is not a resistor: its voltage barely changes with current, so without something to limit it the current is set by how much the pin can supply, which is about 40 mA, which is roughly twice what a small LED survives and exactly the pin’s absolute maximum. The usual outcome is a dead LED; the occasional outcome is a dead pin.

An Arduino Uno already has an LED and a resistor on pin 13, which is why every example in this book uses pin 13 and why none of them requires you to wire anything.

The other way round works too, and you will meet it. An LED can be wired from 5 V through its resistor to the pin, in which case driving the pin *low* lights it. That is called sinking rather than sourcing, and it inverts everything your driver means. It is not wrong; it is a different board, and reading `led_on` without knowing which one you have is how you end up with a program that is exactly backwards. On this part the two directions are specified the same, whatever the folklore about older AVRs says: 20 mA either way.

2.1.5 A button, and why pressed reads zero



Not pressed, the pull-up holds the pin at 5 V and PINB reads 1. Pressed, the button shorts it to ground and PINB reads 0. A pressed button reads zero, which is the wrong way round from every intuition, and is the whole reason `btn_pressed` has to invert what it read.

Figure 2.3 A pin with the internal 20 to 50 k Ω pull-up inside the chip boundary, and a button to ground outside it. The pull-up is switched on by writing 1 to PORTB with DDRB at 0.

Configure the pin as an input and switch the pull-up on. With the button not pressed, nothing else is connected to the pin, and the pull-up holds it at 5 V, so `PINx` reads 1. Press the button and the pin is shorted to ground, so `PINx` reads 0.

A pressed button reads zero. Every driver in this book that reports “is it pressed” therefore has to invert what it read, and forgetting to is a bug that produces a program which does exactly the opposite of what it should, reliably, which is at least easy to spot.

The alternative wiring, a button from the pin to 5 V with an external pull-down resistor, gives the intuitive polarity. Nobody does it, because it needs a resistor you have to fit and the pull-up is free.

What the pull-up does not do is debounce. A mechanical button does not close once; it closes, bounces open, closes again, several times over a few milliseconds. The pin faithfully reports every one of those, so a program that counts presses by watching for a 1-to-0 transition will count one press as four. That is not this chapter’s problem, but it is the reason Chapter 3’s interrupt-driven button behaves in a way you would not predict from this circuit alone.

2.1.6 Arduino pin numbers are not port bits

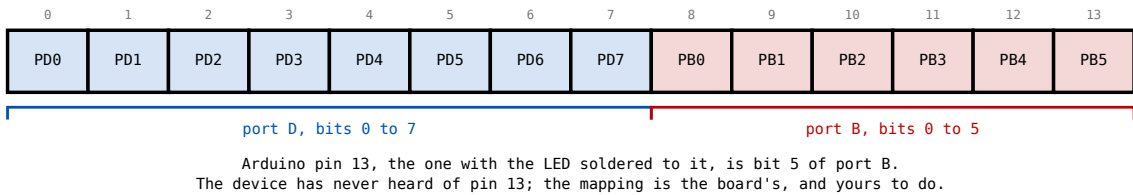


Figure 2.4 Fourteen cells, one per Arduino digital pin: pins 0 to 7 are port D bits 0 to 7, and pins 8 to 13 are port B bits 0 to 5.

The number silkscreened on the board is the board's idea. The device has never heard of it, and a different board with the same chip numbers its pins differently.

The mapping is simple enough to state in one line, which is fortunate, because your driver has to do it at run time:

- Arduino pins **0 to 7** are port **D**, bits 0 to 7.
- Arduino pins **8 to 13** are port **B**, bits **0 to 5**.

The step at pin 8 is where implementations go wrong. Pin 8 is bit **0**, not bit 8, so the driver has to subtract 8 before it shifts. Shifting by 8 in an eight-bit register gives zero (§1.4.3), so the mask is zero, the LED never lights, nothing reports an error, and the symptom looks like a wiring fault.

Port B stops at bit 5 because bits 6 and 7 are the crystal pins on an Arduino Uno. They exist on the chip and are not available to you.

Build this table yourself before reading it again. Pin 13 is the one that matters here, and deriving the other rows from the rule rather than reading them off is the difference between knowing the mapping and having seen it.

2.1.7 Reading, modifying, writing

A port register holds eight pins. Your LED is one of them. So a driver that wants to turn its LED on must not write to `PORTB`; it must change one bit of `PORTB` and leave the other seven as it found them.

Three instructions and five cycles:

1. **Read** the whole register into a scratch register, two cycles.
2. **Modify** it, one cycle, by or-ing in the mask that selects your bit.
3. **Write** the whole register back, two cycles.

The middle step is where the mask earned its keep. To clear a bit instead, the mask is inverted and the or becomes an and, which is why `shift_bits_inverted` exists as its own routine rather than as a com after `shift_bits`.

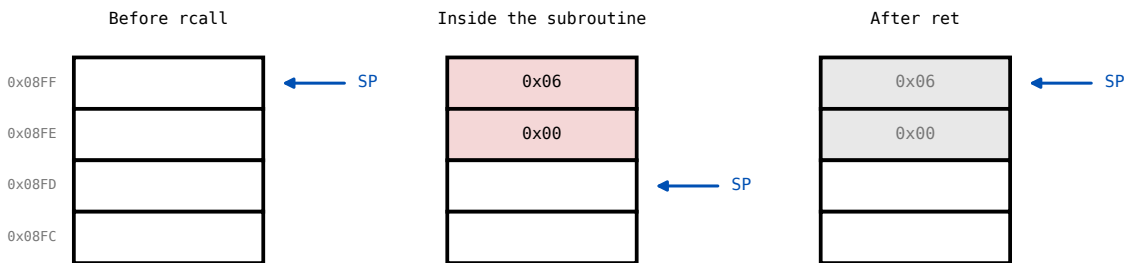
A driver that assigns instead, `sts PORTB + 0x20, r24`, works perfectly with one LED and turns off every other output on the port. It passes any test that only has one LED in it, which is why the test suite has one with two.

Read-modify-write is not atomic, and on this machine that is three separate instructions with two gaps in them. Nothing in this chapter can interrupt those gaps. From Chapter 3 onwards something can, and that is where `sbi`, `cbi` and the `PINx` toggle stop being micro-optimisations and start being correctness.

2.2 Subroutines and the Calling Contract

2.2.1 What `rcall` and `ret` actually do

`rcall` does two things: it pushes the address of the next instruction onto the stack, and it jumps. `ret` pops that address back into the program counter. There is nothing else to it, and in particular there is no frame, no argument list and no register saving.



The return address is a word address, and 0x06 is the low byte of it. `ret` moves the stack pointer back and erases nothing: the two bytes are still there, and are simply no longer anybody's. Every push you do not pop leaves the stack pointer somewhere you did not mean.

Figure 2.5 Four bytes of SRAM, three times: before `rcall` with the stack pointer at `RAMEND`, inside the subroutine with two bytes pushed and the pointer two lower, and after `ret` with the pointer back and the two bytes still there.

Three facts in that figure are worth reading slowly, because all three are commonly got backwards.

The address pushed is a word address. The disassembler prints byte addresses, so an `rcall` at byte 0x0A returns to byte 0x0C, and what goes on the stack is 0x0006. Halving it is not a detail you have to apply yourself anywhere in this book, but it explains a stack dump that otherwise looks like it contains the wrong numbers.

The low byte ends up at the higher address. `SP` starts at `RAMEND` and decrements after each byte, and the low byte goes first, so `RAMEND` holds the low byte and `RAMEND - 1` the high one.

ret erases nothing. It moves the stack pointer and leaves the bytes exactly where they were. Everything below SP is not “empty”; it is stale, and the next `rcall` will overwrite it. A program that reads below its own stack pointer finds the ghosts of previous calls, which is occasionally a useful debugging trick and never a thing to rely on.

`rcall` costs 3 cycles and `ret` costs 4, so the cheapest possible subroutine call costs 7 cycles before the subroutine has done anything. That is not an argument against subroutines; it is the reason the cross-check in Exercise 1.8 came out the way it did.

2.2.2 The stack is one pointer and no protection

SP lives in SPH:SPL, at data space 0x5E:0x5D, and it is the entire stack mechanism. Three different things use it and none of them coordinates with the others:

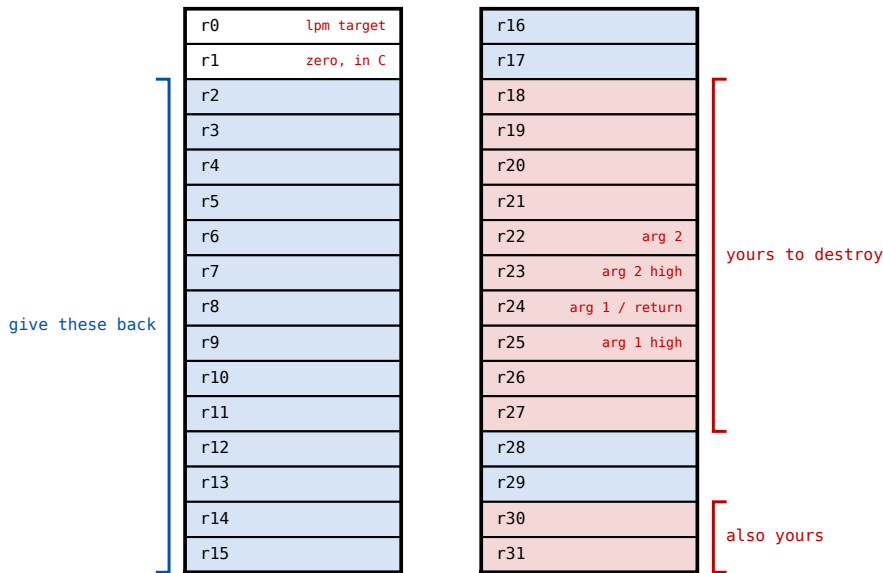
- `rcall` and `ret`, for return addresses.
- `push` and `pop`, for whatever you decide to save.
- An interrupt, which pushes a return address whether you were expecting it or not (Chapter 3).
- Nothing else, because there are no local variables. If a subroutine needs memory, it either uses registers or you allocate it somewhere yourself.

The stack grows **downwards**, from RAMEND towards your variables. Nothing checks. A subroutine that pushes without popping moves SP down permanently, and after enough calls the stack reaches whatever you put at the bottom of SRAM and quietly overwrites it. There is no fault, no message and no crash: just a variable that changes on its own.

Every push needs its pop, in the opposite order. That is not advice, it is the only thing keeping `ret` pointed at a return address rather than at half of one.

2.2.3 The calling contract

The hardware has no opinion about registers. Every convention below is a convention, and the reason to follow it from the first chapter is that the C compiler follows it too, and in Chapter 6 the two of you will be calling each other.



Red is call-clobbered and blue is call-saved. Nothing in the hardware enforces either: this is a convention, and the compiler is the other party to it.

Figure 2.6 The 32 registers coloured by role: r18 to r27 and r30 to r31 call-clobbered, r2 to r17 and r28 to r29 call-saved, with r24 marked as the first argument and return value and r22 as the second.

Registers	Rule
r18–r27, r30, r31	Call-clobbered. A subroutine may destroy these. A caller that wanted one kept must save it itself.
r2–r17, r28, r29	Call-saved. A subroutine that uses one must push it on entry and pop it before ret.
r24, or r25:r24	First argument, and where a byte or word result is returned.
r22, or r23:r22	Second argument.
r1	Assumed to hold zero by compiled C. If you change it, put it back.

So a driver subroutine taking a pointer and a pin number takes them in r25:r24 and r22, and this is why `shift_bits` used r18 and r19 for scratch in Chapter 1 rather than r16 and r17.

What breaks when you ignore it is nothing, for a while. Inside this book’s own library everything is called from code you wrote, so a subroutine that clobbers r16 merely surprises you. The failure arrives in Chapter 6, when the caller is a C function that had a live variable in r16, and the symptom is a value changing across a function call for no reason visible in the source. That is among the least pleasant things this book can produce, and it is entirely avoidable now.

2.2.4 Calling a subroutine from a subroutine

The LED driver calls `shift_bits`, and `shift_bits` is a subroutine like any other, so two things follow that are worth stating before you write the code.

Anything call-clobbered is gone afterwards. `shift_bits` destroys `r18`, `r19` and `r24`. A driver that loaded a port register into `r18` and *then* called `shift_bits` gets the shift result in `r18` and its port contents nowhere. So the read has to happen after the call, not before, and that ordering is a pinned detail of the specification (§2.3.3), not a matter of taste.

The stack grows by another two bytes. A nested call is two return addresses deep. That is fine here and it is worth knowing the depth, because from Chapter 3 an interrupt can arrive on top of it and make it three.

What does survive, in this library, is `Z`. `shift_bits` never touches `r30` or `r31`, so a driver can leave its structure pointer there across the call. Note that this is a fact about `shift_bits`, not a guarantee from the contract: `Z` is call-clobbered, so a stricter subroutine would be within its rights to destroy it. Relying on a specific routine's behaviour rather than on the contract is a decision, and it is one you should make on purpose and write down.

2.2.5 Passing a structure

The driver's five subroutines all operate on one LED, and there is more to an LED than fits in a register: three 16-bit addresses and a bit number, seven bytes in all.

So the caller allocates those seven bytes somewhere in SRAM and passes the **address** in `r25:r24`. The subroutine copies that pair into `Z` with one `movw`, and then reads any field it wants with a single `ldd`, which takes a base register and a constant offset and costs the same two cycles as a plain `ld`. One instruction per field, and the offset is the name from `led.inc` rather than a number.

That is what makes two LEDs possible. Nothing in the driver knows how many exist; each call is told which one to work on, and the answer lives in SRAM rather than in the code. The alternative, a driver that hard-codes `PORTB` and bit 5, is shorter and works exactly once.

Where those seven bytes go is the caller's problem. In this chapter the tests place them at `0x0200`, which is inside SRAM and well clear of the stack. Chapter 4 is where allocating them properly becomes a topic in its own right.

One warning about a pattern you will find in AVR code elsewhere, including in the material this book is based on: allocating driver structures at `RAMEND + 1`. On a device with external RAM that is where external RAM starts. An ATmega328P on an Arduino Uno has none, so `RAMEND + 1` is not memory at all, and storing there writes to nothing while reading gives nothing back. It looks deliberate and it is a bug.

2.2.6 Four instructions you need before Chapter 4 explains them

The driver in §2.3 reaches into a structure, and reaching into a structure needs a pointer. Chapter 4 is where pointers are the subject; here is the minimum to write this chapter's code without waiting for it.

Instruction	Does	Cycles
<code>movw r30, r24</code>	Copies a register pair to a pair, so <code>r25:r24</code> into <code>Z</code> in one go	1
<code>ldd r18, Z+6</code>	Loads the byte 6 past where <code>Z</code> points, without moving Z	2
<code>std Z+6, r18</code>	Stores it back to the same place. The pointer comes first when storing	2
<code>adiw r26, 1</code>	Adds a small constant to a register pair, here stepping <code>X</code> on by one	2

Three limits go with them, and each is a real constraint on how you write the driver rather than a footnote:

- **ldd and std take a constant offset, at most 63.** It is encoded in the instruction, so it cannot come from a register. Every offset in `led.inc` is well under 63, which is not an accident.
- **X has no displacement form.** There is no `ldd r18, X+6`. So `Z` is the pointer for reaching into a structure and `X` is the pointer for holding an address you read out of one, which is exactly how the LED driver uses the two.
- **adiw takes at most 63 and works only on r24, r26, r28 and r30**, the four pairs the instruction can name. Adding more than 63, or to any other pair, takes `subi` and `sbc` with the negated value instead.

Why any of this is shaped the way it is, and what the other addressing modes are for, is §4.1.

2.3 The LED Driver

2.3.1 The task

Write `drivers/source/led.asm`: five subroutines that drive one LED, told which LED to drive by a pointer to a structure in SRAM.

The driver is the first thing in this book that is a *component* rather than a routine. Nothing in it knows how many LEDs exist, which port they are on, or which bit; all of that is in the structure it is handed. That is the whole design, and it is why the same five subroutines serve fourteen pins across two ports.

Creating `drivers/source/led.asm` is all it takes to switch its tests on.

2.3.2 The structure

Seven bytes, laid out as `drivers/include/led.inc` declares. The offsets are given rather than chosen, because the test suite reads your structure out of SRAM at these offsets to check that `led_init` built it correctly.

Offset	Field	Size	Holds
0	LED_PIN_REG	2	Data space address of PINx, low byte first
2	LED_DIR_REG	2	Data space address of DDRx
4	LED_PORT_REG	2	Data space address of PORTx
6	LED_PIN	1	Bit number within the port, 0 to 7
	LED_SIZE	7	Total

Two things to notice. The order is the hardware's own, PINx then DDRx then PORTx one byte apart, so a driver that has the first can reach the others by adding one and two and never has to know which port it is on. And the last field is the **bit number**, not the Arduino pin number: the translation happens once, in `led_init`, and everything afterwards works in bits.

Include it with `.include "led.inc";` make `build puts drivers/include` on the include path.

2.3.3 `led_init`

Fill in the structure and configure the hardware.

Arguments: `r25:r24` is the address of the structure. `r22` is the Arduino pin number.

Returns: `r24` is 0 on success, 1 if the pin number is not one this driver accepts.

Write it in this order:

1. **Copy the structure pointer into Z**, with `movw`. Everything after this is `std` and `ldd` against `Z`, and `Z` survives the calls to `shift_bits` (§2.2.4).
2. **Validate and choose the port**. Below 8 is port D. Below 14 is port B, and **subtract 8** from the pin number so that it becomes a bit number. Anything else returns 1 immediately, before writing anything at all.
3. **Store the bit number and the three addresses**. Load the port's PINx address into `X`, store it at offset 0, `adiw X, 1` and store at offset 2, `adiw X, 1` again and store at offset 4. Use `PINB` and `PIND` from `m328Pdef.inc`, adding `0x20` to reach the data space (§1.1.5).
4. **Make the pin an output**. Call `shift_bits` with the bit number, then read `DDRx`, or the mask in, and write it back.
5. **Leave the LED off**. Call `shift_bits_inverted` with the bit number, then read `PORTx`, and the mask in, and write it back.
6. **Return 0**.

Pinned details

Validate before writing. A rejected pin must leave the structure untouched, so the range check comes before the first `std`. A driver that fills the structure and then discovers the pin is invalid has already half-configured something.

Read the port register after calling `shift_bits`, not before. `shift_bits` destroys `r18`, `r19` and `r24`, so a driver that loads `DDRx` into `r18` and then calls it gets the mask in `r18` and

the port contents nowhere. This is the single most likely way to get a driver that assembles, runs, and does nothing.

Read, modify, write. Never assign to a port register (§2.1.7). Seven other pins share it.

Subtract 8 for port B. Pin 8 is bit 0. Forgetting produces a mask of zero and an LED that never lights (§2.1.6).

2.3.4 `led_on`, `led_off` and `led_toggle`

All three take the structure address in `r25:r24` and return nothing.

`led_on` reads the bit number from the structure, calls `shift_bits`, then reads `PORTx`, ORs the mask in, and writes it back.

`led_off` is the same with `shift_bits_inverted` and an `and` instead of an `or`. Note that this is why `shift_bits_inverted` was written as its own loop in Chapter 1 rather than as `shift_bits` followed by `com`: it is called here on the hot path, and the extra `rcall`, `ret` and `com` would have cost more than the one `inc` per trip that it costs instead.

`led_toggle` calls `shift_bits` and then writes the mask to `PINx` and nothing else (§2.1.3). No read, no OR, no write-back: one `st`. It is the shortest of the three and the only one that cannot be interrupted halfway.

2.3.5 `led_enabled`

Arguments: `r25:r24` is the structure address.

Returns: `r24` is 1 if the LED is on, 0 if it is off.

Read `PINx`, not `PORTx`. For an output pin the two agree, so it makes no difference here, and it makes every difference the moment somebody hands this subroutine a pin that is an input: `PORTx` would report the pull-up setting and `PINx` reports the world.

Mask the value with the bit, and return 1 if the result is non-zero. The natural way to write that ends with a conditional branch, and **that branch is why this subroutine costs one cycle more when the answer is yes** (§2.3.6). Do not try to avoid it; notice it.

2.3.6 What it should cost

Every figure below is measured. You are asked to predict some of them in the exercises (§2.6) before you look.

The shape of all five subroutines is the same: a fixed part that reads the structure and writes a register, plus one call to a shift routine whose cost depends on the bit number. So each of them costs $\text{slope} \times \text{bit} + \text{constant}$, and the slope is the interesting half.

Subroutine	Slope, cycles per bit	Why
led_on	6	one shift_bits
led_off	7	one shift_bits_inverted, which has one more inc per trip
led_toggle	6	one shift_bits
led_enabled	6	one shift_bits
led_init	13	one of each

The test suite checks two of these slopes, **led_on's** and **led_off's**, and none of the constants. It also checks that `led_enabled` costs one cycle more when the answer is yes. That is a deliberate change from Chapter 1, where the specification gave `shift_bits` instruction by instruction and the test asserted its cost exactly. A driver is a bigger thing with more than one reasonable shape, and the slope is the part that follows from what it has to do rather than from how you chose to write it.

The one absolute figure worth having in mind: on the reference implementation `led_on` costs **59 cycles for Arduino pin 13** and **29 for pin 8**, which is $3.7\text{ }\mu\text{s}$ against $1.8\text{ }\mu\text{s}$ at 16 MHz.

2.3.7 What this driver does not do

It does not debounce. Nothing here is about buttons yet, but the same will be true of the button driver in Chapter 3, and it is worth saying once: a driver that reports the state of a pin reports the state of the pin, bounces included.

It does not remember anything. `led_enabled` reads the hardware rather than a flag, which means it stays right if something else changes the port behind its back, and it also means it cannot tell you whether the LED is *meant* to be on.

It does not check that the pin is an output. `led_on` on a pin that something else has made an input would enable a pull-up rather than lighting anything. There is no type system here; the structure is seven bytes and any subroutine will operate on any seven bytes it is given.

It does not protect one LED from another. Two structures pointing at the same bit of the same port both work, and they fight. Nothing detects it.

2.4 Putting It on a Real Board

2.4.1 Nothing in this book needs this

Every exercise is written, assembled, simulated and verified on your laptop. No test knows whether you own a board, and none of them will ever ask.

This section exists because an LED that lights is a different experience from a test that passes, and because the last step of the toolchain is worth having done once. If you have an Arduino Uno in a drawer, this is twenty minutes. If you do not, skip it; the simulator is not a poor substitute here, it is the thing the rest of the book is built on.

2.4.2 What you need

An **Arduino Uno**, or any board with an ATmega328P and a USB serial bridge, and a USB cable. Nothing else: the LED on pin 13 is already fitted, with its resistor (§2.1.4).

avrdude is a separate package, and the book's install lines leave it out on purpose: this is the only section that uses it, and it is optional. Install and check it:

```
sudo apt -y install avrdude
avrdude -v 2>&1 | head -n 2
```

2.4.3 The file avrdude wants, you already have

avrdude wants an Intel HEX file, and avra writes one. `drivers/build/app.hex` is the artefact this section flashes, with no conversion step between the build and the programmer.

That is worth a sentence because it is not the usual arrangement. A GNU-toolchain AVR project links to an ELF and then runs `avr-objcopy -O ihex` to get here; this book skips both, because its assembler has no linker and emits the programmer's format directly (§1.2.2).

The size comes free with the build, near the end of the listing avra writes beside the hex, `drivers/build/app.lst`:

```
Segment usage:
Code          :      17 words (34 bytes)
```

A blinking LED written in assembly comes out at a few dozen bytes against 32 KB of flash. Worth looking at once, if only to see how much of a microcontroller a small program does not use.

2.4.4 Finding the board

Plug it in and look:

```
ls /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
```

An official Uno is usually `/dev/ttyACM0`; a clone with a CH340 bridge is usually `/dev/ttyUSB0`.

If nothing appears, and you are on **WSL**, that is expected rather than broken: WSL does not see USB devices without `usbipd-win` forwarding them. Flashing from Windows, or from a native Linux install, is the shorter path.

If the device appears but you cannot open it, you are not in the right group:

```
sudo usermod -a -G dialout "$USER"
```

Then log out and back in, because group membership is established at login.

2.4.5 Flashing

```
avrdude -c arduino -p atmega328p -P /dev/ttyACM0 -b 115200 \
-U flash:w:drivers/build/app.hex:i
```

Option	Means
-c arduino	Talk to the bootloader over serial, not to an external programmer.
-p atmega328p	The device, so avrdude can check it is talking to what it thinks.
-P	The serial port from §2.4.4.
-b 115200	The bootloader's baud rate. An older Uno may want 57600.
-U flash:w:...:i	Write this Intel HEX file to flash.

The board resets, the bootloader runs, the program is written and verified, and then your code starts. If your program has no final loop, you will see it restart repeatedly (Exercise 1.6, part d).

2.4.6 When it does not work

programmer is not responding. Usually the wrong port, sometimes the wrong baud rate on an older board, occasionally a serial monitor somewhere else holding the port open.

expected signature ... but found The chip is not an ATmega328P. Clone boards sometimes carry an ATmega328PB, which is a different device with a different signature and a slightly different peripheral set.

It flashes cleanly and nothing happens. Three candidates, in order of likelihood: you flashed `drivers.hex` rather than `app.hex` and there is no reset vector; your program ran off the end and is restarting; or the pin is right and the LED is on a different one, because this is not an official Uno.

The bootloader is not the only way in. With an ISP programmer you can write flash directly and skip it, which is also how you would change the fuses. This book never does either. Fuses are worth respecting: the wrong clock setting will make a board that no longer responds to a programmer, and recovering it needs a second board and an afternoon.

2.4.7 What the board shows you that the simulator does not

Not much, honestly, and it is worth being precise about the short list rather than implying the hardware is somehow more real.

That your wiring is right. The simulator models the chip, not the board, so it cannot tell you the LED is on pin 12.

Analogue reality. Voltage, current, rise times, a supply that sags. None of it is in the simulator and none of it is in this book either, but it is where embedded work eventually goes.

That the clock is what you assumed. Every timing number in this book assumes 16 MHz. On a bare ATmega328P running from its internal oscillator with the factory CKDIV8 fuse set,

the clock is 1 MHz and everything is sixteen times slower; clear the fuse and it is 8 MHz and twice as slow. Every cycle count is still exactly right in each case, which is a good demonstration of the difference between cycles and seconds.

What the simulator shows you that the board does not is the entire subject of this book: which register changed, when, and what it cost.

2.5 Review

What you should be able to do now

- Say what each of DDRx, PORTx and PINx does, and give the four states of a pin.
- Configure a pin as an input with its pull-up on, and say what it reads with nothing connected.
- Explain why writing a one to PINx toggles an output, and why writing a zero does nothing.
- Translate an Arduino pin number into a port, a bit, a mask and three register addresses.
- Say exactly what `rcall` pushes, in what order, and what `ret` leaves behind.
- Say which registers a subroutine may clobber, and what breaks when it clobbers the others.
- Write a driver that reads, modifies and writes a port register rather than assigning to it, and say what goes wrong when it assigns.
- Predict how a driver's cost varies with the pin it was given, and check the prediction.

Questions to test yourself

1. DDRx bit is 0 and PORTx bit is 1. Is that pin an input or an output, and what does it read?
2. You want to turn one LED on without disturbing the seven other bits of its port. Write the three-instruction sequence, and say why a single `sts` will not do.
3. A button is wired from a pin to ground with the internal pull-up on. `btn_pressed` should return 1 when it is pressed. What does PINx read at that moment, and what must the driver do about it?
4. `rcall` pushes two bytes. Are they the byte address of the next instruction or its word address, and which byte goes at the higher address?
5. Your subroutine uses `r16` as scratch and does not restore it. Nothing in this book breaks. Give the situation in which something does.
6. `led_on` costs 59 cycles for Arduino pin 13 and 29 for pin 8. Where do the 30 cycles go, and what does that tell you about wiring an LED to a low-numbered pin?

Looking ahead

Chapter 3 is about code that runs when you are not looking:

- The interrupt vector table, and why a vector holds a jump rather than a handler.
- What the hardware saves when an interrupt fires, which is less than you would like.
- Pin change interrupts, and one handler serving eight pins.
- The button driver, and the first bug in this book that only appears sometimes.

2.6 Exercises

How to check your work. Exercise 2.5 is checked by this chapter's test suite: write the file at the path the specification gives, then run `make test`. The suite is cumulative, so it runs Chapter 1's `shift_bits` tests too; if something you write here breaks it, this is where it shows up. The rest will have worked solutions in the companion repository, published later as `lectures/L02/appendix/f_solutions.md`, and the hand calculations have short answers in Appendix A. Work each one before you read it.

No hardware is needed for any of this. §2.4 is optional and nothing depends on it.

Each exercise is labelled with its kind. There is exactly one **Cross-check**, and here it is Exercise 2.8.

Exercise 2.1 Four states

RECALL

- Give the four combinations of `DDRx` and `PORTx` for a single pin, and say what each one does to that pin.
- Which of the four is a mistake rather than a choice? Say what a program reading it gets.
- `PINx` is not in that table. Give both of the things it does, and say why neither of them is what its name suggests.
- You want to toggle one output pin. Give the one-instruction way and the three-instruction way, and one reason to prefer the short one that is not about speed.

Exercise 2.2 Wiring a button

DESIGN

A button is wired between an ATmega328P pin and ground, with nothing else attached.

- Give the values of `DDRx` and `PORTx` for that pin, and say what each one is doing.
- What does `PINx` read when the button is not pressed? When it is?
- A driver subroutine `btn_pressed` should return 1 when the button is down. Write, in words, what it has to do to the bit it read, and say why.

- d) Somebody proposes leaving PORTx at 0 and fitting an external pull-down resistor to ground, wiring the button to 5 V instead. Their polarity is now the intuitive one. Give one real advantage of their scheme and two reasons nobody does it.
- e) Your driver reads the pin every millisecond and counts a press each time it sees a 1 then a 0. Presses are being counted several times each. Explain, without using the word “software”.

Exercise 2.3 Translating pin numbers

HAND CALCULATION

For Arduino pins 3, 8 and 13, give the port, the bit number, the mask, and the data space addresses of the three port registers. Do this from §2.1.6 and §1.1.5, not from a table you have already been shown.

- a) Fill in the three rows.
- b) What is the mask for pin 8, and what would it be if you forgot to subtract 8 first? Say what the LED does in each case.
- c) Arduino pins stop at 13. Port B has eight bits. What are the other two, and why can you not use them?

Check yourself: the port and bit for every pin are in §2.1.6, and deriving a row is quicker than looking it up.

Exercise 2.4 What a call leaves behind

RECALL

- a) An `rcall` at byte address 0x0A calls a subroutine. What two things does the instruction do, in order?
- b) Exactly what value goes on the stack? Say whether it is a byte address or a word address, and give the number.
- c) Which byte of it ends up at the higher address?
- d) SP was 0x08FF. What is it inside the subroutine, and what is it after `ret`?
- e) After the `ret`, what is stored at 0x08FF?
- f) A subroutine does one push and no pop, then `ret`. Describe what happens, and say why there is no error message.

Exercise 2.5 The LED driver

CODE

Write `drivers/source/led.asm` to the specification in §2.3.

- a) Write all five subroutines. `.include "led.inc"` for the field offsets.
- b) Run `make test`.
- c) Disassemble your `led_init` and count how many instructions run in the case where the pin number is rejected. Compare it against the case where it is accepted.

- d) Deliberately move the range check to *after* the structure is filled in, rebuild, and say which test catches it. Then put it back.

Exercise 2.6 Read, modify, write

DESIGN

- a) Write the three-instruction sequence that sets one bit of a port register, given the mask in `r24` and the register's address in `X`.
- b) Somebody replaces it with a single `st X, r24`. Their LED works. Describe precisely what else on that port stops working, and say why their own test did not catch it.
- c) Which test in this chapter's suite does catch it? Look it up.
- d) `led_toggle` needs no read-modify-write at all. Say why, and give the one property that gives it besides being shorter.
- e) The AVR has `sbi` and `cbi`, which set and clear one bit of a low I/O register in a single instruction. Give the reason this driver cannot use them, and note that it has nothing to do with speed.

Exercise 2.7 Predicting the driver's cost

HAND CALCULATION

You know from Chapter 1 that `shift_bits` costs $6n + 10$ cycles and `shift_bits_inverted` costs $7n + 10$, where n is the number of shifts.

- a) Without counting the rest of `led_on`, say how much more it must cost for Arduino pin 13 than for pin 8, and show why.
- b) Do the same for `led_off`.
- c) `led_init` calls both shift routines once each. How much more does it cost for pin 13 than for pin 8?
- d) Which of the five subroutines is the cheapest for a given pin, and why?
- e) Explain why you can answer all of the above without knowing how the rest of each subroutine is written.

Exercise 2.8 A driver whose cost depends on its argument

CROSS-CHECK

Compute it by hand, measure it, reconcile.

Do the parts in order and write each answer down before starting the next.

- a) **By hand.** From Exercise 2.7, predict the difference in cycles between `led_on` for Arduino pin 13 and `led_on` for pin 8. Convert it to microseconds at 16 MHz.
- b) **In full, on paper.** Write the count out in the shape §1.3.4 gives, once for bit 0 and once for bit 5. Your table has to include the nested call, so it needs a row for the `rcall` and then every row of `shift_bits` itself. Do this before (c).
- c) **By measurement.**

```
make measure CALLS="led_init:0x0200:8 led_on:0x0200"
make measure CALLS="led_init:0x0200:13 led_on:0x0200"
```

Two calls, because `led_on` needs a structure that `led_init` had to build first.

- d) **Reconcile.** The **difference** between the two pins should come out the same in both. If your absolute figures disagree while the difference agrees, say what that tells you about where the error is, and find it: a constant offset is a row you left out of the table, and it is the same row for both pins.
- e) **Now the asymmetry.** Predict whether `led_enabled` takes the same number of cycles when the LED is on as when it is off. Write the prediction down, then measure both:

```
make measure CALLS="led_init:0x0200:13 led_on:0x0200 led_enabled:0x0200"
make measure CALLS="led_init:0x0200:13 led_off:0x0200 led_enabled:0x0200"
```

- f) **The discrepancy.** State the difference in cycles and explain it in one sentence. Then say what it would take to write a version of `led_enabled` whose cost did **not** depend on the answer, and whether you think it is worth doing.
- g) **What it means.** You are bit-banging a protocol that needs a pulse held for exactly 5 μs , and you are using this driver. Say which of the five subroutines you could use inside that pulse and which you could not, and give the one change to the *wiring* that would make the problem smaller.

Exercise 2.9 A program that blinks

CODE

Extend `drivers/app/main.asm` so that it initialises the LED on pin 13 and toggles it forever.

- a) Write it. There is no delay routine yet, so it will toggle as fast as the loop runs.
- b) Work out how fast that is, in cycles per toggle, from your `led_toggle` cost and the loop's own `rjmp`. Convert it to a frequency.
- c) An LED toggling at that frequency does not look like it is blinking. Say what it looks like and why, and what would have to change for it to blink visibly. Chapter 5 is where you fix it properly.
- d) `make build` should report that it built `app.hex`. If you own a board, §2.4 will put it on one.

Interrupts

Code that runs when you are not looking, and everything that follows from that.

IN THIS CHAPTER

- The interrupt vector table, and why a vector holds a jump rather than a handler.
- What the hardware saves when an interrupt fires, which is less than you would like.
- `reti` against `ret`, and the global interrupt enable that sits in `SREG`.
- Pin change interrupts: one vector per port, eight pins sharing it, and no way to tell which.
- The ISR contract: saving `SREG`, saving what you use, and why the order matters.
- Atomicity: a sixteen-bit value that is briefly neither of its two correct values.
- The button driver, and the first bug in this book that only appears sometimes.

The chapter starts with the table, before any handler: twenty-six slots, two words each, and one of them is where the machine starts. Work out what .org number a `PCINT0` handler needs before you are told. Then what happens, cycle by cycle: the hardware pushes a program counter and clears one flag, and everything else is yours to do. This is also where the simulator stops being the authority, which is worth knowing about before you trust a measurement.

Then one vector, eight pins: pin change interrupts do not tell you which pin changed, or which way it went, and the driver has to work both out, and the second one it cannot. Then the shared variable, a counter written by a handler and read by the main loop, and the window in which it holds a value it never held. And finally the driver: six subroutines, a structure that is the LED's plus two fields, and a handler that composes it with Chapter 2's LED driver.

Finish Chapter 2 first. The button driver's structure is the LED's with two fields added, and the handler calls `led_toggle`.

Two predictions are worth making before anything runs:

- how long your handler will keep interrupts disabled.
- whether that number is the same each time it runs.

The second is this chapter's cross-check, and the answer is no, twice over.

3.1 Interrupts and the Vector Table

3.1.1 What an interrupt is on this machine

An interrupt is a jump you did not write, taken between two instructions you did.

That is the whole of it. There is no scheduler, no thread, no context to switch. Some piece of hardware sets a flag, the CPU finishes whatever instruction it was executing, pushes the program counter, and jumps to a fixed address. Your handler runs, executes `ret i`, and the program counter comes back off the stack. The interrupted code has no way to know it happened.

Everything difficult about interrupts follows from that last sentence. Code that can be suspended at any instruction boundary and resumed later is code you can no longer reason about one line at a time, and §3.2 is about what to do instead.

3.1.2 The vector table

The first 52 words of program memory are not yours. They are a table of 26 slots, one per interrupt source, and the hardware jumps to a fixed slot for each.

Used in this course

0x0000	RESET	0x001A	TIMER1_OVF
0x0002	INT0	0x001C	TIMER0_COMPA
0x0004	INT1	0x001E	TIMER0_COMPB
0x0006	PCINT0	0x0020	TIMER0_OVF
0x0008	PCINT1	0x0022	SPI_STC
0x000A	PCINT2	0x0024	USART_RX
0x000C	WDT	0x0026	USART_UDRE
0x000E	TIMER2_COMPA	0x0028	USART_TX
0x0010	TIMER2_COMPB	0x002A	ADC
0x0012	TIMER2_OVF	0x002C	EE_READY
0x0014	TIMER1_CAPT	0x002E	ANALOG_COMP
0x0016	TIMER1_COMPA	0x0030	TWI
0x0018	TIMER1_COMPB	0x0032	SPM_READY

Word addresses, not byte addresses. Each slot is two words wide, so vector n is at $2n$, and PCINT0 is at 0x006 rather than at 0x03. Halving these puts every handler in a program at the wrong address at once, which produces behaviour strange enough that people blame the chip.

Figure 3.1 The 26 ATmega328P interrupt vectors with their word addresses, from RESET at 0x0000 to SPM_READY at 0x0032, with the ones this book uses marked.

Four things about that table are worth having straight before you write anything.

RESET is in it. Vector 0 is the address the machine starts at, which is why a program's first instruction has to be a jump to your code rather than the beginning of it: everything from word 0x0002 upwards belongs to the table.

Each slot is two words, so vector n is at word $2n$. PCINT0 is vector 3 and lives at word 0x0006, not 0x0003. Two words, because a `jmp` on a part with 32 KB of flash is a two-word instruction, and the table has to hold one even though a hand-written table usually puts a one-word `rjmp` there and leaves the second word unused.

This book numbers the slots from zero, so that RESET is vector 0 and the arithmetic is $2n$ with nothing added. The datasheet's own table numbers them from **one**, so what it calls "Vector No. 4" is PCINT0, which is vector 3 here. The addresses are the same either way and only the index differs; the offset is worth knowing before you check one against the other.

A slot holds a jump, not a handler. Two words is room for one instruction and nothing else, so what goes in a slot is `rjmp` to wherever your handler actually is. There is one exception, and it is a useful trick: a handler that fits in a single instruction can go directly in its slot, and `reti` alone is a legitimate handler that does nothing but acknowledge the interrupt.

The units already agree. The datasheet's table is in words and AVRASM2 counts `.org` in words, so the datasheet's number is the number you write: PCINT0 is at 0x0006 in the table and `.org 0x0006` in your program. Nothing converts.

Better still, you need not write the number at all. `m328Pdef.inc` names every vector:

```
.org PCI0addr
    rjmp isr_pcint0
```

PCI0addr is 0x0006, WDTaddr is 0x000c, OC1Aaddr is 0x0016, and the device file is the one place any of them is written down. A table built from these names cannot drift from the datasheet, because it *is* the datasheet.

Assembly written for `avr-gcc` counts `.org` in **bytes**, so the same PCINT0 slot is reached by `.org 0x000C` there. If you are reading AVR assembly from elsewhere and every handler seems to sit at twice its address, that is the dialect you are looking at, not a bug (§6.3.3 puts the two side by side).

And a C program does not write a table at all. Where `avr-gcc` links the program, `avr-libc` supplies the whole vector table and a macro fills one slot in:

```
ISR(TIMER1_COMPA_vect) { /* your handler */ }
```

That builds the same two words at the same address. The names are the difference: `avr-libc` spells the slots `TIMER1_COMPA_vect` and `PCINT0_vect` where `m328Pdef.inc` spells them `OC1Aaddr` and `PCI0addr`, and one file never has both. `ISR` also writes the handler's prologue and epilogue for you, saving what the compiler used and ending in `reti`, which is convenient right up to the point where you need to know exactly what was saved. `ISR(vect, ISR_NAKED)` gives you the slot and nothing else: no prologue, no epilogue, no `reti`, and every one of §3.2.2's rules back in your hands. That is the form a kernel's tick handler takes, and it is worth recognising now even though nothing in this book links a C program to a vector.

Build this table yourself before reading it again, in both units. The word address is the vector number times two, and the byte address is twice that again; the byte addresses are what the simulator's program counter holds, so you will want them the moment a test reports where something actually landed.

3.1.3 The one flag that controls all of them

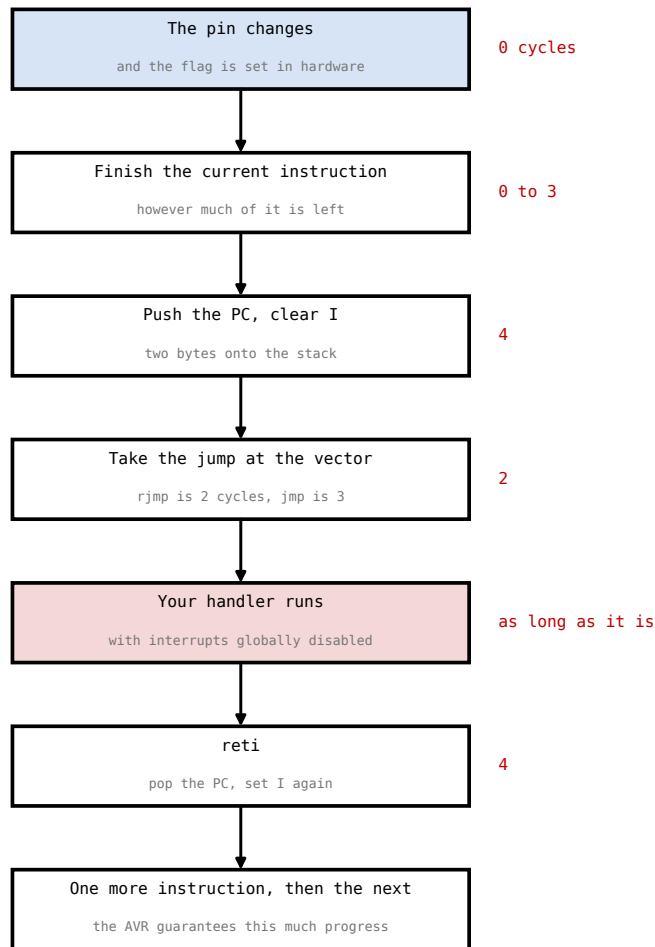
Bit 7 of SREG, the I bit, is the global interrupt enable (§1.1.3). With it clear, no interrupt is served, whatever else is configured. `sei` sets it and `cli` clears it.

Enabling an interrupt therefore takes **two** switches, and forgetting either produces a program that runs perfectly and never interrupts:

- the peripheral's own enable, which for a pin change interrupt is two more bits still (§3.1.5);
- the global I bit.

I is not a convenience. The hardware clears it on entry to any handler and `reti` sets it again, which is what stops a handler from being interrupted by the same interrupt that started it, and what makes the blocked window in §3.2.5 a real quantity rather than a worry.

3.1.4 What happens, in order



Cycle counts from the ATmega328P datasheet, not measured: the simulator finishes the current instruction as the device does, but charges nothing for the four-cycle push. So the shortest possible response is six cycles, and the longest is nine plus your handler.

Figure 3.2 The interrupt sequence as seven stacked steps, from the pin changing, through finishing the current instruction, pushing the program counter, the vector jump, the handler and reti, to one guaranteed main-program instruction, with the datasheet's cycle cost beside each.

Two of those steps deserve more than the box gives them.

“Finish the current instruction” is why latency is a range rather than a number. The hardware does not abandon an instruction half-done, so if the flag is set just as a four-cycle `ret` begins, three more cycles pass before anything else happens. The shortest possible response is therefore **6 cycles** with an `rjmp` in the table, and the longest is **9**.

Both of those numbers assume an `rjmp` in the slot, which is what this book writes. Four cycles for the hardware’s push, plus two for the jump, plus nought to three for the instruction being finished. A table built with `jmp`, which is what `avr-gcc` emits because it has to reach anywhere in flash, costs a cycle more at each end and the range is **7 to 10**. The vector jump is a choice you make, so the entry latency is partly yours, and quoting one range without saying which table it belongs to is how the two figures end up in the same argument.

“One more instruction, then the next” is a guarantee, not an accident. After `reti`, the AVR always executes at least one instruction of the interrupted program before serving another pending interrupt. Without that rule, a program under a steady stream of interrupts could make no progress at all while appearing to run.

What the simulator is and is not good for

Every other cycle count in this book was measured. These were not, and the reason is worth stating plainly rather than hiding.

`simavr` gets half of this right. It finishes the instruction it was executing before it takes an interrupt, exactly as the device does: a flag that is set during a four-cycle `ret` waits up to three cycles in the simulator as it would on the chip, which is also why Chapter 5’s own cross-check can explain a gap that alternates between 5332 and 5334 cycles by the two-cycle `rjmp` the flag had to wait for. What it does not do is charge for the entry itself. It pushes the program counter and jumps to the vector in no time at all, where the datasheet says the push takes **4 cycles**, so every interrupt in the simulator reaches its handler four cycles sooner than it would on the device. For that one quantity the datasheet is the authority and the simulator is not, and the answer is arrived at by reading rather than by measuring.

That is not a complaint about `simavr`, which is exact about the thing this book leans on hardest: what an instruction costs. It is a reminder that a tool is trustworthy about specific things, and that “I measured it” is only an argument when you know the tool models what you measured.

3.1.5 Pin change interrupts

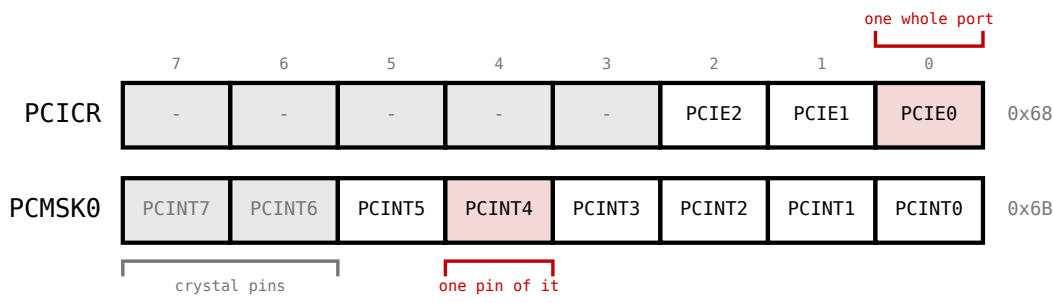
The ATmega328P has two kinds of external interrupt, and the difference matters.

`INT0` and `INT1` are two specific pins, each with its own vector, and each can be told to fire on a rising edge, a falling edge, any change, or a low level. They are precise and there are two of them.

Pin change interrupts cover 23 pins, which is very nearly every I/O pin on the part, including the two that `INT0` and `INT1` already have, so a pin can have both kinds. They are a much blunter instrument:

- **One vector per port**, not per pin. All eight pins of port B share `PCINT0`.

- **Any change fires it.** There is no edge selection: pressed and released both interrupt.
- **It does not tell you which pin.** The handler is given nothing at all beyond the fact that something on that port changed.



Enabling one pin takes both: PCICR turns on the port's single vector, and PCMSK0 says which of its eight pins may raise it. Neither is reachable by in or out.

Figure 3.3 PCICR and PCMSK0 as their bits: PCIE0 is the whole-port enable, PCINT4 the single pin, and PCINT7 and PCINT6 are the crystal pins.

Enabling one pin takes both registers:

Register	Address	What it does
PCICR	0x68	One bit per port: PCIE0 for B, PCIE1 for C, PCIE2 for D.
PCMSK0	0x6B	One bit per pin of port B. PCMSK1 and PCMSK2 are C and D.

Both are at 0x60 and above, which is **extended I/O**, so in and out cannot reach them and lds and sts are the only way in (§1.1.5). This is the first place in the book where that distinction stops being trivia and starts being an assembler error.

So a handler for two buttons on one port has to ask each of them whether it was the one. That is exactly what btn_pressed is for, and it is why the driver stores a structure per button rather than a bit per port.

And there is one question the handler cannot answer at all: **which way the pin went**. The interrupt fires on press and on release, and by the time the handler reads the pin the answer it gets is the pin's state now, which for a fast enough press may already be the other one. The usual fix is to compare against the previous state, kept in a variable, which is a variable shared between a handler and everything else, which is §3.2.

3.2 The ISR Contract, and Atomicity

3.2.1 What the hardware saves

The program counter. That is the complete list.

Not SREG. Not any register. Not the pointer registers. Everything else your handler touches, it destroys, and the interrupted code has no idea it was interrupted and will carry on using whatever it had.

This is the difference between an interrupt handler and a subroutine, and it is not a small one. A subroutine has a caller who agreed to a contract: r18 to r27, r30 and r31 are clobbered, and a caller who wanted one kept saved it first (§2.2.3). A handler has no caller. It interrupts code that agreed to nothing, could not have agreed to anything, and does not know it happened.

So a handler must save everything it uses, and restore all of it. Not the call-saved half: everything.

3.2.2 SREG first, and SREG last

The subtlest part is not a register at all.

The interrupted code may have been between a `cp` and its `brne` (§1.1.3). The flags are live at that instant, in the strongest sense: they are the entire result of the comparison, and they have one instruction left to live. A handler that executes any arithmetic at all overwrites them, and the branch afterwards then tests the handler's flags instead of the comparison's.

The symptom is a branch that goes the wrong way occasionally, and only under load, and never when you are watching. It is among the least pleasant bugs this book can produce.

So a handler's prologue and epilogue have a fixed shape, and the order is not negotiable. On the way in:

1. **Push a register**, one you are willing to work in.
2. **Read SREG into it** with `in`, before anything has had a chance to change the flags.
3. **Push that too**, so the flags are on the stack.

Then your handler does whatever it likes. On the way out, the mirror image:

4. **Pop the saved flags** back into the register.
5. **Write them to SREG** with `out`, before anything can disturb them again.
6. **Pop the register** back to what the interrupted code had in it.
7. **`reti`**.

Save the register before reading SREG, because `push` writes no flags but you need somewhere to put SREG and taking it costs you a register you have not saved yet. **Restore SREG before the final pop**, for the same reason in reverse: `pop` writes no flags either, so this order works, and an order that puts an arithmetic instruction between the `out` and the `reti` does not.

in and out are the right instructions here: SREG is at data space 0x5F, which is I/O address 0x3F, inside the low 64 that in and out reach, and they cost one cycle where lds and sts cost two. Two cycles saved on every interrupt, on the one path where it matters most, and two words of flash on top of that.

3.2.3 Registers, and the cost of a handler

Beyond SREG, save every register your handler writes, including any that a subroutine you call will clobber. That last part catches people: a handler that calls btn_pressed has to save r18, r19, r24 and r25, and the pointer registers X and Z as well, even though it never names r18, X or Z itself, because the subroutines it calls do.

Each push is 2 cycles and each pop is 2, so a saved register costs 4 cycles for the round trip and one byte of stack. A handler saving eight registers plus SREG makes nine pushes and nine pops: with the in and the out around them that is 19 cycles before it does anything and 19 more on the way out, 38 in all.

That is the argument for short handlers, and it is worth being precise about what “short” buys you, which is §3.2.5.

3.2.4 The shared variable

A handler and the main loop that share a variable are two pieces of code with no synchronisation between them, and on an 8-bit machine any variable wider than one byte is read and written in pieces.

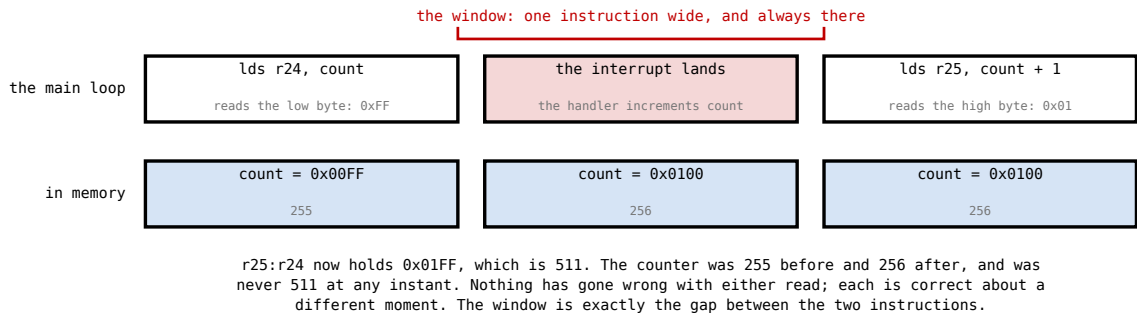


Figure 3.4 Three moments in a row: the main loop reads the low byte of a counter holding 0x00FF, the interrupt lands and the handler increments the counter to 0x0100, and the main loop then reads the high byte, assembling 0x01FF.

The main loop assembled 0x01FF, which is 511. The counter was 255 and then 256. It was never 511, at any instant, and no amount of reading the two lds instructions will show you anything wrong: each of them read the correct value of the byte it read, at the moment it read it.

The window is exactly the gap between the two instructions, which for two `lds` is 2 cycles. At 16 MHz that is 125 ns, and the chance that any one read is torn is that window times the rate at which the handler actually fires: at 1 kHz, about one read in eight thousand. Which means it happens, and it happens rarely, and it will not happen while you are watching.

The fix is to make the read atomic by preventing the interrupt during it: `cli` before the two loads and `sei` after them.

Two instructions and two cycles to close it. Note what `cli` and `sei` are doing here: they are not protecting the *counter*, they are protecting the *pair of reads*. The handler is free to change the counter at any other time.

And note the bug this fix has. That sequence unconditionally enables interrupts at the end. If the code calling it had interrupts disabled for its own reasons, they are now on. The correct version reads `SREG` into a spare register first, clears the flag, does the two loads, and writes that register back to `SREG`; the same prologue and epilogue as §3.2.2, for the same reason. It costs one cycle more, not zero: `cli` plus `sei` is 2 cycles, and in plus `cli` plus out is 3. The `cli` does not go away; only the `sei` does.

And the failure that follows from the same bug, one level up. Nest two of these. The outer one does `cli`, and part-way through it calls something that does `cli ... sei`. The inner section ends by turning interrupts *on*, in the middle of the outer section, which now runs unprotected to its end and has no way to find out. The `sei` version is not merely impolite to its caller; it is impossible to compose with itself. Save-and-restore composes, because each level hands back the flag it was given, and that is the whole reason a kernel's critical sections cannot be `cli/sei` either.

A single byte needs none of this. `lds r24, flag` is one instruction, and an interrupt cannot land in the middle of one. That is why an 8-bit flag shared between a handler and a main loop is safe and a 16-bit counter is not, and it is worth knowing which of your variables is which.

3.2.5 How long are interrupts off

The hardware clears `I` before it pushes the program counter and `reti` sets it again. So the window in which no other interrupt can be served is:

$$\text{blocked} = 4 + \text{vector jump} + \text{your handler, } \text{reti} \text{ included}$$

Four for the push, two more for an `rjmp` in the table, and then everything you wrote. Six cycles are the hardware's and the rest is yours.

On the reference implementation, the button handler measures **104 cycles when the button is not pressed and 163 when it is**, so it blocks interrupts for **110 or 169** cycles, which is 6.875 μs or 10.5625 μs at 16 MHz. Whether that is a lot depends entirely on what else is waiting; the cross-check in Exercise 3.8 is where you work it out for your own handler and check the answer against a measurement.

What being blocked costs is lateness, not loss. A pin change interrupt sets a flag, and the flag stays set until it is served, so a second interrupt arriving during your handler is

delayed rather than dropped. What *is* lost is a second change on the same port during that window: the flag is one bit, and it is already set.

3.2.6 What a handler must not do

Do not call anything long. Not because it is bad style, but because §3.2.5 is a number and calling a delay routine makes it enormous.

Do not busy-wait for anything an interrupt would have to deliver. Interrupts are off. The thing you are waiting for cannot arrive, and the machine will sit there until the watchdog (Chapter 6) rescues it, or until somebody unplugs it.

Do not sei inside a handler, unless you have thought about it very hard. It lets the same interrupt interrupt itself, and the stack grows by one frame each time. Two frames is fine. Two thousand frames is your variables overwritten from the top down, with no fault and no message (§2.2.2).

Do not assume it runs once per event. A mechanical button bounces (§2.1.5), and a pin change interrupt fires on every one of those transitions. A handler that toggles an LED on each press will run four to ten times per press, and the LED will appear to respond about half the time. That is not a fault in your handler and no amount of rereading it will help.

And do not forget it can happen between any two instructions of anything. Including between the two lds of §3.2.4, and including inside a subroutine the main loop was halfway through.

3.3 The Button Driver

3.3.1 The task

Write `drivers/source/btn.asm`, and extend `drivers/app/main.asm` with a handler.

The button driver is the LED driver's mirror image for its first half and something new for its second. Both are handed an Arduino pin number and build a structure of register addresses; where `led_init` makes the pin an output, `btn_init` makes it an input and switches its pull-up on. `btn_pressed` is `led_enabled` with its answer inverted. The new part is the four subroutines that turn a pin change interrupt on and off for one pin without disturbing the other seven that share its vector.

3.3.2 The structure

Ten bytes, as `drivers/include/btn.inc` declares.

Offset	Field	Size	Holds
0	BTN_PIN_REG	2	Data space address of PINx
2	BTN_DIR_REG	2	Data space address of DDRx
4	BTN_PORT_REG	2	Data space address of PORTx
6	BTN_PCMSK_REG	2	PCMSK0 for port B, PCMSK2 for port D
8	BTN_PCIE_BIT	1	PCIE0 for port B, PCIE2 for port D
9	BTN_PIN	1	Bit number within the port, 0 to 7
	BTN_SIZE	10	Total

The first three fields are the LED's three, at the same offsets and in the same order. That is deliberate: a button is an LED's three register pointers plus what it takes to raise an interrupt.

It is also a trap worth naming now, because nothing prevents you passing a button structure to `led_on`. Six bytes at the front are laid out identically, the three pointers, and the seventh is not: offset 6 is `LED_PIN` in one structure and the low byte of `BTN_PCMSK_REG` in the other. So `led_on` finds three valid port registers and then a pin number of `0x6B` or `0x6D`, which is the same failure as reaching for the wrong field within one structure (§3.3.5): `shift_bits` returns 0 for any argument of 8 or more, the mask is zero, and the subroutine writes nothing at all and reports success.

3.3.3 `btn_init`

Arguments: `r25:r24` is the structure address, `r22` the Arduino pin number.

Returns: `r24` is 0 on success, 1 if the pin is not one this driver accepts.

The same shape as `led_init` (§2.3.3), with two more fields to fill and the direction reversed:

1. Copy the structure pointer into `Z`.
2. Validate, choose the port, and subtract 8 for port B. Reject before writing anything.
3. Store the bit number, the three port register addresses, **the mask register address** (`PCMSK0` or `PCMSK2`) and **the PCICR bit number** (`PCIE0` or `PCIE2`).
4. Make the pin an **input**: call `shift_bits_inverted`, then read `DDRx`, and the mask, write it back.
5. Switch the **pull-up** on: call `shift_bits`, then read `PORTx`, or the mask, write it back.
6. Return 0.

Pinned details

Store the PCICR bit number before you call anything. Whatever register you loaded it into is call-clobbered, and `shift_bits` will take it (§2.2.4). The same applies to the mask register address. Fill the structure first, then configure the hardware.

Steps 4 and 5 are `led_init`'s two steps with the masks swapped. The LED clears `PORTx` and sets `DDRx`; the button clears `DDRx` and sets `PORTx`. If you find yourself writing something structurally different, one of the two is wrong.

`PCMSK0` and `PCMSK2` are at `0x6B` and `0x6D`, which is extended I/O, so nothing here can use in or out (§3.1.5).

3.3.4 btn_pressed

Arguments: r25:r24 is the structure address.

Returns: r24 is 1 if the button is down, 0 if it is up.

The same shape as `led_enabled` (§2.3.5), with one difference that is the whole reason the subroutine exists:

1. Copy the structure pointer into Z.
2. Read the pin number from `BTN_PIN` and call `shift_bits` to turn it into a mask.
3. Read `PINx` through the address in `BTN_PIN_REG`.
4. Mask the two together, and return **1 when the result is zero**.

Step 4 is inverted, and the inversion is the point. The pull-up holds the pin high until the button shorts it to ground, so `PINx` reads 0 when the button is down (§2.1.5). Every other “is it on” subroutine in this library returns 1 for a set bit. This one returns 1 for a clear one, and writing it the other way round produces a program that is exactly and reliably backwards.

Pinned details

Read `PINx`, never `PORTx`. On an input pin `PORTx` is the pull-up switch, so reading it would report what you configured rather than what the world is doing, and it would give the same answer pressed and unpressed. This is `led_enabled`’s rule with the stakes raised: there, the two registers agree because the pin is an output; here they never agree.

The branch costs a cycle on one path. As in `led_enabled`, the natural implementation ends in a conditional branch, so this subroutine costs one cycle more on one of its two answers. Here it is the **unpressed** path that is dearer, because pressed is the case that falls through. Notice it rather than avoiding it: the cross-check in Exercise 3.8 asks you to account for that cycle.

It reports the pin, not the press. A mechanical button bounces, and this subroutine has no memory and no timer, so two calls a millisecond apart during a bounce legitimately disagree (§2.1.5). What it reports is the pin, now, and that is all it promises.

3.3.5 The four interrupt subroutines

All four take the structure address in r25:r24.

`btn_enable_interrupt` sets the port’s bit in `PCICR` and the pin’s bit in the structure’s mask register. Both are read-modify-write.

`btn_disable_interrupt` clears the pin’s bit in the mask register and **leaves `PCICR` alone**. `PCICR` enables a whole port’s vector and other buttons on that port may still want it; clearing it here would silently switch off every one of them.

`btn_interrupt_enabled` returns 1 if the pin’s bit in the mask register is set, else 0.

`btn_toggle_interrupt` calls `btn_interrupt_enabled` and then one of the other two.

Pinned details

The driver does not touch the global interrupt flag. No `sei`, no `cli`, anywhere in this file. You will find AVR drivers that run `sei` at the end of their enable function, and it is convenient and it is rude: it decides on your behalf that the whole program is now ready to be interrupted, at a moment chosen by whoever happened to configure a button last. `sei` belongs in your setup code, once, after everything is configured.

`btn_interrupt_enabled` must return 1 for enabled and 0 for disabled, and that sentence is here because the classic mistake is to return them the wrong way round. Nothing about such code looks wrong: the branch is there, both constants are there, and they are swapped. Every caller then does the opposite of what it meant, and `btn_toggle_interrupt` stops toggling and starts latching, which is a symptom two steps removed from its cause.

`btn_disable_interrupt` shifts by the pin number, field at offset 9, not by anything at offset 0. Reaching for the wrong field gives a shift count of 0x23, which produces a mask of zero, which clears nothing, so a button you disabled stays enabled. And clear that bit alone: a second button on the same port has its own bit in the same mask register, and the test with two buttons is the one that checks it survives.

Preserve the structure pointer across the nested call in `toggle`. `btn_interrupt_enabled` sets `Z` to the same value `toggle` had, so `Z` survives by coincidence, and relying on that is a decision rather than a fact about the contract. Pushing `r25:r24` and popping them after is four instructions and eight cycles and needs no footnote. Then jump to the enable or disable subroutine with `rjmp` rather than calling it: it is the last thing you do, so its `ret` can be yours.

3.3.6 The handler

Add to `drivers/app/main.asm`:

A vector entry. `.org` the word address of `PCINT0`, writing `.org PCI0addr` so that the device file supplies it, then `rjmp` to your handler (§3.1.2). AVRASM2 counts `.org` in words, so the datasheet's 0x0006 is the number you write; 0x000C is the byte address and lands you in the watchdog's slot.

A handler that saves what it uses, asks the button whether it is pressed, toggles the LED if it is, restores, and executes `reti`. Its shape is the prologue and epilogue from §3.2.2, with a handful of instructions in the middle. **Call it `isr_pcint0`**, exactly, because two things outside your program look for it by that name: the measurement command in the exercises, and the test suite.

Setup that initialises the LED on pin 13 and the button on pin 12, enables the button's interrupt, and then runs `sei` **once**, after everything else. Then a main loop that does nothing.

Pinned details

Save `r24`, `r25`, `r18`, `r19`, `X` and `Z` as well as `SREG`: nine things. The handler passes a pointer in `r25:r24` and calls subroutines that clobber both, plus `r18` and `r19` inside `shift_bits`, plus `Z`, which `btn_pressed` and `led_toggle` load with their structure, and `X`, which they load with a port register's address. Every one of those has to be saved, because the interrupted code

agreed to nothing. Leaving out the four you never name yourself is the mistake that costs sixteen cycles less and a bug that appears only when the main loop happens to be using a pointer.

That list is built from what these drivers actually do, which is a decision (§2.2.4). The contract would let them clobber r20 to r23 as well, and a handler that calls code you did not write has to save those too.

The LED and the button must be on different pins. Putting both on 13 means `btn_init` makes the pin an input after `led_init` made it an output, and the LED stops working in a way that looks like the handler never runs.

`sei` goes last, in setup, once. Not in the driver, and not before the structures are built. With `sei` last nothing can interrupt setup, so the order of everything before it stops mattering. With it first, setup is safe only while no interrupt source is switched on before the structure its handler reads, and one that arrives early runs your handler against a structure that is half zeros.

This is the only part of the book a unit test cannot reach. Everything else you have written is a subroutine, and a test can call a subroutine; an interrupt has to arrive. So the suite's `app_test.cpp` loads the whole program, runs it, and changes the pin from outside, which is the only way to find out whether the vector entry points at your handler, whether `sei` ran, and whether the handler ends with `reti`. It checks no cycle counts: what your handler costs depends on what your handler saves, and measuring that is Exercise 3.8's job rather than the suite's.

3.3.7 What this driver does not do

It does not debounce. A press produces several interrupts, a few milliseconds apart, and the handler runs for every one. An LED toggled by this driver will appear to respond to about half of your presses, and that is the driver working correctly.

It does not tell you which way the pin went. The interrupt fires on press and on release, and by the time the handler reads the pin it is reading the present, not the edge. Comparing against a remembered previous state is the usual answer, and it needs a variable shared with the handler (§3.2.4).

It does not tell you which pin. One vector serves eight, so a handler with two buttons on one port has to ask both.

It does not stop you enabling an interrupt with no handler behind it. The vector then contains whatever was there, which for an unwritten slot is `0xFFFF`, and the machine executes off into unprogrammed flash and eventually restarts (Exercise 1.6, part d).

3.4 Review

What you should be able to do now

- Turn a vector name into the `.org` directive that puts a handler at it, in both word and byte addresses, and say which of the two `.org` wants and which the simulator reports.
- Say what the hardware saves on entry to a handler and what it does not, and write the prologue and epilogue that make up the difference.
- Explain why `SREG` has to be saved before anything that writes flags, and restored after.
- Configure a pin change interrupt, and say what it tells you and what it cannot.
- Identify a variable shared between a handler and the main loop, say how wide the window is in cycles, and close it.
- Predict how long your own handler blocks interrupts, and check the prediction.
- Say which parts of interrupt timing this book's simulator models faithfully and which it does not, and what you would use instead for the parts it does not.

Questions to test yourself

1. `PCINT0` is vector 3. What goes after `.org`, and what would the same line have to say if this book used `avr-gcc`?
2. An interrupt fires while a `ret` is executing. How many cycles pass before the handler's first instruction, at the earliest and at the latest?
3. Your handler does `inc r24` and nothing else. Name two things wrong with it.
4. Two buttons are on port B. One of them is pressed. Which vector fires, and how does the handler find out which button it was?
5. A handler increments a 16-bit counter and the main loop reads it with two `lds` instructions. Give a value the main loop can read that the counter has never held, and say how many cycles wide the window is.
6. Your handler measures 104 cycles. For how long are interrupts disabled, and why is the answer larger than 104?

Looking ahead

Chapter 4 is about data structures with no compiler to lay them out for you:

- `X`, `Y` and `Z` in earnest: three pointers, and the displacement addressing that makes a struct field access one instruction.
- Where the seven bytes of an LED structure should actually live, and why `RAMEND + 1` is not it.
- The stack, now that an interrupt can arrive on top of two nested calls.
- Arrays of drivers, and a subroutine that walks one.

3.5 Exercises

How to check your work. Exercises 3.5 and 3.6 are checked by this chapter's test suite: write the file at the path the specification gives, then run `make test`. The suite is cumulative and runs Chapter 1's and Chapter 2's tests too.

The rest will have worked solutions in the companion repository, published later as `lectures/L03/appendix/e_solutions.md`, and the hand calculations have short answers in Appendix A. Work each one before you read it.

No hardware is needed for any of this.

Each exercise is labelled with its kind. There is exactly one **Cross-check**, and here it is Exercise 3.8.

Exercise 3.1 The table

RECALL

- How many vectors does the ATmega328P have, how many words does each slot occupy, and how many words of program memory does the whole table therefore take?
- RESET is in the table. What follows from that about the first instruction of your program?
- A slot is two words. Give the two things that can go in one, and say which this book uses.
- The datasheet says PCINT1 is at 0x0008. What do you write after `.org`, and what is the name `m328Pdef.inc` gives that address? Assembly written for `avr-gcc` would need `.org 0x0010` for the same vector. Where does the factor of two come from?
- What is at word address 0x0007, and why is it not a place anything may jump to?

Exercise 3.2 Addresses and response times

HAND CALCULATION

- Give the vector number, word address and byte address for PCINT0, PCINT2 and WDT.
- An interrupt flag is set at the exact moment an instruction finishes, and the vector holds an `rjmp`. How many cycles pass before the handler's first instruction? Show the terms.
- The same, but the flag is set just as a four-cycle `ret` begins.
- The compiler puts `jmp` in vector slots rather than `rjmp`. Redo (b) and (c) for a `jmp` table, and say in one sentence why a compiler would choose the slower one.
- Your handler measures 104 cycles including its `reti`, and the table uses `rjmp`. For how long are interrupts disabled? The answer is not 104.

Exercise 3.3 The prologue

DESIGN

A handler is about to be written. It calls `btn_pressed` and `led_toggle`, which between them use `r18`, `r19`, `r24`, `X` and `Z`, and it uses `r24` and `r25` itself to pass a pointer.

- List every register the handler must save, and say why the list is not “the call-saved ones”.
- Write the prologue and epilogue, including `SREG`. Give the order precisely.
- Explain why the register is pushed *before* `SREG` is read, and why `SREG` is restored *before* the last pop.
- Somebody writes the epilogue as `pop r24, pop r25, out SREG, r24, reti`. Say what is wrong with it, and describe the symptom in the interrupted program.
- How many cycles do your prologue and epilogue cost together? What fraction is that of a handler whose useful work is a single `sts`?

Exercise 3.4 One vector, eight pins

DESIGN

- Two buttons are wired to Arduino pins 12 and 13. How many interrupt vectors are involved, and which?
- One of them is pressed. Describe exactly what the handler is told.
- Write, in words, how the handler works out which button it was.
- The handler wants to know whether the button was *pressed* or *released*. Explain why it cannot find out from the pin alone, and what it would need to keep in order to know.
- A third button is added on Arduino pin 3. What changes in `PCICR`, what changes in the mask registers, and how many vectors are now involved?

Exercise 3.5 The button driver

CODE

Write `drivers/source/btn.asm` to the specification in §3.3.

- Write all six subroutines.
- Run `make test`.
- Deliberately swap the two return values in `btn_interrupt_enabled`, rebuild, and note which tests fail. One of them is not about `btn_interrupt_enabled` at all. Explain why it failed, then put the code back.
- Deliberately make `btn_disable_interrupt` clear `PCICR` as well as the mask bit. Say which test catches it, and why the test with two buttons does not.

Exercise 3.6 The handler

CODE

Extend `drivers/app/main.asm` to the specification in §3.3.6.

- Write the vector entry, the handler, the setup and the main loop. Put the LED on Arduino pin 13 and the button on pin 12.
- `make build` should build `app.hex`.
- Move `sei` from the end of setup to the beginning, before the structures are built. Say whether anything goes wrong in your program as it stands, and why. Then enable the button's interrupt before `btn_init` as well: describe what could now go wrong, and say why testing it would probably not show you.
- Put the button on pin 13 as well, so the LED and the button share a pin. Predict what happens before rebuilding, then check.

Exercise 3.7 The shared counter

HAND CALCULATION

A handler increments a 16-bit counter in SRAM. The main loop reads it with two `lds` instructions, low byte first.

- The counter holds `0x00FF` and the handler runs between the two `lds`. What value does the main loop assemble? Give it in hex and in decimal, and say what the counter actually held before and after.
- How many cycles wide is the window? At 16 MHz, with the handler firing at 1 kHz and the main loop reading once per millisecond, roughly how often would you expect a torn read?
- Write the fix, and give its cost in cycles.
- The obvious fix has a bug of its own. Say what it is and write the version without it.
- The same handler also sets an 8-bit flag that the main loop reads. Does that need protecting? Say why, in one sentence about instructions.

Exercise 3.8 How long is the door shut

CROSS-CHECK

Compute it by hand, measure it, reconcile.

Your handler runs with interrupts disabled. This exercise is about finding out for how long, and about which parts of that number you can measure and which you cannot.

- By hand.** Count the cycles of your handler along the path where the button is **not** pressed: prologue, the call to `btn_pressed`, the compare, the branch taken, epilogue, `reti`. You measured `led_enabled` this way in Chapter 2, and `btn_pressed` measures the same way; if you have not measured it yet, do that now.
- The whole window.** Add the entry to your handler's own cost: four cycles for the hardware's push, two for the `rjmp` in the vector slot, then the handler, then the `reti`. **This is the one number in the chapter you cannot check by measuring**, because `simavr`

charges nothing for the hardware's four-cycle push (§3.1.4). Write it down anyway, and expect (c) to disagree.

c) **By measurement.**

```
make measure IMAGE=app \
  CALLS="led_init:0x0200:13 btn_init:0x0240:12 --drive B4=1 isr_pcint0"
```

--drive B4=1 holds the button's pin high, which is a button that is **not** pressed. IMAGE=app because a handler lives in your program, not in the driver library. This works at all because `reti` pops a return address exactly as `ret` does.

d) **Reconcile.** The two should agree. If they do not, the most likely culprits are the branch you charged one cycle instead of two, and forgetting that the measured figure already includes the `reti`.

e) **Now the other path.** Predict the pressed-path cost before measuring it:

```
make measure IMAGE=app \
  CALLS="led_init:0x0200:13 btn_init:0x0240:12 --drive B4=0 isr_pcint0"
```

State the difference between the two paths and account for every cycle of it. There are five terms and two of them are negative.

f) **The part you cannot measure.** Add the response time from §3.1.4, not forgetting the one main-program instruction it guarantees after `reti`, to get the total worst-case delay before a *second* interrupt can be served. Then set that against what the simulator does: it finishes the interrupted instruction as the device would, and then charges nothing at all for the push (§3.1.4). Note that there is no measurement to make here, because the harness offers none for this quantity, which is itself part of the answer. Say which of your sources you believe and why, and state the general rule you would apply next time a tool and a datasheet disagree.

g) **What it means.** A second button on the same port is pressed while your handler is running. Is its interrupt lost or delayed? Now the *same* button bounces twice during the window. Same question, and the answer is different. Explain both.

Exercise 3.9 Why it responds to half your presses

DESIGN

You build the program, wire a button, and press it. The LED toggles about half the time.

- Explain what is happening, in terms of the circuit rather than the code.
- Roughly how many interrupts does one press generate, and over what timescale?
- Why does “about half” make sense as an outcome, and what would the LED do if a press generated an odd number of interrupts instead?

- d) Give three ways to fix it, and say what each one costs. One of them needs something this book has not covered yet; name the chapter.
- e) Which of your three is wrong for a button and right for a rotary encoder? Say why.

Pointers, Arrays and Structs in SRAM

Data structures with no compiler to lay them out for you.

IN THIS CHAPTER

- X, Y and Z: three 16-bit pointers made out of six 8-bit registers.
- `ld`, `st`, and the post-increment and pre-decrement forms that make a walk one instruction.
- `ldd` and `std`: displacement addressing, and why X does not have it.
- `lpm`, and the address space no ordinary load can reach.
- Laying a structure out by hand: offsets, size, and what “passing by reference” costs.
- Arrays of structures, and the stride that is not one byte.
- Where in SRAM your structures should actually live, and why `RAMEND + 1` is not it.
- The stack, now that an interrupt can land on top of two nested calls.

The chapter starts with six registers and three pointers. Work out, before you are told, what distinguishes `ld r16, Z` from `ld r16, Z+` in the value it returns. The answer is nothing, and that is the point. Then a structure is seven bytes: not a type, not a record, but seven bytes at an address, and the only thing that makes them a structure is that you and your code agree on the offsets. Then an array is a stride: walking one means adding the *structure's* size to a pointer, which is not one and is not a power of two, and that turns out to matter.

Then where to put it: SRAM is 2048 bytes shared between your variables and the stack, and nothing on the device notices when they meet. And finally how deep the stack goes, a number you can compute in advance, and this chapter's cross-check is computing it and then measuring it.

Finish Chapter 3 first. This chapter's array is an array of the LED structures Chapter 2 defined, and the stack arithmetic counts Chapter 3's handler.

Two predictions are worth making before anything runs:

- how many bytes of stack Chapter 3's program uses at its worst moment.
- what happens if you store a structure at `RAMEND + 1`.

The first is the cross-check. The second is a trap that a great deal of published AVR code falls into.

4.1 The Pointer Registers

4.1.1 Six registers, three pointers

The AVR has no 16-bit registers. What it has is three *pairs* of 8-bit registers that certain instructions treat as one 16-bit address:

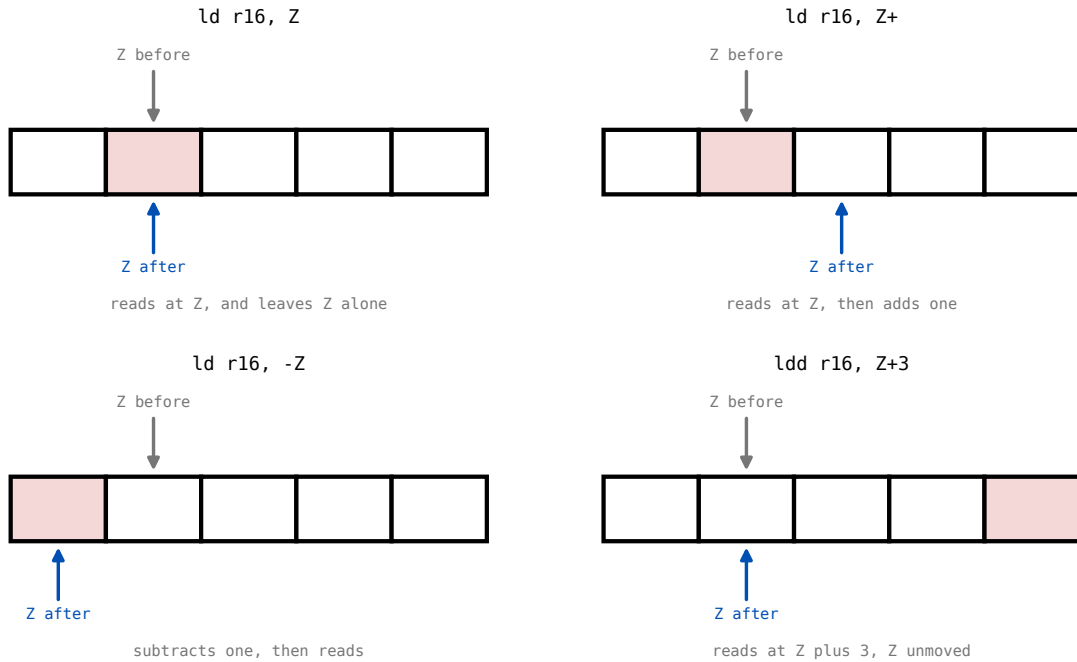
Pointer	Low byte	High byte	Displacement?
X	r26 (XL)	r27 (XH)	no
Y	r28 (YL)	r29 (YH)	yes
Z	r30 (ZL)	r31 (ZH)	yes, and <code>lpm</code> uses it

The low byte comes first, which is the same order everything else on this machine uses and the reason `movw r30, r24` copies r25:r24 into Z in one instruction: `movw` copies a pair to a pair, and both pairs are low-byte-first.

They are still ordinary registers. Nothing stops you keeping a loop counter in r30, and `ldi r30, 0` is legal because r30 is in the upper half (§1.1.2). What changes is that the moment you want to reach memory through a pointer, these six are the only ones that can, so a subroutine that has casually filled Z with something else has to put it back first.

Y is worth avoiding. r28 and r29 are call-saved (§2.2.3), so a subroutine using Y must push both halves on entry and pop them before returning: four instructions and eight cycles that X and Z cost nothing. Compiled C uses Y as a frame pointer, which is why it is call-saved and why leaving it alone is the easy choice.

4.1.2 One instruction, four ways



All four cost two cycles and all four put a byte in r16. What differs is where the byte came from and where the pointer is afterwards, and neither of those is visible in r16. Note that -Z decrements first and Z+ increments last: the sign is on the side the change happens.

Figure 4.1 The four addressing modes, one strip of memory each, showing which byte is read and where the pointer sits before and after: ld reads at Z and leaves it, ld Z+ reads then advances, ld -Z retreats then reads, and ldd Z+3 reads three along without moving Z.

All four put a byte in the destination register and all four cost **two cycles**. What differs is the byte they chose and where the pointer is afterwards, and neither of those is visible in the register you just loaded.

Form	Reads	Leaves the pointer
ld r16, Z	at Z	unchanged
ld r16, Z+	at Z	one higher
ld r16, -Z	at Z - 1	one lower
ldd r16, Z+q	at Z + q	unchanged

st is the mirror of all four, and the operand order flips: the pointer comes first when storing and second when loading. Writing them the other way round is a natural mistake and the assembler's message about it is not especially helpful.

The sign is on the side the change happens. Z+ increments *after* the access and -Z decrements *before* it, which is why walking forwards uses the first and walking backwards uses the second, and why the two are not the mirror images they look like.

4.1.3 Displacement, and its two limits

ldd reads the byte a constant number of places past Z without moving Z, and this is what makes structures cheap: a field at offset 6 is one two-cycle instruction away, not an add, a load and a subtract.

Two limits shape how you use it.

The displacement is a constant, and it is at most 63. It is encoded in the instruction, so it cannot come from a register and cannot be worked out at run time. A structure larger than 64 bytes has fields ldd cannot reach, and an array index is never a displacement.

X has no displacement form at all. There is no ldd for it; the encoding space went elsewhere. So X is the pointer for walking, Z is the pointer for structures, and a driver that needs both at once uses both, which is exactly what the LED driver does: Z holds the structure and X holds the port register address it read out of it.

4.1.4 Program memory needs its own instruction

lpm reads a byte of **program** memory through Z, and it is the only instruction that can.

Three things are different about it, and all three follow from the Harvard split (§1.1.4):

- **It costs three cycles**, not two.
- **Z holds a byte address into flash**, so reaching word address 0x0006 means putting 0x000C in Z. This is the same doubling as the vector table (§3.1.2), and it catches people twice rather than once.
- **The result lands in r0** unless you name a destination: lpm on its own writes r0, and lpm Rd, Z or lpm Rd, Z+ writes the register you asked for. One more reason to leave r0 alone.

The failure mode when you forget is silent and specific: an ordinary ld with a flash address in Z reads SRAM at that address and gets whatever is there. Nothing is checked, and a constant table you carefully placed in program memory reads back as your own variables.

This book does not use lpm for anything. It is here because the moment you want a lookup table you will need it, and because “a pointer is a pointer” is the intuition it breaks.

4.1.5 What a pointer costs

Worth having the numbers, because the reason to use displacement addressing is not elegance.

Doing this	Instructions	Cycles
Read a field at a known offset, with <code>ldd</code>	1	2
The same, by adding the offset to Z and taking it off again	3	6
Copy a structure address into Z with <code>movw</code>	1	1
Advance a pointer by 63 or less, with <code>adiw</code>	1	2
Advance a pointer by more than 63	2	2

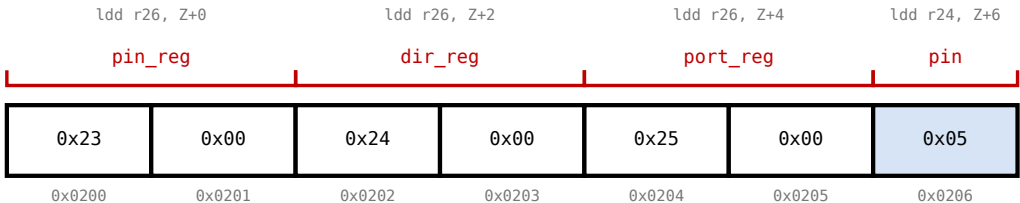
`adiw` adds an immediate to a register pair, and it has limits of its own: **the immediate is at most 63**, and it works only on `r24`, `r26`, `r28` and `r30`. Advancing by more than that takes `subi` and `sbc` with the negated value, which is two instructions and reads strangely the first time. Subtracting a negative is how you add to a pair, because there is no `addi` and there never was.

That is not a detail you need to enjoy. It is one you need to recognise, because it is what walking an array of seven-byte structures looks like.

4.2 Structures, Arrays, and Where They Live

4.2.1 A structure is an agreement

There is no `struct` here. There is an address, some bytes after it, and an agreement between your code and itself about which byte means what.



Seven bytes, and every one of them is reachable in a single two-cycle instruction because `ldd` takes a constant displacement from Z. A 16-bit field is two bytes, low byte first, so `0x23` followed by `0x00` is the address `0x0023`, which is `PINB`.

Figure 4.2 An LED structure as seven byte cells at `0x0200` to `0x0206`, holding `0x23 0x00`, `0x24 0x00`, `0x25 0x00` and `0x05`, braced into the four fields `pin_reg`, `dir_reg`, `port_reg` and `pin`, each labelled with the `ldd` that reaches it.

That is Chapter 2’s LED structure after `led_init` has run for Arduino pin 13. Four things in it are worth saying out loud, because in C the compiler would say them for you:

A 16-bit field is two bytes, low byte first. `0x23` followed by `0x00` is the address `0x0023`, which is `PINB`. Read the two bytes in the other order and you get `0x2300`, which is not memory at all, and the driver will store through it without complaint (§4.2.4).

The offsets are the interface. They are declared in `drivers/include/led.inc` for your driver, and copied into `device/include/avr/drivers.hpp` for the test suite. A structure whose fields are in different places is a different structure, however similar the code looks.

Every field is one instruction away. All four offsets are below 63, so `ldd` and `std` reach any of them directly (§4.1.3). That is the whole design: a driver copies the structure address into `Z` once and then never does pointer arithmetic again.

Nothing checks the type. A button structure begins with an LED structure's three pointers, at the same offsets, so `led_on` will accept one and find three plausible port registers in it. The fourth field is where they part: offset 6 is the LED's pin and the button's `pcmsk_reg`, so `led_on` reads a pin number of `0x6B`, shifts by that, gets a zero mask, writes nothing and reports success. There is no type system here; there is only what you passed, and the failure is silence rather than an error.

4.2.2 An array is a stride

An array of structures is a run of them, one after another, with nothing in between. To get from one to the next you add the structure's **size**, and on this device that means:

- the LED structure's stride is 7,
- the button structure's stride is 10.

Neither is one, and neither is a power of two, and both of those facts have consequences.

Adding 7 is not incrementing. Walking an array is `adiw` by the structure size, or the `subi/sbci` pair when the size is over 63. It is not `Z+`, which advances by one byte and would put the second structure's first field on top of the first structure's second field.

Indexing needs a multiply. The address of entry n is the base plus n times the stride, and $7 \times n$ is not a shift. The AVR does have a hardware `mul`, which costs two cycles and leaves its result in `r1:r0`; using it means remembering to clear `r1` afterwards, because compiled C assumes `r1` holds zero (§2.2.3). The alternative is to walk from the start, which costs a loop and no surprises.

A wrong stride is not an error. It is a second structure that overlaps the first, and the symptom is one LED that works and a second that behaves like a corrupted copy of it. There is nothing to catch it: every address involved is a legal address, and every store succeeds.

4.2.3 Passing an array

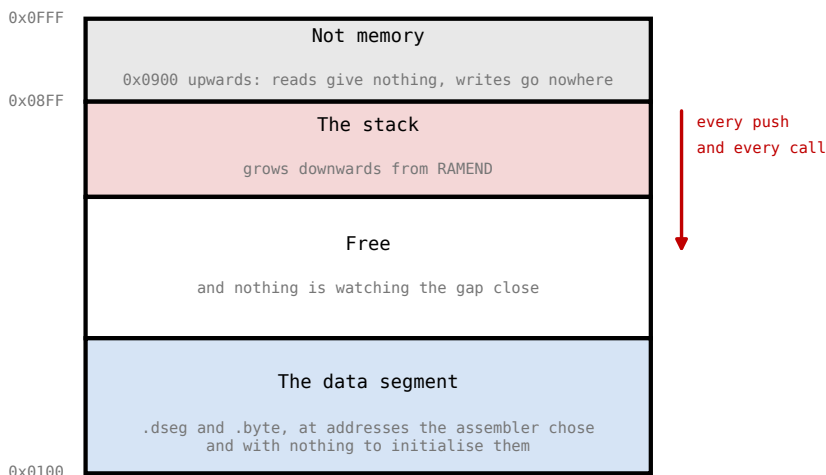
A subroutine that works on an array needs three things, and this book passes them in the argument registers the calling contract gives it: the **base address** in `r25:r24`, the **count** in `r22`, and, where there is one, an **index** in `r20`.

The count is not optional and it is not a courtesy. Nothing in memory marks where an array ends; the byte after the last structure is an ordinary byte that will read as a plausible structure. A subroutine that walks until something looks wrong walks forever.

So every loop over an array tests its count at the top, not the bottom. A count of zero is a real input, it means "do nothing", and a loop that tests at the bottom does one entry

anyway. That is the single most common array bug, it is invisible for every other count, and the test suite has a case for exactly it.

4.2.4 Where a structure may live



RAMEND + 1 is 0x0900, and on an ATmega328P it is not memory. On a device with external RAM it would be where that begins, which is why you will find AVR code allocating driver structures there. Here, storing to it writes nothing and loading from it reads nothing.

Figure 4.3 A map of SRAM with high addresses at the top: not memory above 0x0900, the stack growing down from RAMEND at 0x08FF, free space, then the data segment reserved by .byte up from 0x0100.

An ATmega328P has 2048 bytes of SRAM, from 0x0100 to 0x08FF. Above and below that are addresses that a store instruction will accept and that are not memory you may use:

Address	What it is	What a store does
0x0000 to 0x001F	the register file	changes a register, quietly
0x0020 to 0x005F	the I/O registers	reconfigures the hardware
0x0060 to 0x00FF	extended I/O	the same
0x0100 to 0x08FF	SRAM	what you wanted
0x0900 upwards	nothing	reaches no memory; nothing reliable reads back

RAMEND + 1 is the one to know about. It is 0x0900, it looks like the obvious place to start putting things, and a good deal of published AVR code allocates driver structures there, including the material this book is based on. On a device with external RAM it *is* where that RAM begins. On an ATmega328P on an Arduino Uno there is no external RAM, so

it is nothing at all: the stores are accepted and reach no memory, the datasheet does not say what a load from there returns, and the driver ends up working from values it never wrote. On a part that reads them back as zeros, that looks as though every structure were full of zeros. The simulator is stricter than the chip here: `simavr` stops the program at the first access above `RAMEND` and reports an invalid address, so under the test suite this mistake is loud rather than quiet.

The right answer is to let the assembler place them. `AVRASM2` has a data segment for exactly this: `.dseg` switches to it, `label: .byte n` reserves *n* bytes and gives you the address the assembler chose, and `.cseg` switches back. Two reservations cannot overlap, and neither occupies a byte of flash, because nothing in the data segment is emitted: it is an address allocator and nothing more.

```
.dseg
my_led:      .byte LED_SIZE
my_button:   .byte BTN_SIZE
.cseg
```

Nothing zeroes it for you. `.bss` in a C program is cleared by startup code that runs before `main`, and there is no startup code here: `avra` has no linker and no runtime, so the bytes hold whatever the SRAM powered up with. Every field a driver reads before it has written is a field you initialised or a field that is garbage. That is what `led_init` is for.

Hard-coding `0x0200` works too and is what the test suite does, because a test wants to know exactly where it put something; in a program, letting the assembler do it is one fewer number to keep consistent.

Whichever you choose, the arithmetic in §4.3 is still yours to do. Nothing on this device notices when your variables and your stack meet.

4.3 The Stack, and How Deep It Goes

4.3.1 One pointer, four users, no referee

`SP` lives in `SPH:SPL` at data space `0x5E:0x5D`, it starts at `RAMEND`, and it moves downwards. Four unrelated things use it and none of them coordinates with the others:

What	Costs
<code>rcall</code> and <code>ret</code>	2 bytes, for a return address
<code>push</code> and <code>pop</code>	1 byte each
An interrupt	2 bytes, pushed by the hardware before your handler starts
Your handler's own saves	1 byte each

`SP` points at the next free byte, not the last used one. So the number of bytes in use is `RAMEND - SP`, and the lowest byte actually occupied is one *above* where `SP` is. That off-by-

one is worth getting right once: with SP down at 0x08F0, it is the difference between “my structure at 0x08F0 is safe” and “my structure at 0x08F0 was overwritten”.

4.3.2 Counting the worst case

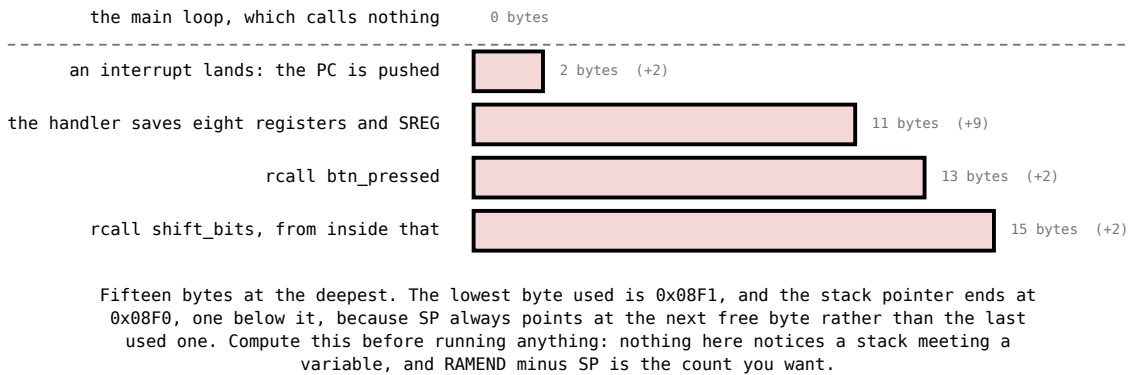


Figure 4.4 Stack use accumulating in Chapter 3’s program: nothing in the main loop, which calls nothing, two bytes when an interrupt pushes the program counter, eleven after the handler saves eight registers and SREG, thirteen and fifteen for the two nested calls inside it.

That is Chapter 3’s program at its deepest, and every number in it is measured. The rule behind them is simple enough to apply to any program:

$$\begin{aligned}
 \text{bytes} &= 2 \times (\text{calls deep in the main program}) \\
 &\quad + 2 + (\text{registers the handler saves}) \\
 &\quad + 2 \times (\text{calls deep inside the handler})
 \end{aligned}$$

The + 2 in the middle is the hardware’s, and it is there even for a handler that is nothing but `reti`. The handler’s saves are one byte each, SREG’s holder included.

The worst case is not the common case, and it is the only one worth computing. The interrupt does not arrive politely between main-loop iterations; it arrives while the main loop is as deep as it ever gets, because that is the state the program spends its time in. Assume it lands at the worst possible moment, because eventually it will.

Everything is measurable except the worst moment. You can watch the stack pointer and record how low it went, and this book’s harness does exactly that, but a measurement only tells you about the interrupts that actually arrived and where they actually landed. The arithmetic tells you about the one that has not happened yet. The cross-check in Exercise 4.8 is about the two agreeing, and about what it would mean if the measurement came out *lower*.

4.3.3 What running out looks like

Nothing.

There is no stack limit register, no guard page, no fault and no message. The stack grows down into whatever is below it and keeps going. What you observe is one of these, days later:

- a variable that changes on its own, because a push landed on it;
- a subroutine that returns somewhere it was never called from, because something wrote over a saved return address;
- a program that restarts for no reason, because that somewhere was unprogrammed flash (Exercise 1.6, part d).

None of those points at the stack, which is what makes the arithmetic worth doing in advance rather than the debugging worth doing afterwards.

Recursion is the fast way there. A handler that enables interrupts inside itself, or a subroutine that calls itself, adds a frame per repetition with nothing counting them (§3.2.6). Neither appears in this book, and both appear in real AVR code.

4.3.4 How much room is there really

2048 bytes sounds like a lot until you write the subtraction down.

Chapter 3's program uses 15 bytes of stack at its worst and 17 bytes of structures, one LED and one button, which leaves 2016 free. An array of six LEDs adds 42 bytes of structures, and a main loop that walks it goes three calls deep, `led_array_all_on` into `led_on` into `shift_bits`, so the figures move to 21 and 59, leaving 1968. Neither is close to trouble, and that is the point of computing it: you want to know *how much* headroom you have, so that when a later change halves it you notice.

Do that subtraction whenever the program grows a call level or a handler, and write the answer somewhere the next reader will find it. It is the kind of calculation that is done once, believed for a year, and quietly falsified by the third feature nobody re-checked.

4.3.5 The stack pointer is a register you may load

Everything above treats SP as something the hardware moves. It is also an ordinary pair of I/O registers, and you may read it and write it like any other.

That is worth saying plainly, because §1.4.6 writes SP once, at reset, and then explains that the hardware had already done it, which leaves the impression that touching SP is ceremonial. It is not. It is the only way to answer two questions this section has been asking on paper.

Reading it: how deep did I actually go?

```

in    r24, SPL
in    r25, SPH          ; r25:r24 = SP, and RAMEND - SP is the depth

```

Writing it: high byte first.

```

out    SPH, r25
out    SPL, r24

```

The order matters and only in one direction. SP has no shadow register, and unlike 0CR1A both halves are directly writable, so the hazard is not a latch, it is the moment in between. Write SPL first and there is one instruction during which the low half is new and the high half is old: SP points somewhere neither value describes, and an interrupt arriving in that instruction pushes two bytes there. Writing SPH first leaves the intermediate value inside the *new* region rather than in an arbitrary one. Interrupts being off makes the question moot, and writing them in the safe order costs nothing, so the rule is worth following rather than reasoning about.

These are I/O addresses, and this section has been quoting the other kind. SPL is data space 0x5D and I/O 0x3D; SPH is 0x5E and 0x3E. `in` and `out` take the I/O numbers, and `m328Pdef.inc` gives you the names, so `in r24, SPL` is what you write (§1.1.5). Reaching them with `lds` and `sts` works too and costs a cycle more each.

What this is for. Nothing in this book needs it: the drivers you are writing never move SP, and the depth figures in §4.3.2 come from the harness watching SP from outside rather than from your program reading it. It is here because of what the four instructions above become when you write them in the other order. Save every register on the stack you are on, store SP somewhere, load SP from somewhere else, restore every register from *that* stack, and `ret`, and you have returned onto a stack you were not called on. That is a context switch, it is about forty lines, and the two instruction pairs in this section are the two that make it a switch rather than a subroutine.

4.4 What to Build

4.4.1 The task

Write `drivers/source/led_array.asm`, four subroutines that treat a run of LED structures as one thing.

It is not large. What it is for is the first of the two skills this chapter is about, walking memory at a stride that is not one. The second, knowing in advance that two things sharing 2048 bytes will not meet, is arithmetic you do on paper (§4.3) and then check in the simulator (§4.6).

Creating `drivers/source/led_array.asm` is all it takes to switch its tests on.

4.4.2 The array

An array of count LED structures, laid out end to end with no gaps, starting at an address the caller chose. Entry n therefore begins $n \times \text{LED_SIZE}$ bytes after the base, and `LED_SIZE` is 7 (`led.inc`).

Nothing marks the end. The count is passed in, and it is the only thing that stops a walk.

4.4.3 `led_array_init`

Arguments: `r25:r24` the base address, `r22` the count, `r20` the Arduino pin of the first LED.

Returns: `r24` is 0 on success, 1 if refused.

Initialise count LEDs on **consecutive** Arduino pins, starting at the given one. Entry 0 gets the first pin, entry 1 the next, and so on.

Write it like this:

1. **Refuse an array that would run off the end of the pin numbering.** The last LED would be on pin `first + count - 1`, and there is no pin 14. Check this once, before initialising anything, rather than discovering it partway through and leaving half an array behind.
2. **A count of zero succeeds and does nothing.** Test the count before the first iteration.
3. **For each entry,** call `led_init` with the entry's address and the entry's pin number, then advance the pointer by `LED_SIZE` and the pin number by one.

Details that are pinned rather than up to you

`led_init` clobbers the argument registers, and it clobbers Z. Your loop variables cannot live in `r18` to `r27` or in `r30/r31` across the call, and the pointer you are walking is in `r25:r24` precisely because that is where `led_init` wants it. Pushing what you need before the call and popping it after is the straightforward answer, and it costs four cycles per register per iteration; the alternative is to keep the loop state in call-saved registers and pay to save those once, which is cheaper for a long array and more code. Either is fine. Choosing without noticing there was a choice is not.

Advance by `LED_SIZE`, taken from the include file. Not by 7 written out, and certainly not by 2 or 6, which are the sizes of the fields rather than of the structure. A stride of 6 puts each entry one byte on top of the last, and everything still looks like a structure.

Do not call `led_init` for entries beyond the count. The byte after the last structure is an ordinary byte and will be initialised as happily as any other.

Check yourself: six LEDs from Arduino pin 8 gives entries whose pin fields are 0, 1, 2, 3, 4 and 5, all on port B, at addresses seven apart. Four LEDs from pin 6 straddle the ports: two on port D at bits 6 and 7, then two on port B at bits 0 and 1.

4.4.4 `led_array_all_on`, `led_array_all_off` and `led_array_toggle_at`

`led_array_all_on` and `led_array_all_off` take the base in `r25:r24` and the count in `r22`, and call `led_on` or `led_off` for every entry. They return nothing.

led_array_toggle_at takes the base in r25:r24, the count in r22 and an index in r20. It toggles that one entry and returns 0, or returns 1 without touching anything if the index is not less than the count.

Details that are pinned rather than up to you

The bound is **index < count**, and the interesting value is **index == count**. That is the first address past the end of the array; it looks like a structure, the driver will accept it, and nothing else will notice. An implementation that checks **index <= count** passes every test with a small index in it.

Two of these three walk and one indexes. Walking is a loop and an `adiw`; indexing is a multiply, or a walk of index steps. Either is acceptable; if you use `mul`, clear r1 afterwards (§4.2.2).

Test the count at the top. A count of zero means do nothing, for all three.

4.4.5 What none of this does

It does not stop you. The arithmetic in §4.3 produces a number and nothing else. Nothing on the device checks your assembly against it, and a program whose stack collides with its variables will assemble and run exactly as happily as one whose does not.

It does not know about anything you did not count. A subroutine that pushes two registers adds two bytes that your worst case will not include unless you put them there. A bound is only as good as the depth you hand it, which is the usual bargain and is why the exercises ask you to measure as well.

And the measurement does not know about the interrupt that has not arrived yet, which is why neither on its own is enough.

4.5 Review

What you should be able to do now

- Name the three pointer register pairs and say which registers each is made of.
- Choose between `ld`, `ld Z+`, `ld -Z` and `ldd Z+q` for a given job, and say what each leaves the pointer holding.
- Say why `ldd` exists, why it is limited to a displacement of 63, and why X cannot do it.
- Read a byte from program memory, and say why it takes a different instruction and a different kind of address.
- Lay out a structure by hand and reach any field of it in one instruction.
- Walk an array of structures, and say what the stride is and why it is not the field size.
- Say which addresses in the data space a structure may occupy, and which look usable and are not.

- Compute the deepest the stack will reach in a program with interrupts, and check that it does not meet your variables.

Questions to test yourself

1. Z is r31:r30. Which is the high byte, and what does that mean for `movw r30, r24`?
2. `ld r16, Z+` and `ldd r16, Z+0` both read the byte Z points at. Give two differences.
3. Why can `ldd` reach Z+63 but not Z+64, and why is there no `ldd` for X at all?
4. An array of LED structures starts at 0x0300. Where does the fourth one start, and what is the arithmetic you would write to get there from the first?
5. A great deal of AVR code allocates driver structures at `RAMEND + 1`. What is at that address on an ATmega328P, and what happens when you store there?
6. Your program is two calls deep when an interrupt arrives, and the handler saves five registers and makes two more calls. How many bytes of stack are in use at the deepest moment?

Looking ahead

Chapter 5 is about counting time in hardware, and the arithmetic of not quite being able to:

- Counting time in hardware instead of in a loop that does nothing else.
- Prescalers, and the five ratios you actually get.
- Turning a period you want into a register value that cannot represent it exactly, and saying by how much you missed.
- A blinker that keeps time while the processor does something else.

4.6 Exercises

How to check your work. Exercise 4.5 is checked by this chapter's test suite: write the file at the path the specification gives, then run `make test`. The suite is cumulative and runs Chapter 1 to Chapter 3's tests too.

The rest will have worked solutions in the companion repository, published later as `lectures/L04/appendix/f_solutions.md`, and the hand calculations have short answers in Appendix A. Work each one before you read it.

No hardware is needed for any of this.

Each exercise is labelled with its kind. There is exactly one **Cross-check**, and here it is Exercise 4.8.

Exercise 4.1 Three pointers

RECALL

- a) Name the three pointer register pairs and the registers each is made of, and say which register of a pair holds the high byte.
- b) Which of the three has no displacement form, and which one does compiled C reserve?
- c) `movw r30, r24` copies a pointer in one instruction. Say why one instruction is enough, and what would be needed if the two pairs did not agree about byte order.
- d) Nothing stops you putting a loop counter in `r30`. Give the situation in which that turns out to be expensive.

Exercise 4.2 Four ways to read a byte

HAND CALCULATION

`Z` holds `0x0204`. The bytes at `0x0202` to `0x0208` are, in order: 11 22 33 44 55 66 77.

For each of the following, give the value loaded into `r16` and the value of `Z` afterwards:

- a) `ld r16, Z`
- b) `ld r16, Z+`
- c) `ld r16, -Z`
- d) `ldd r16, Z+3`
- e) Two of the four leave `Z` unchanged. Which, and why is that the property that makes structures cheap?
- f) All four cost the same. How many cycles, and what does that say about choosing between them?

Exercise 4.3 Which pointer to use

DESIGN

- a) A subroutine walks a buffer forwards, one byte at a time, and also has to read fields of a structure at known offsets. How many pointer registers does it need, which would you choose, and why not the third?
- b) A colleague's subroutine uses `Y` and does not save it. Say what breaks, when, and why the symptom appears in their caller rather than in their subroutine.
- c) Write down, in words, the four instructions a subroutine using `Y` has to add, and give their total cost in cycles.
- d) `ldd` cannot reach a displacement of 70. Give two ways to read that byte anyway, and say what each costs.

Exercise 4.4 Array arithmetic

HAND CALCULATION

An array of LED structures begins at 0x0300.

- a) Give the address of entries 0, 1, 2 and 5.
- b) What is the stride, and where is it declared?
- c) A colleague advances the pointer by 6 instead. Give the address their code computes for entry 2, say which byte of which real entry that is, and describe what the driver would then read as a port register address.
- d) Their code produces no error and their first LED works. Explain both.
- e) An array of *button* structures begins at the same address. Where is entry 3, and why is the answer different?
- f) How many LED structures fit in the ATmega328P's SRAM if nothing else used any of it? How many buttons? Say why neither number is the real limit on how many drivers a program can have.

Exercise 4.5 The LED array

CODE

Write `drivers/source/led_array.asm` to the specification in §4.4.

- a) Write all four subroutines.
- b) Run `make test`.
- c) Change your stride from `LED_SIZE` to 6, rebuild, and note which tests fail and which still pass. Then put it back.
- d) Move the count test in `led_array_init` from the top of the loop to the bottom, rebuild, and note which single test catches it. Explain why no other test could.

Exercise 4.6 Where to put things

DESIGN

- a) For each of 0x0005, 0x0025, 0x0068, 0x0200, 0x08FF and 0x0900, say what is there and whether a driver structure may live at it.
- b) A widely copied piece of AVR code allocates its structures at `RAMEND + 1`. Say what that address is on an ATmega328P, what a store to it does, and what the driver appears to do as a result.
- c) The same code on a different AVR, one with external RAM fitted, works. Explain.
- d) Give two ways to choose an address for a structure, and one advantage of each.
- e) Your program has one LED structure, one button structure, and an array of six LEDs. How many bytes of SRAM is that? What fraction of the total?

Exercise 4.7 How deep does it go

HAND CALCULATION

Use the rule in §4.3.2.

- a) A program with no interrupts, whose deepest path is a main loop calling a driver which calls `shift_bits`. How many bytes of stack?
- b) Chapter 3's program: a main loop that calls nothing, and an interrupt whose handler saves eight registers and SREG and calls `btn_pressed`, which itself calls `shift_bits`. How many bytes?
- c) Where does the stack pointer end up, and what is the lowest byte actually occupied? These are not the same number.
- d) A handler that is nothing but `reti` still costs stack. How much, and why?
- e) You add the LED array from Exercise 4.5, and the main loop now calls `led_array_all_on`, which calls `led_on`, which calls `shift_bits`. Redo (b).
- f) A structure is placed at `0x08F0`. Using your answer to (b), is it safe? Show the comparison.

Exercise 4.8 How deep the stack really goes

CROSS-CHECK

Compute it by hand, measure it, reconcile.

- a) **By hand.** Your answer to Exercise 4.7(b).
- b) **The lowest byte.** Turn that depth into an address: SP points at the next free byte, so the lowest byte the stack occupies is `RAMEND - depth + 1`. Do this before (c).
- c) **By measurement.** Let the program run, press the button, and read the low-water mark:

```
make measure IMAGE=app \
  CALLS="--run 4000 --drive B4=0 --run 300 --drive B4=1 --run 300"
```

--run lets the program run on its own, which is the only way to reach a handler: an interrupt cannot be called, it has to arrive. The last line reports how far down the stack pointer went.

- d) **Reconcile.** The byte count should agree with (a). The stack pointer's own value will be one below the lowest byte you computed in (b), and that is not a disagreement; say why.
- e) **Now run it without pressing anything.**

```
make measure IMAGE=app CALLS="--run 4000"
```

The measured depth is much smaller. Say what number you get, why it is smaller, and then answer the question this exercise is really about: **which of your three answers is now wrong, and which would you ship?**

- f) **The rule.** Write down, in one sentence, when a measurement of stack depth is worth more than the arithmetic and when it is worth less. Then say what a measured depth *larger* than your computed one would mean, and which of the two you would go and check first.
- g) **The margin.** Take your worst-case depth and your structures from Exercise 4.6(e). How many bytes are free? If a later change added one more level of nesting inside the handler, how much would that cost, and would you notice?

Exercise 4.9 When it goes wrong

DESIGN

- a) A wrong stride and an off-by-one loop bound are the two characteristic array bugs. For each, describe what memory looks like afterwards.
- b) Neither produces an error. Say what you would actually observe in a program with six LEDs and a wrong stride.
- c) The test suite catches both. Say what a test has to do that a program does not, in order to notice.
- d) A structure that ends at 0x08F0 and a stack that reaches 15 bytes down. Nothing fails for a week, and then it does. Give a plausible account of what changed.
- e) Name one thing you could add to the *program* that would make a stack overflow noticeable rather than silent, and say what it would cost.

Timers

Counting time in hardware, and the arithmetic of not quite being able to.

IN THIS CHAPTER

- Why a timer, and what is wrong with a loop that counts.
- The prescaler: five ratios, and the two places the ladder stops multiplying by eight.
- CTC mode, and why the period is $OCR1A + 1$ ticks rather than $OCR1A$.
- Turning a frequency you want into a prescaler and a compare value, and the error left over.
- Frequencies no timer on this device can reach, in both directions.
- Extending a timer in software, so one interrupt can drive several rates.
- A blinker that keeps time while the processor is doing something else.

The chapter starts with the counting loop, and its problem. Chapter 2 ended with an LED blinking at about 127 kHz because nothing was slowing it down. The fix is not a delay loop, and §5.1 is about why. Then the hardware, once: a counter, a prescaler, a compare register and a flag. Five paragraphs of datasheet, and then it is arithmetic for the rest of the chapter.

Then the arithmetic: given a frequency, which prescaler and which compare value, and how far off do you land. Work the table out rather than reading it; the rule is short and the boundary cases are where it lives. Then what cannot be reached: an 8-bit timer at 16 MHz cannot produce 1 Hz. Not badly: at all. And finally counting the interrupts, the software timer that fixes it, and that lets one hardware timer drive four events at four rates.

Finish Chapter 4 first. The timer's compare match is an interrupt like any other and its handler needs the same prologue, and its software timers are structures in SRAM, laid out and reached the way Chapter 4's were.

Two predictions are worth making before anything runs:

- what compare value gives exactly 1 kHz, and whether “exactly” is really exact.
- whether two consecutive interrupts are the same number of cycles apart.

The second is this chapter's cross-check, and the answer is: sometimes.

5.1 Timers

5.1.1 What is wrong with counting

Chapter 2 ended with an LED blinking about 127,000 times a second, because nothing was slowing it down (Exercise 2.9). The obvious fix is a loop that counts to a large number and does nothing, and it works.

It is still the wrong answer, for three reasons that are worth separating.

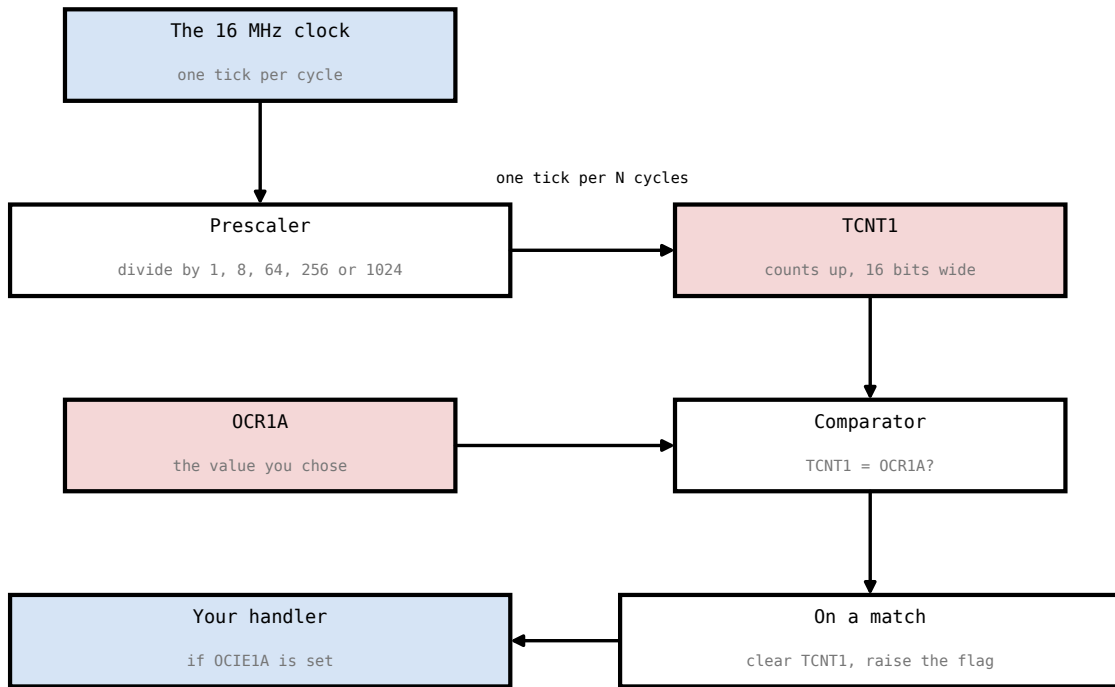
The processor is doing nothing, and it is doing it at full speed. Sixteen million cycles a second spent decrementing a register. Anything else the program wants to do has to be interleaved into that loop by hand, and every such thing changes the delay.

The delay is only right if you counted correctly. A counting loop's duration is an arithmetic claim about instructions, and every edit to the loop invalidates it (§1.3).

And an interrupt makes it wrong anyway. A handler that fires during the loop adds its own cycles to the delay, at a moment nothing predicts (§3.2.5). The more your program does, the less accurate its delays become, which is precisely backwards.

A timer has none of those problems, because the counting is not done by the processor.

5.1.2 The hardware



Everything above the handler happens in hardware, whether or not any code is running. That is the whole argument for a timer over a counting loop: the counting is not done by the processor, so the processor is free to be doing something else.

Figure 5.1 Timer1 as a block diagram: the 16 MHz clock into a prescaler that divides by 1, 8, 64, 256 or 1024, into the 16-bit counter TCNT1, compared against OCR1A; on a match the counter is cleared, a flag is raised and the handler runs if OCIE1A is set.

Four pieces, and only two of them are yours to choose.

The prescaler divides the system clock, so the counter advances once every N cycles rather than every cycle. Five ratios are available: **1, 8, 64, 256 and 1024**. That is Timer0's and Timer1's set; Timer2 has a different and longer one, and every calculation in this chapter assumes the five (§5.1.3).

The counter, TCNT1, counts up. Timer1's is 16 bits wide, which is what makes it the useful one; Timer0 and Timer2 have 8-bit counters.

The compare register, OCR1A, holds a value you choose. Hardware compares the counter against it continuously.

On a match, in CTC mode, the hardware clears the counter and raises a flag. If OCIE1A is set in TIMSK1 and interrupts are enabled globally, that flag becomes an interrupt.

Everything above the handler happens whether or not any of your code is running. That is the whole argument: the counting continues while the processor is doing something else, including while it is inside a different interrupt handler.

5.1.3 Why the prescaler exists

A 16-bit counter at 16 MHz overflows in about four milliseconds. Anything slower than 244 Hz is out of reach without help, and the help is the prescaler.

The ratios are **1, 8, 64, 256 and 1024**, and the step between them is not constant. Write the four steps out and the shape is obvious:

1 -> 8	x8
8 -> 64	x8
64 -> 256	x4
256 -> 1024	x4

Two eights and then two fours. That matters more than it sounds. When a frequency does not fit at one prescaler, the next one up multiplies your available period by eight at the bottom of the ladder and by four at the top, so “try the next one” is not a smooth adjustment and the number of ticks you end up with jumps around.

These five are Timer0’s and Timer1’s. Timer2 selects from **1, 8, 32, 64, 128, 256 and 1024**, because it is the timer meant to be driven from a watch crystal and needs the extra steps to reach a second from 32.768 kHz. Nothing in this book uses Timer2, and mixing its ladder into any calculation below would give a wrong answer that still looks plausible, which is why `atmega328p.hpp` pins the five and no others.

A prescaler is a loss of resolution, not just a change of range. At a prescaler of 64 the counter advances once every 64 cycles, so the finest period you can express is 64 cycles. Every frequency you can reach is a multiple of that, and everything between two of them is unavailable. That is the whole of §5.2.

5.1.4 CTC mode

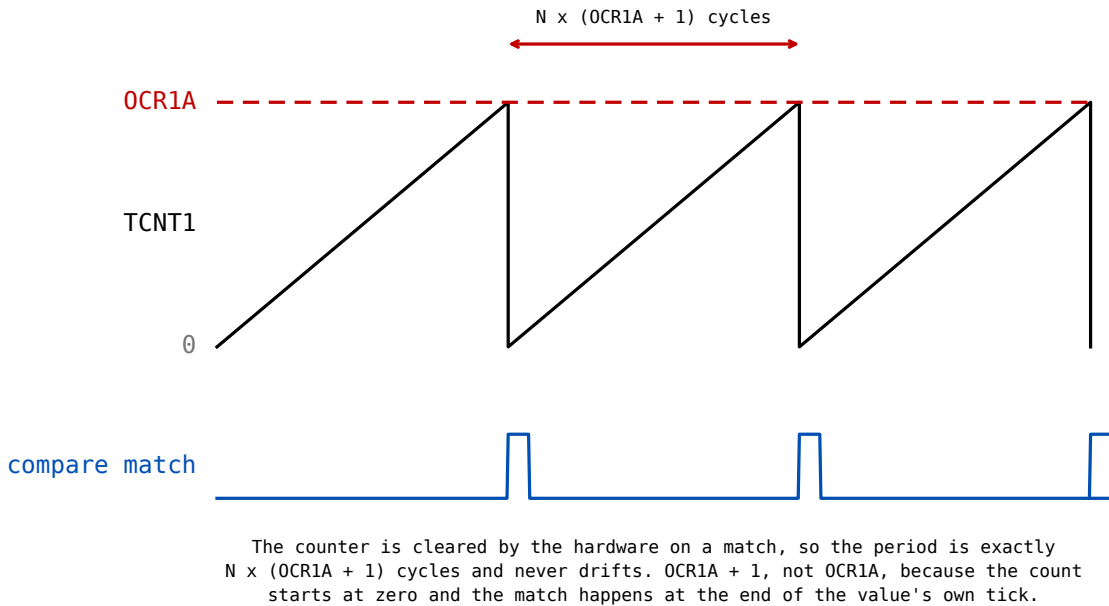


Figure 5.2 A sawtooth: TCNT1 counts up to the OCR1A line and drops straight to zero, three times, with a compare-match pulse under each drop and the period marked as $N \times (\text{OCR1A} + 1)$ cycles.

CTC is **Clear Timer on Compare**: the counter counts up to OCR1A and the hardware clears it. So the period is:

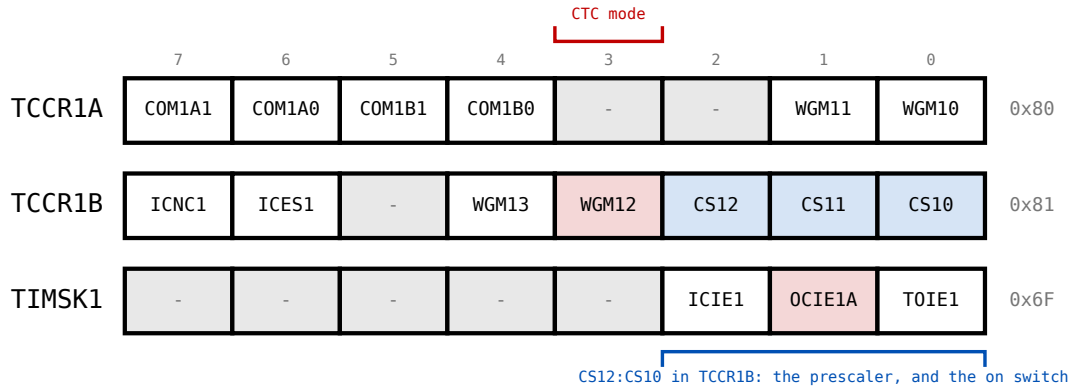
$$T = \frac{N \times (\text{OCR1A} + 1)}{f_{\text{clk}}}$$

OCR1A + 1, not OCR1A. The counter starts at zero, so counting to 249 visits 250 values. Writing the tick count itself instead of one less gives a period one tick too long, every period, which is 0.4% at 250 ticks and invisible at 16000. It is the single most common mistake in timer setup and it produces a clock that is slightly slow rather than obviously broken.

The period does not drift. The hardware restarts the count at the match, not when your handler gets round to running, so a handler that starts late makes that one event late and nothing more. A software delay has no such anchor: every cycle it runs long, and every interrupt that lands in it, is added to every period after it. A CTC period that is exactly $N \times (\text{OCR1A} + 1)$ stays exactly that for as long as the power is on, which is also why the previous paragraph matters: an OCR1A that is one out is one tick out in every period, and that does accumulate.

The alternative, **normal mode**, lets the counter run to its maximum and wrap, interrupting on overflow. The period is then fixed by the counter's width and the prescaler, and the only thing you can choose is the prescaler. It is simpler and much less useful, and it is what you fall back to on a timer with no compare register.

5.1.5 The registers



CTC with a prescaler of 64 is WGM12 set in TCCR1B and CS11 and CS10 with it. All three bits are in one byte, so a careless write to TCCR1B stops the timer.

Figure 5.3 TCCR1A, TCCR1B and TIMSK1 as their bits: WGM12 is the CTC mode bit, CS12 to CS10 are the prescaler and the on switch, and OCIE1A is the interrupt enable.

Four registers, and three cautions.

Register	Address	What you put in it
TCCR1A	0x80	Zero, for CTC with no pin output.
TCCR1B	0x81	WGM12 for CTC, and the three clock select bits.
OCR1A	0x88	The compare value, two bytes, low byte first.
TIMSK1	0x6F	OCIE1A, to let the match interrupt.

All four are at 0x60 or above, which is extended I/O, so in and out cannot reach any of them and lds and sts are the only way in (§1.1.5). Unlike the port registers, getting this wrong is an assembler error rather than a silent one.

The clock select bits are the on switch. CS12:CS10 choose the prescaler *and* whether the timer runs at all: all three zero means stopped. They live in the same byte as WGM12, so a careless write to TCCR1B that means to change the mode also stops the counter, and a timer that has stopped looks exactly like a timer whose interrupt was never enabled.

Write OCR1A high byte first. The 16-bit registers are reached through a shared temporary byte in the hardware, and the datasheet's rule is: write the high byte first, read the low byte first. Getting it backwards is not a race and is not occasional: the low-byte write is the one that commits, so it latches whatever stale value TEMP was already holding into OCR1AH, and the high-byte write that follows goes into TEMP and never reaches the register at all. It is wrong every time, deterministically, for any value whose high byte is not already correct. This book's arithmetic does not depend on it and the simulator will not punish you; a real device will.

5.1.6 What a timer does not solve

It does not make your handler punctual. The compare match is exact; the moment your code runs is not, because the interrupt still waits for the current instruction and the response still takes its four cycles (§3.1.4). The period is exact and each individual arrival is not, which is the cross-check in Exercise 5.8.

It does not give you more timers. There are three, one of them 16-bit, and the Arduino environment has already claimed Timer0 for its own timekeeping if you are sharing a board with it. Wanting four independent rates from three timers is normal, and the answer is §5.3.4.

And it does not stop while you debug. A timer keeps counting when the processor is halted at a breakpoint, so a program stepped through by hand sees a flood of pending interrupts the moment it resumes. Nothing in this book does that, and it is the first thing that will confuse you on real hardware with a debugger attached.

5.2 The Frequency You Asked For

5.2.1 The arithmetic

Given a clock, a prescaler and a frequency you want, the number of ticks in one period is:

$$\text{ticks} = \frac{f_{\text{clk}}}{N \times f}$$

and the compare value is one less than that, because the counter starts at zero (§5.1.4).

At 16 MHz, 1 kHz and a prescaler of 1: $16000000/1000 = 16000$ ticks, so OCR1A = 15999. That is exact, and $16000000/16000$ is exactly 1000 Hz.

Most frequencies are not like that.

5.2.2 Ticks are whole numbers

At 16 MHz and a prescaler of 1, 3 kHz needs $16000000/3000 = 5333.33$ ticks. The counter cannot count a third of a tick, so you get 5333 or 5334 and nothing in between.

Take 5333, so $OCR1A = 5332$. The frequency you actually get is:

$$\frac{16000000}{1 \times 5333} = 3000.1875 \text{ Hz}$$

which is **0.0063% high**. That is the error, and it is not a rounding artefact of your arithmetic; it is the closest frequency the hardware can produce.

More ticks means less error. The gap between two adjacent achievable frequencies is roughly f/ticks , so a period of 5333 ticks lands you within about 0.02% and a period of 83 ticks within about 1.2%. That single sentence is the whole reason to prefer a small prescaler and a wide counter.

5.2.3 Choosing the prescaler

The rule is: **the smallest prescaler whose tick count still fits in the counter.**

Smallest, because a smaller prescaler means more ticks per period, and more ticks means finer resolution. You move up only when forced, and you are forced when the ticks no longer fit.

Worked at 16 MHz for 1 kHz:

Prescaler	Ticks	Fits in 16 bits?	Fits in 8 bits?
1	16000	yes	no
8	2000	yes	no
64	250	yes	yes
256	62.5	yes	yes
1024	15.6	yes	yes

So Timer1 takes prescaler 1 with $OCR1A = 15999$, and Timer0 takes prescaler 64 with $OCR0A = 249$. **Same frequency, different numbers**, because the counters are different widths; both are exact here, and §5.2.4 says why.

5.2.4 What you actually get

Build this table before reading it. Every row is the same three steps, and doing them by hand seven times is what turns the rule into something you can apply to a frequency that is not here.

Wanted	Timer1, 16-bit	Timer0, 8-bit
1 Hz	/256, OCR = 62499, exact	not reachable
2 Hz	/256, OCR = 31249, exact	not reachable
10 Hz	/64, OCR = 24999, exact	not reachable
1 kHz	/1, OCR = 15999, exact	/64, OCR = 249, exact
3 kHz	/1, OCR = 5332, +0.0063%	/64, OCR = 82, +0.40%
100 kHz	/1, OCR = 159, exact	/1, OCR = 159, exact
3 MHz	/1, OCR = 4, +6.67%	/1, OCR = 4, +6.67%

Three things in that table are worth stopping on.

3 kHz misses by sixty-four times more on the 8-bit timer. Same clock, same frequency, same arithmetic; the only difference is that one has 5333 ticks to work with and the other 83.

At 3 MHz both are equally bad. With only five ticks in a period, the width of the counter stops mattering: you are choosing between 3.2 MHz and 2.67 MHz and nothing between. High frequencies are where a timer stops being a precision instrument, whatever hardware you have.

Several rows are exact. That is not an accident of these examples: 16 MHz divided by a power of two is a whole number, and the prescalers are powers of two, so frequencies that divide the clock cleanly come out exactly. It is worth knowing which of your frequencies are the lucky ones.

5.2.5 Frequencies you cannot have

Two limits, and they are limits of different kinds.

Too fast: above the clock. The shortest period is one tick at a prescaler of 1, so the fastest compare rate is the clock itself, 16 MHz. Anything above that is not “approximated badly”, it is unreachable, and asking for it should be an error rather than a number.

Too slow: past a full counter at the largest prescaler.

$$f_{\min} = \frac{f_{\text{clk}}}{1024 \times \text{counter values}}$$

At 16 MHz that is **0.238 Hz** for a 16-bit counter and **61.04 Hz** for an 8-bit one.

So an 8-bit timer at 16 MHz cannot produce 1 Hz. Not badly; at all. There is no prescaler that gets sixteen million cycles into 256 ticks, and no value of OCR0A helps, because the problem is that the counter runs out of values rather than that you chose the wrong one.

That is a real constraint on real projects, and the answer to it is not a bigger register. It is to accept an interrupt rate the hardware *can* reach and count those interrupts yourself, which is §5.3.4 and the reason this chapter has a driver at all.

5.2.6 Rounding is a decision

16000000/(1 × 3000) is 5333.33, and 5333 is nearer than 5334, so round to nearest.

That is a choice, and it should be made deliberately rather than by whichever way integer division happens to truncate. Truncating always rounds down, so it always produces a period that is too short and a frequency that is too high, systematically, in every calculation you ever do with it. Rounding to nearest is wrong half the time by half as much.

Round to nearest, with a half rounding up, and use that rule everywhere rather than choosing per calculation: one rule everywhere is one rule to check.

And note what rounding does *not* do. It does not make the answer exact, and it does not make the error small; it makes it as small as this prescaler allows. If that is not small enough, the fix is a different prescaler, a different timer, or a different clock crystal, and the exercises ask you to work out which.

5.2.7 Why the gaps alternate

Everything above is about the period the timer produces. This is about the period you can *observe*, and the two are not the same number even when the timer is exact.

Ask for 3 kHz, get $0CR1A = 5332$ and a period of 5333 cycles, then measure the gaps between successive handler entries. They come out **5332, 5334, 5332, 5334** and not one of them is 5333.

Nothing is broken. The compare match really does happen every 5333 cycles; what is quantised is the *handler's first instruction*, because an interrupt is taken at an instruction boundary and not in the middle of one (§3.1.4). A main loop of `rjmp` offers a boundary every two cycles. A period of 5333 is odd. So each match falls on the other side of a boundary from the last one, and the instant you can observe alternates by a cycle either way.

Three things follow, and the third is the useful one.

The average is exact. $(5332 + 5334)/2 = 5333$. Over any even number of periods the total is right, so nothing drifts. Jitter and drift are different failures and this is only the first.

An even period does not alternate. $0CR1A = 5331$ gives a period of 5332 and four measured gaps of 5332. That is the prediction the explanation makes, and it is worth making before measuring rather than after.

And it is not the period alone that decides it. The handler is what determines where the main loop resumes, and therefore where the next boundary falls, so its cost is a term too:

The gaps alternate when period + handler round trip is odd, and are steady when it is even. Against a two-cycle main loop, nothing else about either number matters.

Which means you can invert the whole table by adding a single `nop` to the handler. An even period that was steady starts alternating, and the odd one goes quiet. That is a strange result to arrive at from first principles and an obvious one afterwards, and it is the clearest example this book has of a measurement that is telling you about the code being interrupted rather than about the thing you were measuring.

The exercises have you reproduce the first two, and the kernel course that follows this one meets the same rule again with the scheduler's own round trip added to the sum.

5.3 What to Build

5.3.1 The task

Two things: `drivers/source/timer.asm`, which counts interrupts so that one hardware timer can drive several rates; and the hardware setup in `drivers/app/main.asm`, which uses the prescaler and compare value you worked out by hand in §5.2.

5.3.2 The structure the driver uses

Five bytes, as `drivers/include/timer.inc` declares.

Offset	Field	Size	Holds
0	TIMER_TARGET	2	Interrupts that make one event. Never 0
2	TIMER_COUNT	2	Interrupts counted since the last event
4	TIMER_RUNNING	1	0 or 1
	TIMER_SIZE	5	Total

Both counters are sixteen bits, because a target of 1000 is one second at a 1 kHz interrupt and that is an ordinary thing to want. A driver that keeps them in one byte works perfectly up to 255 and then silently does something else.

5.3.3 The idea: count the interrupts

An 8-bit timer at 16 MHz cannot reach 1 Hz (§5.2.5), and even a 16-bit one has to use its largest prescaler and most of its range to get near it. Meanwhile a program usually wants several different rates at once, and there are only three timers.

Both problems have the same answer. Set the hardware to a rate it reaches comfortably and accurately, say 1 kHz, and then **count those interrupts in software**. A target of 1000 gives an event every second; a target of 500 gives two a second; and both can exist at the same time, from the same interrupt, because each has its own structure.

What you give up is resolution: every event lands on a 1 kHz boundary, so the finest interval you can express is one millisecond. What you get is any multiple of that, from one millisecond to just over a minute, from one timer, with no extra hardware.

5.3.4 What to build in assembly: `drivers/source/timer.asm`

Five subroutines.

timer_init takes the structure address in `r25:r24` and the target in `r23:r22`. It stores the target, clears the count, leaves the timer **off**, and returns 0 in `r24`; or returns 1 without storing anything if the target is zero.

timer_on and **timer_off** take the structure address and switch it. **timer_off** also **clears the count**.

timer_enabled returns 1 if it is on, else 0.

timer_tick takes the structure address, and is what your compare-match handler calls. If the timer is off it does nothing and returns 0. Otherwise it adds one to the count; if the count has reached the target it clears the count and returns 1, and otherwise it stores the count and returns 0.

Details that are pinned rather than up to you

A target of zero is refused. A count that starts at zero and is compared against zero either fires on every interrupt or never fires, depending on where in the subroutine you compare, and neither is obviously wrong from outside. Reject the value instead of picking one.

timer_init leaves it off. A timer that starts running the moment it is created begins counting before the rest of the program is ready, and the first event arrives at a moment nobody chose.

timer_off clears the count, so switching a timer off and on again starts a whole period rather than finishing the one that was interrupted.

Fire on the target-th call, not the one before or after. With a target of 4, the fourth call returns 1 and the three before it return 0. Increment first, then compare; the other order is off by one, and shows up as every period one interrupt long, a rate 20% slow at a target of 4 and 0.1% slow at 1000, rather than as anything obviously broken.

Clear the count after firing, to zero and not to one. Otherwise every period after the first is one interrupt short.

Compare both bytes. cp then cpc on the high bytes, and note the order: the low bytes are compared first, and the carry from that comparison feeds the second. A driver that compares only the low byte fires 256 times too often for any target above 255, and passes every test with a small target in it.

There is no adiw for r19:r18. It only works on r24, r26, r28 and r30 (§4.1.5). Adding one to an ordinary pair is subi with low(-1) and sbci with high(-1), which is subtracting minus one, and reads strangely exactly once.

5.3.5 The hardware setup

In drivers/app/main.asm, add to your existing setup:

1. **Zero TCCR1A.** No pin output, no waveform generation bits from this half.
2. **Write OCR1A,** high byte first (§5.1.5).
3. **Set OCIE1A in TIMSK1.**
4. **Set TCCR1B** to WGM12 for CTC plus the clock select bits for your prescaler.
5. Leave sei where it is, at the end, once.

TCCR1B goes last, and the order is the point. Writing the clock select bits is what starts the counter (§5.1.5), so anything written after it is written to a timer that is already running. Set the compare value first and the timer counts from zero to a value that was already there; set it afterwards and the first period is whatever OCR1A happened to hold, which after reset is 0x0000: a compare match on every tick until your write lands.

And add a handler at the TIMER1_COMPB vector, which is vector 11, so .org 0x16, or .org 0C1Aaddr, which is the same number out of the device file (§3.1.2). It has the same shape as Chapter 3's: save what you use and SREG, call timer_tick on each software timer, act on the ones that returned 1, restore, reti.

Blink a second LED, not Chapter 3's. Put it on Arduino pin 8 and leave the one on pin 13 to the button: Chapter 3's integration tests run in this suite too, and one of them watches pin 13 for eight million cycles and expects nothing to change it.

Use the numbers you worked out. Work out the prescaler and compare value for 1 kHz on a 16-bit counter by hand, by the rule in §5.2.3, and write those in.

All four registers need `lds` and `sts`. They are in extended I/O and out cannot reach them.

5.3.6 What this does not do

It does not make the handler punctual, only regular. The compare match is exact to the cycle; when your code actually runs depends on what it interrupted (the cross-check in Exercise 5.8).

It does not survive a handler that is too slow. If your handler takes longer than the period, the next match arrives while you are still in the previous one, and the flag is a single bit: you lose interrupts, silently, and the software timers run slow. At 1 kHz you have 16000 cycles, which is a great deal; at 100 kHz you have 160, which is not.

And it does not know about the hardware at all. `timer_tick` counts calls. Calling it twice in one interrupt, or forgetting to call it, changes the rate and nothing detects either.

5.4 Review

What you should be able to do now

- Say what a prescaler does, name the five ratios, and say which steps between them are not a factor of eight.
- Configure Timer1 for CTC mode at a chosen prescaler, from the datasheet's bit names.
- Turn a wanted frequency into a prescaler and a compare value, by hand.
- Say why the compare value is one less than the tick count, and what going wrong there costs.
- State the fastest and slowest frequencies a given timer can reach, and say what to do about a frequency below the slowest.
- Explain why an 8-bit timer is so much worse at hitting a frequency exactly than a 16-bit one, in terms of ticks rather than of bits.
- Extend a timer in software so that one interrupt drives several events at different rates.
- Predict the number of cycles between two compare-match interrupts, and say when that number is the same every time and when it is not.

Questions to test yourself

1. Why is a delay loop a worse way to wait than a timer, given that both work?
2. The prescaler ratios are 1, 8, 64, 256 and 1024. Which steps are the odd ones, and what does that mean when a frequency does not fit at one prescaler?
3. You want 1 kHz from Timer1 at 16 MHz. Give the prescaler and the compare value, and say whether the result is exact.
4. You want the same 1 kHz from Timer0. Give the prescaler and compare value, and say why they are different numbers for the same frequency.
5. An 8-bit timer at 16 MHz cannot produce 1 Hz. Say why, in one sentence, and say what you would do instead.
6. Two compare-match interrupts are 5333 cycles apart on average, but no individual gap is 5333. What is going on, and is the timer at fault?

Looking ahead

Chapter 6 finishes the library, and connects it to the language everything else is written in:

- The watchdog: a timer whose job is to notice that your program has stopped.
- Timed write sequences, and hardware that refuses to be configured carelessly.
- Sleep modes, and what is still running in each.
- Calling your assembly from C, and C from your assembly, which is where the calling contract you have been following since Chapter 1 finally has another party to it.

5.5 Exercises

How to check your work. Exercise 5.5 is checked by this chapter's test suite: write the file at the path the specification gives, then run `make test`. The suite is cumulative and runs Chapter 1 to Chapter 4's tests too.

The rest will have worked solutions in the companion repository, published later as `lectures/L05/appendix/e_solutions.md`, and the hand calculations have short answers in Appendix A. Work each one before you read it.

No hardware is needed for any of this.

Each exercise is labelled with its kind. There is exactly one **Cross-check**, and here it is Exercise 5.8.

Exercise 5.1 Why not a loop

RECALL

- a) Give three reasons a counting loop is a worse way to wait than a timer. One of them is about the processor, one about arithmetic, and one about interrupts.
- b) Name the four pieces of hardware in the timer and say which two you choose.
- c) What does CTC stand for, and what does the hardware do on a match that normal mode does not?
- d) A delay loop and a CTC timer are each one cycle out per period. After an hour, how do the two errors compare? Say why.

Exercise 5.2 Prescaler and compare

HAND CALCULATION

At 16 MHz, for each frequency below, give the prescaler and the compare value for a 16-bit timer and for an 8-bit one, and the error in each case.

- a) 500 Hz
- b) 440 Hz
- c) 3 kHz
- d) For one of the three, both timers are exact. Say which and why it is not luck.
- e) For another, the 8-bit timer is more than an order of magnitude worse. Say which, give the ratio, and explain it in terms of ticks rather than of bits.

Check yourself: the 3 kHz row is worked in §5.2.4, and the rule that produces every row is §5.2.3.

Exercise 5.3 Choosing a prescaler

DESIGN

- a) State the rule for choosing a prescaler, and justify it in one sentence.
- b) The ratios are 1, 8, 64, 256, 1024. A frequency needs 70000 ticks at a prescaler of 1. Which prescaler do you end up at, how many ticks is that, and how much resolution did you lose compared with the one that did not fit?
- c) The four steps in that ladder are not all the same size. Say which are which, and when you would notice.
- d) Somebody suggests always using a prescaler of 1024 “to be safe”. Give two things that goes wrong.

Exercise 5.4 The registers

RECALL

- a) Name the four registers Timer1 needs for a CTC interrupt, and what goes in each.
- b) All four are at 0x60 or above. What does that rule out, and is the failure loud or quiet?
- c) TCCR1B holds both the mode bit and the clock select bits. Describe the bug that follows from that, and what it looks like from outside.
- d) The datasheet says to write a 16-bit register's high byte first. Say why, and say whether the simulator would catch you getting it wrong.

Exercise 5.5 The software timer

CODE

Write `drivers/source/timer.asm` to the specification in §5.3.4.

- a) Write all five subroutines.
- b) Run `make test`.
- c) Change `timer_tick` so that it compares before incrementing rather than after. Note which tests fail, then put it back.
- d) Change it to compare only the low bytes of the count and the target. Note which single test catches it, and say why every other test passes.

Exercise 5.6 The hardware setup

CODE

Extend `drivers/app/main.asm` to the specification in §5.3.5.

- a) Configure Timer1 for a 1 kHz compare-match interrupt, using the prescaler and compare value you worked out by hand.
- b) Add the handler at the `TIMER1_COMPA` vector. Give the `.org` address you used and say where the number came from.
- c) Initialise a second LED, on Arduino pin 8, and have the handler call `timer_tick` on a software timer with a target that blinks it at a rate you can see. Say what target you chose and why.
- d) `make build` should build `app.hex`.

Exercise 5.7 What cannot be reached

HAND CALCULATION

- a) Give the formula for the slowest frequency a timer can reach, and evaluate it at 16 MHz for both counter widths.
- b) Can a 16-bit timer produce 0.5 Hz? Give the prescaler and compare value, or say why not.
- c) Can it produce 0.1 Hz? Same question.

- d) An 8-bit timer cannot produce 1 Hz. Explain in one sentence why no value of OCR0A helps.
- e) You need 0.1 Hz and you have a 16-bit timer. Describe the arrangement you would use, and give one interrupt rate and one target that achieve it.
- f) What is the fastest compare rate any of these timers can produce, and why is asking for more not merely inaccurate?

Exercise 5.8 How far apart are two interrupts

CROSS-CHECK

Compute it by hand, measure it, reconcile.

- a) **By hand.** For a 1 kHz compare-match interrupt from Timer1 at 16 MHz, give the prescaler, the compare value, and the number of CPU cycles between two interrupts.
- b) **In cycles.** Turn your OCR1A back into a period: $\text{prescaler} \times (\text{OCR1A} + 1)$ cycles, and then into microseconds. Do this before (c).
- c) **By measurement.** Build a program whose handler toggles a pin and does nothing else: make PB0 an output, and let the handler be `sbi PINB, 0` and `reti`. Then:

```
make measure IMAGE=app CALLS="--watch B0 --run 200000"
```

--watch records every transition of a pin and reports the gaps between them, and a period is a gap. Ignore the first gap; part (h) is about why.

- d) **Reconcile.** All three should agree exactly, and they do. Say in one sentence why a timer is the one thing in this book where that is unsurprising.
- e) **Now ask for 3 kHz instead.** Compute the prescaler, compare value and period in cycles, then measure it the same way.
- f) **The discrepancy.** Your computed period is 5333 cycles. **No measured gap is 5333.** Write down what you actually measured, then answer three questions:
 - Is the timer wrong?
 - Is the average right?
 - What is quantising the measurement, given that the timer is exact?
- g) **Test your explanation.** Whatever you decided in (f), it makes a prediction about compare values that give an **even** period. Change OCR1A to 5331 and then to 5333, measure both, and say whether your explanation survived.
- h) **The first gap.** In every one of these runs the first gap is different from all the others: a few cycles longer than a period. Explain it, and say whether it would still happen on real hardware.

Exercise 5.9 When the handler is too slow

DESIGN

- a) Your handler takes 800 cycles and the compare match happens every 16000. What fraction of the processor is the handler using?
- b) The same handler, with the interrupt rate raised to 100 kHz. Now what?
- c) Describe precisely what happens when a compare match occurs while the previous handler is still running. Is the interrupt lost or delayed?
- d) Two matches occur during one long handler. Now what, and how does this differ from (c)?
- e) The software timers built on that interrupt now run slow. Say by how much, in terms of the interrupts that were missed, and say whether anything in the program could detect it.
- f) Give two ways to make the problem visible rather than silent, and say what each costs.

Watchdog, Sleep and the C ABI

Finishing the library, and connecting it to the language everything else is written in.

IN THIS CHAPTER

- What a watchdog protects against, and the much longer list of what it does not.
- The timed write sequence: hardware that refuses to be configured carelessly, and why.
- The one typo in this chapter that leaves a device resetting every sixteen milliseconds.
- Sleep modes, and which of them a timer can wake you from. It is fewer than you would like.
- The AVR C ABI in full, and where you have been following it since Chapter 1 without a second party.
- Calling your assembly from C, and reading what the compiler generates for the same job.

The chapter starts with what a watchdog is for. Read §6.1.1 and decide, before you are told, which of five failures a watchdog would catch. The answer is two of them, and a third eventually. Then the timed sequence: two writes, four cycles apart at most. Get it wrong and the register holds a value that is not merely wrong but specifically dangerous.

Then sleeping: what stops in each mode, and the two things that can always wake you. Then the other party to the contract. §6.3 is the one place in this book where somebody else's code has to agree with yours about registers, and it turns out you have been getting it right since Chapter 1. And finally the capstone: call `shift_bits`, unchanged from Chapter 1, from a C program, then read what the compiler produces for the same job and compare.

Finish Chapter 5 first. The watchdog is configured from a table of scattered bits, in the same way the timer was configured from a prescaler and a compare value.

Two predictions are worth making before anything runs:

- whether a watchdog would have caught the atomicity bug from Chapter 3.
- how many instructions the compiler needs for `shift_bits` against your nine, and what its cost comes to against your $6n + 10$.

The second is this chapter's cross-check, and the honest answer is not the flattering one.

6.1 The Watchdog

6.1.1 What it actually detects

A watchdog is a timer that resets the device unless your program keeps telling it not to. That is the whole mechanism, and everything useful and everything disappointing about it follows from that one sentence.

It detects exactly one thing: that your program stopped reaching the instruction that pets it.

So it catches a program stuck in a loop that never exits, a program blocked waiting for an interrupt that will never arrive, and a program that jumped somewhere meaningless and is executing whatever it found. Those are real failures and they are the ones hardest to catch any other way.

Here is what it does **not** catch, and the list is longer:

- **A program computing wrong answers quickly.** It is petting the watchdog on schedule and is completely broken.
- **The atomicity bug from Chapter 3.** The main loop reads a torn 16-bit value and carries on. Nothing is stuck.
- **A stack that has grown into your variables,** right up until it corrupts a return address and the program jumps somewhere that stops petting (§4.3.3).
- **A driver writing to the wrong port register.** The program runs perfectly and the hardware does the wrong thing.
- **Anything about correctness at all.**

And it introduces a failure of its own: **a program that pets the watchdog from inside the wrong place.** A handler that runs on a timer and pets the watchdog will keep petting it while the main loop is completely stuck, which converts a watchdog from a safety mechanism into decoration. Pet it from one place, in the main loop, at the point that proves the whole loop ran.

6.1.2 The two modes

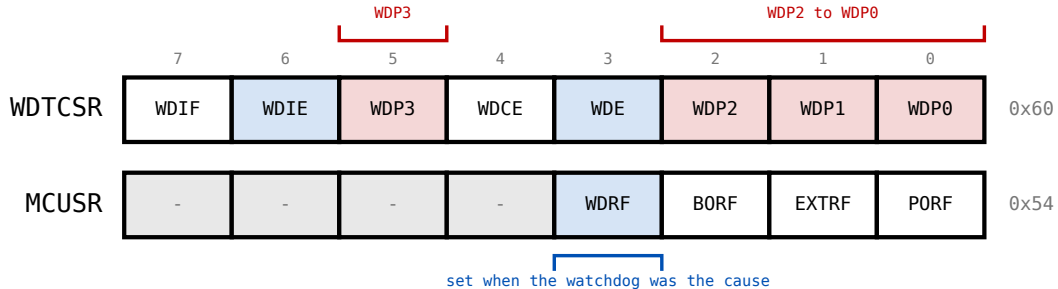
The watchdog can do one of two things when it times out, and it is worth being clear that these are different tools.

System reset, with WDE set. The device restarts as though it had been power cycled, except that SRAM keeps its contents and MCUSR records that the watchdog was the cause. This is the one you want in a product.

Interrupt, with WDIE set. The timeout raises the WDT interrupt instead, and your handler runs. Nothing is reset. This is useful for waking from sleep (§6.2) and for recording diagnostics before a deliberate reset, and it is **not** a safety mechanism, because a program too broken to pet the watchdog is usually too broken to run a handler sensibly.

Both bits can be set at once, which the datasheet calls interrupt-and-reset mode: the first timeout interrupts, and because the hardware clears WDIE on entry, the second one resets. That gives you one chance to save something before the restart.

6.1.3 The registers



The four timeout bits are not adjacent: WDP3 is bit 5, with WDCE and WDE between it and the other three. A timeout number of 9 is the byte 0x21, not 0x09.

Figure 6.1 WDTCSR drawn as its eight bits, with WDP3 at bit 5 and WDP2 to WDP0 at bits 2 to 0, and MCUSR below it with WDRF marked as the bit set when the watchdog caused the last reset.

Register	Address	Holds
WDTCSR	0x60	The timeout selector, WDE, WDIE, and the window bit WDCE.
MCUSR	0x54	Why the last reset happened: power-on, external, brown-out, or watchdog.

WDTCSR is in extended I/O and MCUSR is not, so the first needs `lds` and `sts` and the second can use `in` and `out`. Two registers in one subroutine, two addressings.

The timeout selector is four bits that are not next to each other. WDP3 is bit 5, and WDP2 to WDP0 are bits 2 to 0, with WDCE and WDE in between. The datasheet numbers the timeouts 0 to 9, and that number is not the byte you write:

Timeout	Selector	Byte
16 ms	0	0x00
125 ms	3	0x03
2 s	7	0x07
4 s	8	0x20
8 s	9	0x21

Writing 0x08 because you wanted four seconds is the most expensive typo in this chapter. 0x08 is WDE with the selector left at zero, so it means “reset me if I go quiet for sixteen milliseconds”. For most programs that is immediately, and for ever: the device resets, runs a few hundred microseconds of startup, and resets again. Reprogramming it then means winning a race against the watchdog, which is possible and unpleasant.

Nobody remembers that table, and nobody should try to. What is worth remembering is its shape: the four selector bits are not contiguous, so the byte is never the timeout number.

6.1.4 The timed write sequence

The watchdog is the one peripheral on this device that refuses to be configured carelessly, and the reason is that a program which has gone wrong must not be able to switch off the thing that would rescue it.

So changing WDE or the timeout takes two writes:

1. Write WDCE and WDE **together**. This opens a window.
2. Within **four cycles**, write the value you actually want, with WDCE clear.

Four cycles, not four instructions. Two consecutive `sts` instructions are two cycles each, so they fit with one cycle to spare: a single `nop` between them still works, and anything longer does not.

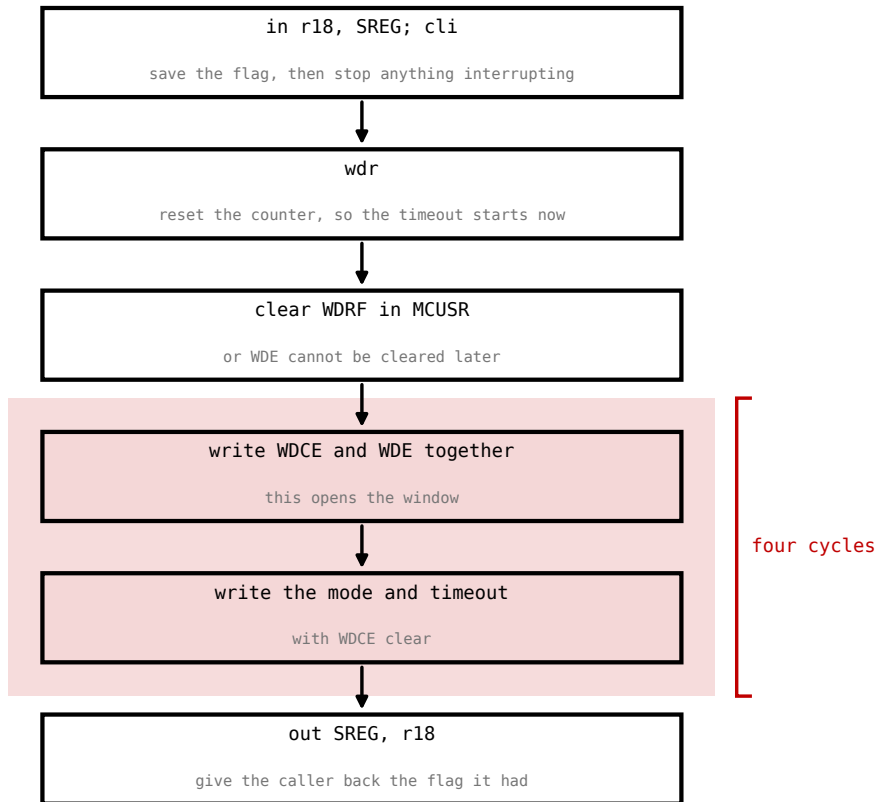
This is why the sequence starts with `cli`. An interrupt landing between the two writes takes at least six cycles to reach your handler and then runs the whole thing (§3.1.4). The window would be long gone. Turning interrupts off is not defensive style here; it is the only way the sequence can be made to work at all.

And it is why WDRF must be cleared first. The hardware will not let you clear WDE while WDRF is set in MCUSR. That rule exists so that a device which has just been reset by the watchdog cannot immediately disable it and carry on being broken. The consequence for you is specific: a driver that forgets to clear WDRF works perfectly the first time it runs and cannot switch the watchdog off after a watchdog reset, which is precisely when you most want to.

What a failed sequence leaves behind

If the second write misses its window, the hardware ignores it. WDCE clears itself after four cycles, and what remains is whatever the *first* write was allowed to change on its own: WDE is set, because setting it never needs the window, and the timeout selector is where it was, because changing it does. On a device fresh from reset the selector is zero, so the register holds 0x08.

That is system reset mode with the shortest timeout. **A sequence that is merely slightly too slow does not fail safe; it fails into system reset mode, and from reset into the most aggressive setting the register has.** The simulator models this exactly, and the test suite checks for it: if `WatchdogDriver.InitWritesTheByteAsked` ever reports WDTCSR holding 0x08, that is what happened.



The two shaded writes must be within four cycles of each other. Not four instructions: four cycles. An interrupt landing between them takes far longer than that, which is why the sequence begins by turning interrupts off and not merely by being written carefully. It ends by restoring SREG rather than by `sei`, so that a caller which had interrupts off still has them off afterwards.

Figure 6.2 Six steps in order: save SREG and `cli`, `wdr`, clear WDRF in MCUSR, then two shaded writes that must fall within four cycles of each other, then restoring SREG.

6.1.5 Choosing a timeout

Ten timeouts, from 16 ms to 8 s, roughly doubling. The choice is a genuine engineering decision with a cost in both directions.

Too short and the watchdog fires during normal operation, on the one iteration that took longer than usual. A program that resets itself occasionally under load is far worse than one that does not have a watchdog at all, because now you are debugging the watchdog as well.

Too long and a program that has stopped stays stopped for that long. Eight seconds is a long time for a machine to be doing nothing.

The rule is: **the shortest timeout that comfortably exceeds your worst-case loop time**. Worst case, not typical: the iteration that reads a sensor, and handles two interrupts, and takes the slow branch.

And the timeouts are nominal. They come from an internal oscillator that runs at roughly 128 kHz and varies with supply voltage and temperature. The datasheet’s “1.0 s” and the “1024 ms” you will find in other people’s code are the same setting described two ways, and neither is a promise. Leave real margin, and do not build anything on the exact value.

6.2 Sleep

6.2.1 What sleeping is

`sleep` is an instruction that stops the processor until something wakes it. What “something” can be, and what else stops alongside the processor, depends on the mode you selected first.

The point is power. An ATmega328P running flat out at 16 MHz draws on the order of ten milliamps; in power-down it draws microamps. For anything battery powered that is the difference between days and years, and it is available to any program whose main loop is mostly waiting.

Sleeping is not idling. A main loop that spins doing nothing is running at full speed and full current. Replacing that spin with `sleep` changes nothing about the program’s logic and everything about its power, which makes it one of the few genuinely free improvements in embedded work, provided you choose the mode correctly.

6.2.2 Setting it up

Two registers and one instruction.

SMCR holds the mode in SM2:SM0 and the enable in SE. SE must be set for `sleep` to do anything at all, and the datasheet’s advice is to set it immediately before the `sleep` and clear it immediately after, so that a stray `sleep` reached by accident does nothing.

The layout is one byte with three bits that matter and one that switches it on:

bit	7	6	5	4	3	2	1	0
	-	-	-	-	SM2	SM1	SM0	SE

SE is bit 0 and the mode occupies bits 3 to 1, which means the mode is the encoding **shifted left by one**: power-down is SM2:SM0 = 010, so the byte is 0x04, and 0x05 with SE set. Power-down written without that shift, 010 with SE beside it, is 0x03: ADC noise reduction, one mode too shallow, and the program still sleeps, which is why this is worth writing down rather than deriving each time.

SMCR is inside the I/O window, so in and out reach it, unlike the watchdog’s register: it is data address 0x53 and I/O address 0x33, and the name in m328Pdef.inc is the second (§1.1.5).

The sequence is: choose the mode, set SE, execute sleep, and clear SE when you wake. What wakes you runs its interrupt handler first, and then execution continues at the instruction after the sleep.

6.2.3 What each mode leaves running

	CPU	Timers	Watchdog	Pin change
Idle everything but the CPU	stopped	runs	runs	runs
ADC noise reduction the ADC, with the I/O clock stopped	stopped	stopped	runs	runs
Power-down the deepest one anything wakes from	stopped	stopped	runs	runs
Power-save as power-down, plus Timer2	stopped	stopped	runs	runs
Standby power-down with the crystal left running	stopped	stopped	runs	runs

The watchdog and pin change interrupts survive every mode, which is what makes them the two ways back. A timer does not survive power-down, so a program that sleeps deeply and expects a timer to wake it does not wake up.

Figure 6.3 Five of the ATmega328P’s six sleep modes against four things that might still be running: idle stops only the CPU, and the deeper modes stop the timers as well, while the watchdog and pin change interrupts run in all five. Extended standby is omitted.

The datasheet gives this as a dozen clock domains. Four of them decide whether a given program can use a given mode, and they are the four in that figure.

Idle stops the CPU and nothing else. Every peripheral keeps running and any interrupt wakes you. It saves the least and costs nothing in capability, and it is what most programs should use first.

ADC noise reduction stops the CPU and the I/O clock but leaves the ADC running, so a conversion happens with the digital noise switched off. It exists for one job and this book

does not do that job; it is in the figure because it sits between idle and power-down in the SM2:SM0 encoding and you will meet it in the datasheet's table.

Power-down stops nearly everything, including the main clock, and is where the large savings are. The consequence is the important part: **a timer cannot wake you from power-down**, because the timer's clock has stopped too.

Power-save is power-down with Timer2 left running, if Timer2 has been set up to run asynchronously from a separate crystal. That is the arrangement for a device that must sleep deeply and still keep time, and it is why Timer2 exists in the form it does. **An Arduino Uno has no such crystal**, so on the board this book assumes, power-save wakes from exactly what power-down wakes from and costs the same to leave; the mode is real and the reason to choose it is not fitted.

Standby is power-down with the crystal oscillator left running, so waking is faster. It costs more current than power-down and less than idle. **Extended standby** is the same trade applied to power-save, and the figure leaves it out for space: six modes fit in SM2:SM0 and five of them fit on a page.

So the ladder is six modes deep, from idle through ADC noise reduction, power-down, power-save and standby to extended standby, and the two that matter for anything in this book are the first and the third.

6.2.4 The two things that always wake you

A pin change interrupt, because it is asynchronous: the pin changing is itself the signal, and no clock is needed to notice it. That is the mechanism behind a device that sleeps until a button is pressed.

The watchdog, because it runs from its own oscillator and does not care what else has stopped. Configure it in interrupt mode (§6.1.2), sleep, and the timeout wakes you.

Those two together are the whole toolkit for a low-power device: **sleep until something happens, or until the watchdog says enough time has passed**. A program that wants to do something every eight seconds and nothing in between can do exactly that, and spend almost all of its life drawing microamps.

And this is the thing to get right. A program that sleeps in power-down and expects Timer1 to wake it in a second does not wake up. Nothing reports an error; the device simply sits there, drawing microamps, for ever. It is one of the more memorable ways to lose an afternoon, and it is entirely predictable from Figure 6.3.

6.2.5 What sleep does not do

It does not stop time. Or rather, it stops the timers, which is exactly the problem: a program that sleeps for a while and then reasons about elapsed time from a timer count is reasoning from a counter that was not running.

It does not save anything if you wake constantly. Waking has a cost in cycles, and in the deeper modes it is dominated by waiting for the oscillator to stabilise: a millisecond,

at 16 MHz, which §6.2.6 puts a number on. A program that sleeps and wakes a thousand times a second in power-down spends all of its time starting an oscillator, and may use more energy than one that simply idles.

It does not survive being interrupted by your own peripherals. A UART receiving in the background, a timer you forgot to stop: any enabled interrupt that is still capable of firing will wake you, and a device that mysteriously refuses to stay asleep is usually being woken by something it configured and forgot.

And it is not modelled here. This book does not test sleep, because the interaction between sleep modes, clock domains and wake sources is exactly the kind of thing a simulator approximates and a datasheet specifies. The material is here because you will need it and because the power-down trap is worth meeting on paper first; the checking is yours to do on real hardware.

6.2.6 What it costs to come back

Waking is not free in any mode, and in the deep ones the cost is almost entirely the oscillator.

Mode	SM2:SM0	Oscillator restart	At 16 MHz
Idle	000	0 cycles	0 μ s
ADC noise reduction	001	0 cycles	0 μ s
Power-down	010	16384 cycles	1024 μ s
Power-save	011	16384 cycles	1024 μ s
Standby	110	6 cycles	0.375 μ s
Extended standby	111	6 cycles	0.375 μ s

The cycle counts are pinned in `atmega328p.hpp` as `SleepModes`. Work the right-hand column out before you read it: one cycle is 62.5 nanoseconds, so it is a single multiplication, and the answer is more surprising than the arithmetic.

The 16384 is a fuse setting, not a property of the part. It is the 16K CK start-up time the SUT and CKSEL fuses select for a crystal, chosen so that the oscillator has certainly settled before any instruction executes. A part running from its internal RC oscillator pays six cycles instead, because there is nothing mechanical to settle. So the largest single term in a power-down wake-up is a decision somebody made with a fuse programmer, and it is **not in the source code**.

The two standby modes exist precisely to buy that term back: they are power-down and power-save with the oscillator left running, so waking costs six cycles rather than 16384. What they cost in return is the oscillator's current for the whole time the part is asleep, which is most of the time, which was the point of sleeping. That is the trade in its clearest form, **standby spending current all night to save a millisecond in the morning**, and it is a genuine engineering decision rather than a right answer.

Waking up is not the same event as running again, and the gap between them is not in this table. What wakes you is an interrupt, so the first thing that runs is a handler, and your

program continues at the instruction after `sleep` only once that handler has returned. On a bare program that is the handler's own cost and nothing else. Under a scheduler it is the handler, plus whichever task the scheduler then decides should run, which need not be the one that went to sleep. That is the whole of the difference between a wake-up *latency* and a wake-up *cost*, and the kernel course this book leads to spends a lecture on it.

6.3 The C ABI

6.3.1 The other party

Since Chapter 1 you have been following a calling convention (§2.2.3): arguments in `r24` and `r22`, results in `r24`, `r18` to `r27` and `r30`, `r31` free to destroy, `r2` to `r17` and `r28`, `r29` to be given back.

Up to now that convention has had one party. Every caller of your subroutines was code you wrote, so following the rules was a discipline rather than a requirement, and breaking them would only ever have surprised you.

This section is the other party arriving. The C compiler follows the same convention, so a function it generates can call a subroutine you wrote, and vice versa, with no glue and no declaration beyond a prototype. **Everything you have been doing since Chapter 1 was for this.**

6.3.2 The convention, exactly

Figure 2.6, back in Chapter 2, has the registers coloured by role; this is the rest of what it implies.

Arguments go in `r25` downwards, in pairs, left to right. The first argument takes `r25:r24`, the second `r23:r22`, the third `r21:r20`, and so on down to `r8`. An 8-bit argument takes only the lower register of its pair, so the first is `r24`, the second `r22`, the third `r20`, and the odd-numbered ones are simply not used.

That is worth stating plainly because it is the rule people guess wrong: a byte argument does **not** move up to fill the gap. Three byte arguments arrive in `r24`, `r22` and `r20`.

Results come back the same way. A byte in `r24`, a 16-bit value in `r25:r24`, a 32-bit value in `r25:r22`.

Anything larger than fits in the registers goes on the stack, and so does any argument past the last pair. Allocation runs from `r25` down to `r8`, which is eighteen registers and therefore **nine pairs**: `r25:r24`, `r23:r22`, `r21:r20`, `r19:r18`, `r17:r16`, `r15:r14`, `r13:r12`, `r11:r10`, `r9:r8`. A byte argument takes a pair and wastes the odd half rather than closing the gap, so six `uint8_t` arguments land in `r24`, `r22`, `r20`, `r18`, `r16` and `r14`. This book never comes close; a kernel's `task_init` does.

`r1` is assumed to hold zero. The compiler relies on it constantly and never checks. A subroutine that uses `r1` as scratch and does not clear it before returning breaks C code that

had nothing to do with it, at a distance, in a way that looks like a compiler bug. If you use `mul`, whose result lands in `r1:r0`, clear `r1` afterwards (§4.2.2).

`r0` is scratch for everybody and nobody expects it preserved.

6.3.3 Calling your assembly from C

Two things are needed, and neither is a wrapper.

On the assembly side, the subroutine has to be visible to a linker. Everywhere else in this book that is free, because `avra` makes every label global (§1.2.2). **Here it is not**, and that is the first of two reasons this chapter is the one place the book uses a second assembler.

On the C side, a prototype whose types match what the subroutine actually expects:

```
extern uint8_t shift_bits(uint8_t shifts);
```

That is all. The compiler puts the argument in `r24` because the prototype says it is a byte, and reads the result from `r24` for the same reason, and your subroutine has been doing exactly that since Chapter 1.

The prototype is a promise nothing checks. Declare the argument as `uint16_t` and the compiler will pass it in `r25:r24` and your subroutine will read `r24` and ignore `r25`. That gives the same answers as before, because C would have cut the argument to its low byte anyway, and it is the dangerous kind of wrong: nothing changes, so nothing tells you. Declare the result as `uint16_t` too and it stops being harmless. The caller reads `r25:r24`, `r25` still holds the argument’s high byte because your subroutine never touched it, and `shift_bits(300)` comes back as 256. There is no header, no name mangling and no link-time type check; the linker matches a name to a name.

The one place this book uses two assemblers

`avr-gcc` drives GNU `as`, and GNU `as` does not read `AVRASM2`. It cannot assemble `drivers/source/utils.asm`, and no flag makes it: the two are different languages describing the same machine. `avra`, for its part, has no linker and cannot combine anything with a compiled object file.

So for this section, and for nothing else in the book, you translate your own `shift_bits` into GNU `as` and save it as `drivers/source/utils_gnu.S`. The instructions do not change; only the directives around them do:

Your <code>utils.asm</code>	<code>utils_gnu.S</code>
<code>.include "m328Pdef.inc"</code>	<code>#include <avr/io.h></code>
<i>(nothing; avra exports every label)</i>	<code>.global shift_bits</code>
<code>.equ NAME = value</code>	<code>.equ NAME, value</code>
<code>out PORTB, r16</code>	<code>out _SFR_IO_ADDR(PORTB), r16</code>
<code>sts PORTB + 0x20, r16</code>	<code>sts PORTB, r16</code>
<code>.org 0x0006 (words)</code>	<code>.org 0x000C (bytes)</code>

`shift_bits` touches no port, defines no constant and sits in no vector slot, so in practice the translation is: change the include, add one `.global` line, and copy the body across unchanged. **The file must be `.S` and not `.s`**, because the capital is what runs the preprocessor that makes `#include` work.

The same program, twice

Here is a program that exercises every row of that table at once. It is not one of the book's exercises; it exists to be read side by side. In AVRASM2, which is what you have been writing:

```
.include "m328Pdef.inc"

.equ LED_BIT = 5

.cseg
.org 0x0000
    rjmp reset
.org PCI0addr          ; 0x0006, a WORD address
    rjmp on_pin_change

.org 0x0034             ; past the table: 26 vectors, 2 words each
reset:                 ; global already; avr exports every label
    ldi r16, (1 << LED_BIT)
    out DDRB, r16      ; DDRB is 0x04, an I/O address
    sts TCCR1B, r16     ; 0x81, already a data space address
loop:
    rjmp loop

on_pin_change:
    in r16, PINB        ; PINB is 0x03, an I/O address
    reti
```

And the same program for `avr-gcc`, which is the dialect nearly all published AVR assembly is written in:

```

#include <avr/io.h>

.equ LED_BIT, 5                /* comma, not = */

.section .text
.org 0x0000
    rjmp reset
.org 0x000C                    /* 0x0006 doubled: bytes, not words */
    rjmp on_pin_change

.org 0x0068                    /* 0x0034 doubled, for the same reason */
.global reset                  /* or nothing outside can call it */
reset:
    ldi r16, (1 << LED_BIT)
    out _SFR_IO_ADDR(DDRB), r16 /* 0x24 here; the macro subtracts 0x20 */
    sts TCCR1B, r16            /* unchanged: 0x81 in both dialects */
loop:
    rjmp loop

.global on_pin_change
on_pin_change:
    in r16, _SFR_IO_ADDR(PINB)
    reti

```

Every instruction is identical, and so is every byte. Assemble both and disassemble the results:

```

0: 33 c0      rjmp    .+102      ; to 0x68
c: 32 c0      rjmp    .+100      ; to 0x72
...                               ; the rest of the vector table

68: 00 e2      ldi     r16, 0x20
6a: 04 b9      out     0x04, r16
6c: 00 93 81 00 sts     0x0081, r16
70: ff cf      rjmp    .-2
72: 03 b1      in      r16, 0x03
74: 18 95      reti

```

The `.org 0x0034` is not padding. Without it `reset:` starts at word `0x07`, which is the second word of `PCINT0`'s slot, and the body runs straight through the slots for `PCINT1`, `PCINT2` and the watchdog. The program still works right up to the moment one of those interrupts is enabled, and then the vector jumps into the middle of your setup code. Twenty-six vectors of two words each end at word `0x33`, so `0x34` is the first word that is yours (§3.1.2).

That listing came out of both files. Nothing in the two columns of the table is a difference of *machine*; every one of them is a difference of *notation*, and the assembler resolves all of it before a single byte is emitted. `out DDRB, r16` and `out _SFR_IO_ADDR(DDRB), r16` are two ways of writing `b9 04`, and the chip has never heard of either.

Three of those rows are worth dwelling on:

- **.org PCI0addr against .org 0x000C.** The same vector slot. AVRASM2 counts words as the datasheet does, GNU as counts bytes, and the factor of two between them is the single most productive source of wrong-looking interrupt behaviour when code moves between the dialects (§3.1.2).
- **DDRB against _SFR_IO_ADDR(DDRB).** Also the same register. `m328Pdef.inc` calls it `0x04` and `<avr/io.h>` calls it `0x24`, and the macro subtracts the `0x20` back off so that `out` can reach it (§1.1.5).
- **sts TCCR1B, r16, which did not change at all.** `TCCR1B` lives above the I/O window, so both dialects give it the same data space address and neither can reach it with `out`. The registers that need translating are exactly the ones `out` can touch.

Doing this once by hand is the point. That is the whole of the difference between the dialect this book writes and the dialect most published AVR assembly is written in, and you now have a reason to read it properly (§1.2.7).

Building the two together

Hand both files to `avr-gcc`, which recognises `.S` and `.c` and does the right thing with each:

```
avr-gcc -mmcu=atmega328p -Os \
    drivers/app/main.c drivers/source/utils_gnu.S \
    -o drivers/build/mixed.elf
```

Note that this is a *different* build from the rest of the book, which assembled one unit with `avra` and never involved a linker at all (§1.2.2). A C program needs the startup code: it is what sets up the stack pointer, clears `.bss`, sets `r1` to zero, and calls `main`. All the things your `main.asm` had to do by hand.

6.3.4 Calling C from your assembly

The same convention read backwards. Put the arguments in `r25:r24` and `r23:r22`, `rcall` the function's name, and take the result from `r24`.

What changes is whose problem the registers are. **A C function will destroy `r18` to `r27` and `r30`, `r31`**, because the convention says it may, and the compiler takes full advantage. Assembly that calls C and expects Z to survive is relying on nothing.

And **the C function assumes `r1` is zero on entry**, so if your assembly has been using it, put it back before the call and not merely before the return.

6.3.5 Reading what the compiler produces

`avr-gcc -S` writes assembly instead of an object file, and it is worth doing at least once with your own code beside it.

```
avr-gcc -mmcu=atmega328p -Os -S drivers/app/main.c -o -
```

Two things are usually surprising the first time.

The compiler is better at some things than you expect. A function whose last act is to call another becomes a `jmp` rather than a `call` and a `ret`. On this part the compiler writes `call` and `jmp`, because 32 KB of flash is further than `rcall` and `rjmp` reach: the `call` and `ret` it removes cost eight cycles between them and the `jmp` that replaces them costs three, so the saving is five cycles and two bytes of stack. That is the tail call from Chapter 3's driver specification (§3.3.5), applied automatically and everywhere.

And worse at others. It does not know that a value fits in a byte when the C types say `int`, it will not use a register you know is free, and it cannot make the arithmetic decisions you made in Chapter 5. This chapter's cross-check (Exercise 6.8) compares the compiler's `shift_bits` against yours, and the result is not the flattering one you might expect in either direction.

Read `-Os` output, not `-O0`. Unoptimised AVR code spills everything to the stack and is not representative of anything anybody ships.

6.3.6 What the ABI does not give you

It does not check anything. The prototype is a promise, the linker matches names, and a mismatch is a program that runs and is wrong.

It does not make your assembly portable. The convention is `avr-gcc`'s. Another compiler for the same chip may differ, and the same compiler for a different AVR family does differ in the details of where large values go.

And it does not make mixed code easy to reason about. Every call is a boundary where one side knows what it destroyed and the other knows what it needed, and neither can see the other. That is the price of the arrangement, and the reason the convention is worth following from the first subroutine you write rather than from the first one that C calls.

6.3.7 Reading a C variable from assembly

Everything so far has been about *calls*: C hands your subroutine registers, your subroutine hands them back. There is a second half to the boundary and this book has not needed it once, because every driver here is handed the address of everything it touches. `led_on` gets a pointer in `r25:r24`; it never looks anything up by name.

Code that shares *state* with C rather than being called by it does look things up by name, and it is worth doing on paper before you need it.

Given, in some C file:

```
volatile uint8_t ticks;
volatile uint16_t deadline;
```

the assembly is:

```
lds r24, ticks           ; one byte, one instruction
inc r24
sts ticks, r24

lds r24, deadline        ; two bytes, low first, two instructions
lds r25, deadline + 1
```

Three things in that are worth stating outright.

A C global is a data space address, and the linker supplies it. `ticks` is a name for a byte in SRAM, somewhere above 0x0100, chosen when the program was linked. That is why it is `lds` and `sts` and never `in` and `out`: `in` and `out` reach the first 64 I/O registers and nothing else, and no C variable ever lives there (§1.1.5). The mistake reads well and assembles cleanly, and the linker only warns: it cuts the address down to the six bits `in` has room for and builds the program anyway, so `in r24, ticks` reads whichever I/O register that lands on and never your variable.

A 16-bit variable is two loads and the low byte is first. Which means the same tearing Chapter 3 warned about (§3.2.4), on the same terms: two instructions with a gap, and an interrupt that writes `deadline` can land in it.

A pointer costs one more load than you think. If C declares `struct thing* current`, then `current` is a *pointer variable*: two bytes in SRAM holding the address of the structure. Reading the structure's first field is two loads and then a third:

```
lds r30, current          ; Z = the pointer's value, not its address
lds r31, current + 1
ld r24, Z                 ; and now the field
```

Loading `Z` with the address of `current` and dereferencing that gives you the pointer's own bytes, interpreted as data, which is a plausible-looking small number rather than an error. It is the single most common way this boundary is got wrong, and the reason is that C hides the difference: `current->field` is one arrow and two dereferences.

There is no AVRASM2 half of this section, and that is the point rather than an omission. `avra` has no linker (§1.2.2), so there is nothing to supply an address that some other file chose; a name in a `.asm` file has to have been defined in the unit being assembled. Sharing a variable with C is a thing you can only do in a build that links, which is this chapter's build and no earlier one.

Where this goes next. A kernel is exactly the code that shares state with C rather than being called by it. Its context switch is assembly, its scheduler is C, and the two meet across a single `struct tcb*` that the C half assigns and the assembly half reads, dereferences and writes back: the three instructions above, in that order, plus the stack pointer of §4.3.5.

6.4 What to Build

6.4.1 The task

Two things, and the second of them is the book's capstone: `drivers/source/watchdog.asm`, which writes the watchdog's control byte correctly; and a C program that calls the assembly you wrote in Chapter 1, unchanged.

6.4.2 What to build in assembly: `drivers/source/watchdog.asm`

Four subroutines.

watchdog_init takes the control byte in `r24` and performs the timed sequence (§6.1.4):

1. Save `SREG`, then `cli`.
2. `wdr`, so the timeout starts from now rather than from wherever the counter had got to.
3. Clear `WDRF` in `MCUSR`, leaving the other three cause bits alone.
4. Write `WDCE` and `WDE` together into `WDTCSR`.
5. Write `r24` into `WDTCSR`.
6. Restore `SREG`.

watchdog_reset is a single `wdr` and a `ret`. It runs in your main loop, thousands of times, and must do exactly that one thing.

watchdog_disable is `watchdog_init` with a written value of zero.

watchdog_caused_reset returns 1 in `r24` if `WDRF` was set in `MCUSR`, else 0, and clears `WDRF` either way, leaving the other cause bits alone.

Details that are pinned rather than up to you

Steps 4 and 5 must be adjacent. Two `sts` instructions, nothing between them. Two cycles each, so they fit the four-cycle window with one cycle to spare; one single-cycle instruction between them still fits, and anything more leaves `WDTCSR` in system reset mode, holding `0x08` on a device fresh from reset (§6.1.4).

Save and restore `SREG`, do not just `cli` and `sei`. The sequence must run with interrupts off, but leaving them off on the way out is a bug: every other interrupt in the program quietly stops, at a moment determined by whoever last configured the watchdog. This is the same discipline as Chapter 3's atomic read (§3.2.4).

Clear `WDRF` before the window, not after. The hardware refuses to clear `WDE` while `WDRF` is set, so a driver that clears it afterwards works the first time and cannot disable the watchdog after a watchdog reset, which is exactly when it matters.

MCUSR is inside the I/O window, so in and out reach it; WDTCSR is not, and needs `lds` and `sts`. The two numbers are the usual pair: MCUSR is data space 0x54 and I/O 0x34, and the name in `m328Pdef.inc` is the I/O one, so in `r24`, MCUSR is what you write (§1.1.5). WDTCSR is 0x60 in both, because above the window there is only one. Two registers, two addressings, one subroutine.

Leave the other reset causes alone. MCUSR records four different reasons in four bits, and clearing all of them because you cared about one destroys information nothing can recover.

6.4.3 The capstone: call it from C

Write `drivers/app/main.c`, a C program that calls `shift_bits` from Chapter 1 and uses the result.

Change nothing in `utils.asm`. That is the point. It has followed the calling convention since Chapter 1, so a C prototype is the whole of the interface (§6.3.3).

Build it with a normal `avr-gcc` invocation rather than the book's usual one: a C program needs the startup code that sets the stack pointer, zeroes `.bss` and `r1`, and calls `main`, all the things your `main.asm` did by hand.

Then generate the compiler's own version of the same job with `-S`, and compare it against what you wrote (the cross-check, Exercise 6.8).

Details that are pinned rather than up to you

The prototype's types must match. `uint8_t shift_bits(uint8_t)`. Declaring the argument wider passes it in `r25:r24` and your subroutine reads only `r24`, which changes nothing you can see and is still wrong; declaring the result wider as well reads back an `r25` your subroutine never wrote (§6.3.3).

Do not add `extern "C"`. This is C, not C++, and there is no name mangling to suppress.

6.4.4 What none of this does

The driver does not choose the byte. It writes whatever `r24` holds. Working that byte out from §6.1.3 is your program's job, and so is deciding to write it at all.

The driver does not decide when to pet. Where `watchdog_reset` is called from is the entire difference between a watchdog that works and one that is decoration (§6.1.1), and nothing in the driver can help.

And none of it is tested against sleep. The interaction between sleep modes and wake sources is specified by the datasheet and approximated by the simulator, so this book teaches it and does not check it (§6.2.5).

6.5 Review

What you should be able to do now

- Say what a watchdog detects, and name three failures it does not.
- Write the timed sequence correctly, and say what the hardware does when you do not.
- Choose a timeout from a worst-case loop time, and say why the choice is not obvious in either direction.
- Explain why WDRF must be cleared before WDE can be, and what goes wrong on the second run of a program that forgets.
- Choose a sleep mode from what the program needs left running, and name the two wake sources that survive every mode.
- State the AVR C calling convention: argument registers, return registers, call-saved registers, and the register the compiler keeps at zero.
- Call an assembly subroutine from C, and a C function from assembly, and say what each side is promising the other.
- Read `avr-gcc -S` output and compare it against assembly you wrote for the same job.

Questions to test yourself

1. Your program deadlocks waiting for an interrupt that never comes. Does the watchdog help? What if instead it computes a wrong answer very quickly?
2. The two writes of the timed sequence must be within four cycles. Why is `cli` before them not optional?
3. You write the byte `0x08` into `WDTCSR` meaning “the four second timeout”. What did you actually ask for, and what happens next?
4. A program is reset by the watchdog, restarts, and tries to switch the watchdog off. Why might it fail, and what has to happen first?
5. You put the device into power-down and expect `Timer1` to wake it in a second. What actually happens?
6. You pass a 16-bit value to an assembly subroutine from C. Which registers does it arrive in, and which half is in the lower-numbered one?

After the book

This is the last chapter. What you have built is a driver library in assembly, and the habit that goes with it: **compute the number first, then measure it, and take the disagreement seriously.**

The companion repository has two written papers in exam/, if you want to test yourself on paper. They gate nothing and nothing in this book requires them.

Real-Time Kernel Design is the course that follows this one: the same part, the same harness, the same check by hand and by measurement, building a preemptive kernel one context switch at a time. It names this material a prerequisite and it means it. Chapter 3's ISR contract, Chapter 4's stack arithmetic and its loadable stack pointer (§4.3.5), Chapter 5's timer arithmetic, and this chapter's C ABI in both directions are used there rather than recapped. The half of a kernel that must be assembly is about two files out of a dozen, and knowing *which* two is most of what that course is for.

6.6 Exercises

How to check your work. Exercise 6.5 is checked by this chapter's test suite: write the file at the path the specification gives, then run `make test`. The suite is cumulative and runs Chapter 1 to Chapter 5's tests too. Exercise 6.6 is the C capstone, which no test covers: it needs the C runtime and is built by hand, with the command in §6.3.3.

The rest will have worked solutions in the companion repository, published later as `lectures/L06/appendix/f_solutions.md`, and the hand calculations have short answers in Appendix A. Work each one before you read it.

No hardware is needed for any of this.

Each exercise is labelled with its kind. There is exactly one **Cross-check**, and here it is Exercise 6.8. It is the last one in the book.

Exercise 6.1 What it catches

RECALL

- a) State, in one sentence, the only thing a watchdog detects.
- b) For each of the following, say whether a watchdog would catch it:
 - a loop that never exits;
 - a driver writing to the wrong port register;
 - the torn 16-bit read from Chapter 3;
 - a program waiting for an interrupt that will never arrive;
 - a stack that has grown into the variables.
- c) One of those five is “sometimes, eventually”. Which, and what has to happen first?
- d) Give the two modes a timeout can produce, and say which of them is a safety mechanism.

Exercise 6.2 Where to pet it

DESIGN

- a) A program has a main loop and a timer interrupt firing every millisecond. Somebody puts `watchdog_reset` in the timer handler, because it is called reliably. Explain precisely what that achieves and what it destroys.
- b) Where should it go instead, and what property does that location need?
- c) The main loop has two branches, one of which runs rarely. Where do you pet the watchdog so that a fault in the rare branch is still caught?
- d) A program pets the watchdog in three different places. Say what has probably gone wrong with its design.

Exercise 6.3 Control bytes

HAND CALCULATION

Using §6.1.3, and without running anything:

- a) Give the byte for a 1 second timeout in system reset mode.
- b) Give the byte for an 8 second timeout in interrupt mode.
- c) Give the byte for a 250 ms timeout in interrupt-and-reset mode.
- d) Somebody wants four seconds and writes `0x08`. Say exactly what they asked for instead, and describe what the device does from that moment on.
- e) Why is their mistake so much worse than, say, asking for 2 seconds and getting 250 ms?

Check yourself: the timeout table is in §6.1.3, and the four selector bits are not contiguous, so no control byte is ever the timeout number.

Exercise 6.4 The timed sequence

RECALL

- a) Give the two writes, in order, and the constraint between them.
- b) Four cycles is the limit. Roughly how many cycles would an interrupt take if one landed between the two writes? Say why `cli` is therefore not optional.
- c) On a device fresh from reset, what is in `WDTCR` if the second write arrives too late? Say why that particular value is the worst possible outcome.
- d) Why must `WDRF` be cleared before `WDE` can be? Give the reason the hardware behaves that way, and the consequence for a driver that forgets.
- e) The sequence begins with `wdr`. What would go wrong without it?

Exercise 6.5 The watchdog driver

CODE

Write `drivers/source/watchdog.asm` to the specification in §6.4.2.

- a) Write all four subroutines.
- b) Run `make test`.
- c) Insert four `nop` instructions between the two writes of the sequence, rebuild, and note what `WatchdogDriver.InitWritesTheByteAsked` reports. Say what that value means, then remove them.
- d) Replace the save-and-restore of `SREG` with a plain `cli` and `sei`. Note which test fails and describe the bug it is protecting you from.

Exercise 6.6 Calling it from C

CODE

Write `drivers/app/main.c` to the specification in §6.4.3.

- a) Write a C program that calls `shift_bits` and does something with the result. Change nothing in `utils.asm`; the `utils_gnu.S` you translate from it is a transcription, not a redesign.
- b) Translate `utils.asm` into `drivers/source/utils_gnu.S` using the table in §6.3.3, then build with `avr-gcc`, giving it both the `.c` and the `.S`. This build shares no flag with the book's usual one: say what `-mmcu=atmega328p` is deciding that `avra` decided some other way, and say what the startup code does that your `main.asm` used to do by hand.
- c) Declare the prototype's argument as `uint16_t` instead, rebuild, and say what now happens for an argument of 5 and for an argument of 300. Then declare the result as `uint16_t` as well and answer the same question. Then put both back.
- d) Write a C function and call it from your assembly. Say which registers you had to assume it would destroy.

Exercise 6.7 Sleeping

DESIGN

- a) For each of idle, power-down and power-save, say what is still running.
- b) A program sleeps in power-down and expects `Timer1` to wake it after one second. Describe what actually happens, and how long it lasts.
- c) Name the two wake sources that survive every sleep mode, and say why each of them does.
- d) Design, in words, a device that does something once every eight seconds and spends the rest of its life drawing microamps. Say which peripheral wakes it and in which mode.
- e) A device refuses to stay asleep: it wakes immediately, every time. Give two plausible causes.

Exercise 6.8 Your assembly against the compiler's**CROSS-CHECK***Compute it by hand, measure it, reconcile.*

The last cross-check in the book, and the only one where the thing you are checking against is somebody else's work.

- a) **By hand.** From Chapter 1, your `shift_bits` costs $6n + 10$ cycles (§1.3.4). Write that down.
- b) **Write the same thing in C.** A function taking a byte, returning 1 shifted left that many times, using a loop. **Call it `c_shift_bits`**, so that both live in one ELF without colliding. Compile it at `-Os` and read the assembly with `avr-gcc -S`. Count the instructions and predict its cost as a formula in n , before measuring.
- c) **By measurement.** `drivers/build/drivers.hex` holds no C, so build one image that holds both, the same hand build the capstone uses, and point `make measure` at it:

```
avr-gcc -mmcu=atmega328p -Os \
    drivers/app/main.c drivers/source/utils_gnu.S \
    -o drivers/build/mixed.elf
make measure IMAGE=mixed \
    CALLS="shift_bits:0 c_shift_bits:0 shift_bits:8 c_shift_bits:8"
```

`IMAGE` names the stem and not the extension, so the same variable reaches `drivers.hex` and this `mixed.elf`; `ci/measure.sh` takes whichever of the two exists.

- d) **Reconcile.** Give both formulas. They are not the same shape, and one of them wins at some arguments and loses at others: say where they cross.
- e) **The surprise.** At one specific argument the compiler's version is **faster** than yours. Which argument, by how much, and what did the compiler do that you did not? Look at the instruction it used and say what it does.
- f) **Code size.** Count the instructions in each. Now say which version you would ship, and note that "the hand-written one is smaller and faster" is not the answer the numbers support.
- g) **What this is evidence for.** In one paragraph, say what this comparison does and does not establish about writing assembly by hand. Be specific about the size of the win and about what it cost you to get it.

Exercise 6.9 What you would actually reach for**DESIGN**

The book is over. For each of the following, say whether you would write assembly, write C, or do something else entirely, and why.

- a) A driver for a new sensor on the SPI bus.
- b) An interrupt handler that must respond within two microseconds.

- c) A routine that toggles a pin for a fixed 3 microseconds, exactly, every time.
- d) The main loop of a battery-powered logger.
- e) Working out why a program that worked yesterday now resets every few seconds.
- f) One of the five answers above is the one this book was really for. Say which.

Short Answers

The numbers only, for the hand-calculation exercises, so that an answer worked on paper can be checked on paper. How each one is reached, and the plausible wrong answers, will be in the worked solutions, which are published later in the companion repository.

Exercise 1.2 Assembling four instructions by hand

a) 0xE20A b) 0xE28A c) 0xEFFF d) 0xE043 e) Rejected: ldi can only name r16 to r31. A word forced out anyway would load r31.

Exercise 1.7 Counting by hand

a) 10 cycles b) 52 cycles c) $6n + 10$ d) cp 4 times, breq 4 times, 1 of them taken e) $7n + 10$: the extra inc on each trip is the extra cycle per bit.

Exercise 2.3 Translating pin numbers

a) Pin	Port	Bit	Mask	PINx	DDRx	PORTx
3	D	3	0x08	0x0029	0x002A	0x002B
8	B	0	0x01	0x0023	0x0024	0x0025
13	B	5	0x20	0x0023	0x0024	0x0025

b) 0x01; without the subtraction 0x00, and the LED never lights c) PB6 and PB7, the crystal's pins.

Exercise 2.7 Predicting the driver's cost

a) 30 cycles more b) 35 c) 65 d) led_toggle: one st after the shift, with nothing read e) Only the shift loop's trip count depends on the pin.

Exercise 3.2 Addresses and response times

a) Vector	Number	Word	Byte
PCINT0	3	0x0006	0x000C
PCINT2	5	0x000A	0x0014
WDT	6	0x000C	0x0018

b) 6 cycles, $4 + 2$ c) $9, 3 + 4 + 2$ d) 7 and 10; jmp reaches all of flash and rjmp only $\pm 2K$ words e) $110, 4 + 2 + 104$.

Exercise 3.7 The shared counter

a) $0x01FF$, which is 511; the counter held 255, then 256 b) 2 cycles; about one read in eight thousand, so a wrong value every eight seconds or so c) `cli` before and `sei` after, 2 cycles d) It enables interrupts whatever they were before; save `SREG`, `cli`, load, restore `SREG`: 3 cycles e) No: one `lds` is one instruction.

Exercise 4.2 Four ways to read a byte

a) $0x33$, `Z` stays $0x0204$ b) $0x33$, `Z` becomes $0x0205$ c) $0x22$, `Z` becomes $0x0203$ d) $0x66$, `Z` stays $0x0204$ e) (a) and (d) f) Two cycles each, so the choice is only about where the pointer is left.

Exercise 4.4 Array arithmetic

a) $0x0300$, $0x0307$, $0x030E$, $0x0323$ b) 7, `LED_SIZE` in `led.inc` c) $0x030C$, the sixth byte of entry 1, so the driver reads $0x0100$ ($0x0n00$ for bit n) as a port register d) Every address is legal, and entry 0 is at the base under either stride e) $0x031E$: the stride is 10 f) 292 LEDs, 204 buttons; the real limit is the 14 pins.

Exercise 4.7 How deep does it go

a) 4 bytes b) 15, $0 + 2 + 9 + 2 \times 2$ c) `SP` ends at $0x08F0$; the lowest byte used is $0x08F1$ d) 2 bytes, the program counter e) 21, $2 \times 3 + 2 + 9 + 2 \times 2$ f) No: it occupies $0x08F0$ to $0x08F6$, and the stack reaches down to $0x08F1$.

Exercise 5.2 Prescaler and compare

a-c) Wanted	Timer1, 16-bit	Timer0, 8-bit
500 Hz	/1, <code>OCR</code> = 31999, exact	/256, <code>OCR</code> = 124, exact
440 Hz	/1, <code>OCR</code> = 36363, -0.0010%	/256, <code>OCR</code> = 141, $+0.032\%$
3 kHz	/1, <code>OCR</code> = 5332, $+0.0063\%$	/64, <code>OCR</code> = 82, $+0.40\%$

d) 500 Hz: 32000 ticks divide the clock exactly, and the prescalers are powers of two e) 3 kHz, 64 times worse: 83 ticks against 5333.

Exercise 5.7 What cannot be reached

a) $f_{clk} / (1024 \times \text{counter values})$: 0.2384 Hz for 16 bits, 61.04 Hz for 8 b) Yes: `/1024`, `OCR1A` = 31249, exact c) No: 156250 ticks, and the counter holds 65536 d) The counter runs out of values: a full 8-bit count at `/1024` spans only 1/61 of a second e) A 1 kHz interrupt and a software target of 10000 f) 16 MHz, one tick at a prescaler of 1; nothing is shorter than one tick.

Exercise 6.3 Control bytes

a) $0x0E$, selector 6 plus `WDE` b) $0x61$, selector 9 as $0x21$ plus `WDIE` c) $0x4C$, selector 4 plus both d) System reset at 16 ms: the device resets, starts up, and resets again, for ever e) A short timeout is a wrong setting; $0x08$ is a device that no longer runs.